3A35 Computer Architecture

8 Lectures
Michaelmas 2000
Rev 16/9/00

Prof D W Murray
david.murray@eng

Overview

In Year 1 you learned how to use the basic logic elements to build up **combinational** logic circuits, some involving the larger more complex, combinational elements such as PLAs and ROMs. You also learned how to design **sequential** logic, designing simple machines that have a number of identifiable states either using latches, or using ROMs. You were introduced to a method of defining hardware called Register Transfer Language.

In these lectures, we are going to build on these ideas to understand the hardware architecture of the most sophisticated sequential machine, the general purpose computer. Our emphasis will be on underlying principles, rather than specific designs. As with the architecture of buildings, the detailed shape and plumbing of different computers will vary, with the design engineer choosing to add a circuit here and remove a circuit there that will give his design optimal performance/cost for certain tasks. But the underlying building regulations are still obeyed.

The variety of detail in CPUs can cause confusion — each book on computer architecture seems to present a slightly different diagram. From Lecture 2 onwards we will try to establish a "Bog Standard" cpu hardware that will allow you to understand where and why others differ. Much of the operational description in the lecture will be in terms of RTL, but we will also introduce Assembler Language as a way of describing several RTL statements, thus avoiding too much detail. Further we shall look at how certain aspects of high-level languages get compiled into assembler — the important thing there is to see how the hardware is made to support programmer preferences. We will also want to look at how input and output to external devices is managed both in hardware and software.

This course describes a much simplified Complicated Instruction Set Computer (CISC) architecture. However, by stressing principles rather than a particular instance of a CISC machine, you should find it straightforward to the changes incorporated in the Reduced Instruction Set Computer (RISC) machine. The recommended text by Clements includes an introduction to RISC if you are interested. However, I suggest you read this only when you are completely happy with CISC.

Lecture Content

Lecture 1 revises and refreshes ideas taught in the 1st year Digital Logic course. Lectures 2–5 deal with the basic operation of the core of the computer. Lectures 6–8 extend the picture to include the hardware and software functionality of the typical machine.

0

1. The Design of State Machines and Register Transfer Language.
2. Overall structure of CPU and Memory. Instruction Fetching.
3. Control 1: The random logic controller.
4. ALU and Memory hardware. The Status word.
5. Control 2: Microprogrammed controller
6. The macro level. Memory addressing and stacks.
7. Programmed and interrupt-driven I/O.
8. Memory I/O and Hierarchies. Summary.

Essential Texts

Hill and Peterson, Digital Logic and Microprocessors, Wiley and Sons Inc. (1984). ISBN 0-471-82979-X. Chapters 9 and 10 provide essential reading about RTL. The earlier Chapters will provide useful revision on logic circuits. My advice is DO NOT read Chapter 11 on small computer organization!

Clements, The Principles of Computer Hardware 2nd ed, Oxford (1990), ISBN 0-19-853764-6. The first three chapters cover basic combinational and sequential logic and Chapter 4 has a good discussion of computer arithmetic. But the book really comes into its own in Chapter 5 with an excellent account of the cpu, and later chapters cover memory, alu and i/o in detail, along with descriptions of modern hardware. Some of the sections and examples are specific to a realistically complicated architecture, the Motorola 68000, but the author is careful to separate the principles from the detail. So despite its age, this book is still a goer. There is a useful introductory chapter on RISC machines — though unfortunately we no longer have time for RISC in this course.

Other Newer Texts

One book that used to be used on this course is **Tanenbaum, Structured Computer Organization**, a useful text as it really stressed hierarchical design. This is still true of the latest **4th edition, ISBN 0-13-020435-8**, but to update the book the author has introduced Java and the Java virtual machine, and uses the Pentium II in examples. It is a valuable approach, and if we had another 8 lectures ...

Lecture 1
State machines, Data versus Control, and RTL

1.1 State machines using flip-flops: revision

Problem. Design using JK flip flops a modulo-4 counter that if x=0 would count repeatedly from 0,1,2,3,... and if x=1 would count 3,2,1,0,...

Solution. If we designate the output bits as the outputs Q_0 and Q_1 of the JKs, then there are 4 states, for which we require 2 flip-flops. The truth table and transition list of the JK are

J	K	Q^{k+1}
0	0	Q^k
0	1	0
1	0	1
1	1	$\overline{Q^k}$

J	K	$Q^k \to Q^{k+1}$
0	X	$0 \to 0$
1	X	$0 \to 1$
X	0	$1 \to 1$
X	1	$1 \to 0$

and so (for the reader to complete):

x=0					
O_1	O_0	J_1	K_1	J_0	K_0
0	0	0			
0	1	1			
1	0	X			
1	1	X			

x=1					
O_1	O_0	J_1	K_1	J_0	K_0
0	0	1			
1	1	X			
1	0	X			
0	1	0			

Thus J_1 is a function of O_1, O_0 and x, and its Karnaugh map is

	x	0	1
O_1	O_0		
0	0	0	1
0	1	1	0
1	1	X	X
1	0	X	X

whence $J_1 = O_1.\overline{x} + \overline{O_0}.x$

and similarly for K_1, J_0, and K_0.

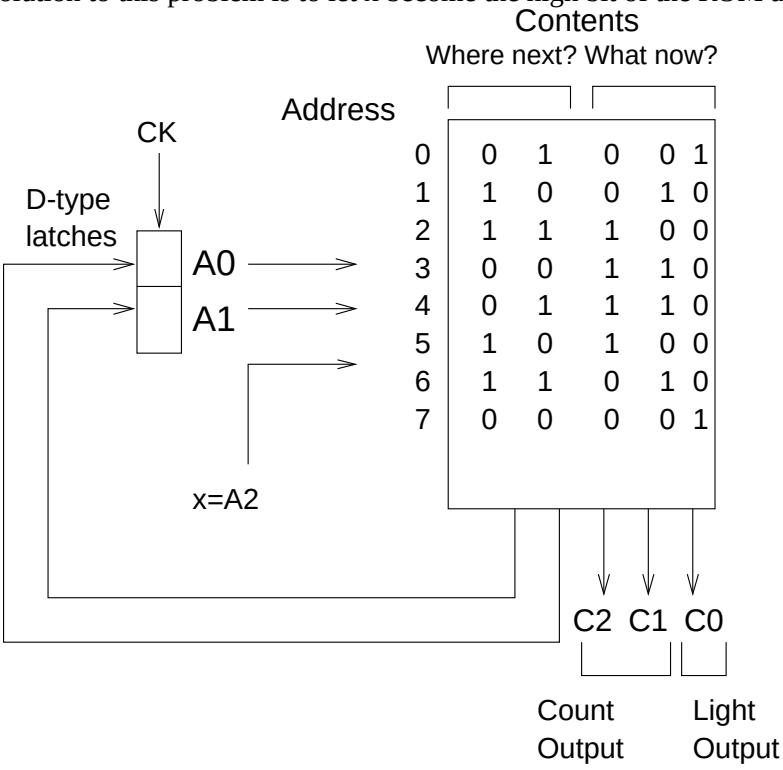
1.2 State machines using ROMs

You also saw in Year 1 that an easy method of building a sequence state machine was to use a ROM, where:

- one part of the contents at a particular address tells you **what to do now**, and
- the other part tells you the address of **where to go next**

Problem. Design using a ROM a modulo-4 counter that if x=0 would count repeatedly from 0,1,2,3,... and if x=1 would count 3,2,1,0,... Whenever the output is 0, a light should light.

One Solution. One solution to this problem is to let x become the high bit of the ROM address, as in



It is tempting to ask whether there is not a more efficient solution. For one example, here the light output could be made by ANDing the inverses of C_2 and C_1. For another, by reorganizing the "where next" bits of addresses 4 to 7, we could make the content bits C_2 and C_1 the same as A_1 and A_0, and therefore redundant.

Such questions miss the main points of using a ROM, which are

- to provide a solution with minimal thought and
- to provide a solution which is alterable merely by re-programming the ROM.

1.3 Algorithmic State Machine Charts

The two methods of designing sequential circuits are specific hardware solutions, but they do not address a particular area of the design problem, the translation of the *written description* of what a machine should do into a *formal description* of the state machine.

One way of doing this is to use an ASM chart, similar to flow diagrams used in software design. There are three symbols in such diagrams:

- **State definition (rectangles):** There is one per state. It gives the name of the state, the binary flip-flop values that define the state, and a list of outputs.
- **Decision (diamonds):** There are any number of these for each state. Each senses an input condition or flag, and takes a binary decision.
- **Conditional outputs (curvy boxes):** These occur at the exit lines of decision diamonds, and describe outputs that become true only when a condition is satisfied. Note that these outputs cannot "change" the outputs given in the state box. If you need to do that, you need an extra state.

The idea is best understood by example, and we use the vending machine example from Hill and Peterson (pp204-). This does not use conditional outputs — see Hill and Peterson for further examples.

1.3.1 Vending machine example

Design problem. A vending machine holds items which cost 20 cents, and accepts 5 cent (nickel), 10 cent (dime) and 25 cent (quarter) coins. Sensors in the coin shute cause two flags to be set as follows

Coin	I1	I2
No Coin	0	0
25	0	1
5	1	0
10	1	1

When coins to the value of 20 cents are deposited, the activate signal A is set to 1 for one clock cycle. If a 25 cent coin is deposited, a change signal C is set to one. Flag R is set when the user wishes to have all money returned.

Example Solution. A possible solution is shown in Figure 1.1. Any press of the Return button and every completed transaction puts the system in the Q0 state. From there one enters Q25 if a 25 cent coin is deposited, Q10 if a 10 cent coin and Q5 if a 5 cent. Notice that in this solution if in Q5, Q10 or Q15 when a 25 coin is deposited, the coin is ignored. However, also notice that in the solution given when the machine is in state Q15, *any* coin will be accepted and no change given.

There are 6 states, so at least 3 flip-flops are required. There is no method to determine how to assign the configuration of flip-flop outputs to the particular states, but the choice will alter the complexity of the resulting circuit. Thus, the choice of Q0=000 and Q5=100 etc is not arbitrary, but nonetheless there are no formal rules to obtain the optimal assignment.

1.4 Separating Data and Control

There are more economical graphical methods that can be used to represent state machines. However, whatever graphical method is used there is an obvious difficulty using a method which represents every state. Given n flip-flops we require 2^n states. While this number is manageable for a few flip-flops, the method becomes unworkable for a machine with several tens, let alone a computer with a 16MB memory!

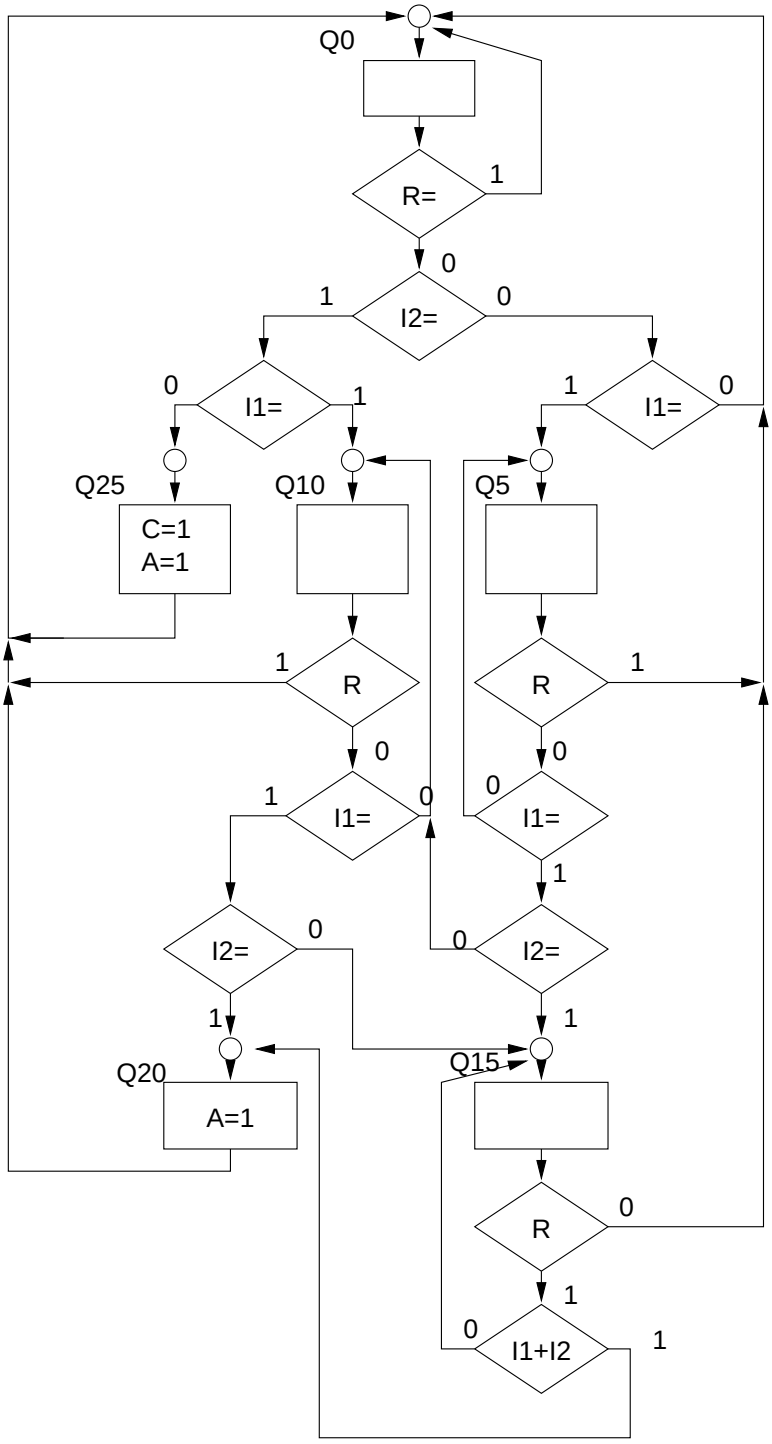


Figure 1.1: The ASM for the vending machine. From Hill and Peterson Figure 9.7. The state of the three registers for each of the ASM states needs adding!

The large numbers of flip-flops are often storing *data* upon which the machine operates in largely the same way irrespective of what the data is exactly. Our understanding then of how the machine functions depends on those control operations, not on the particular data. It makes sense then to separate the representation of control from that of data.

Of course the control section will need at times to know something about the data — for example, to take a different action if a datum is zero rather than positive — so there will be two way communication between the control and data sections, but only decision flags about a datum, not the datum itself need be communicated.

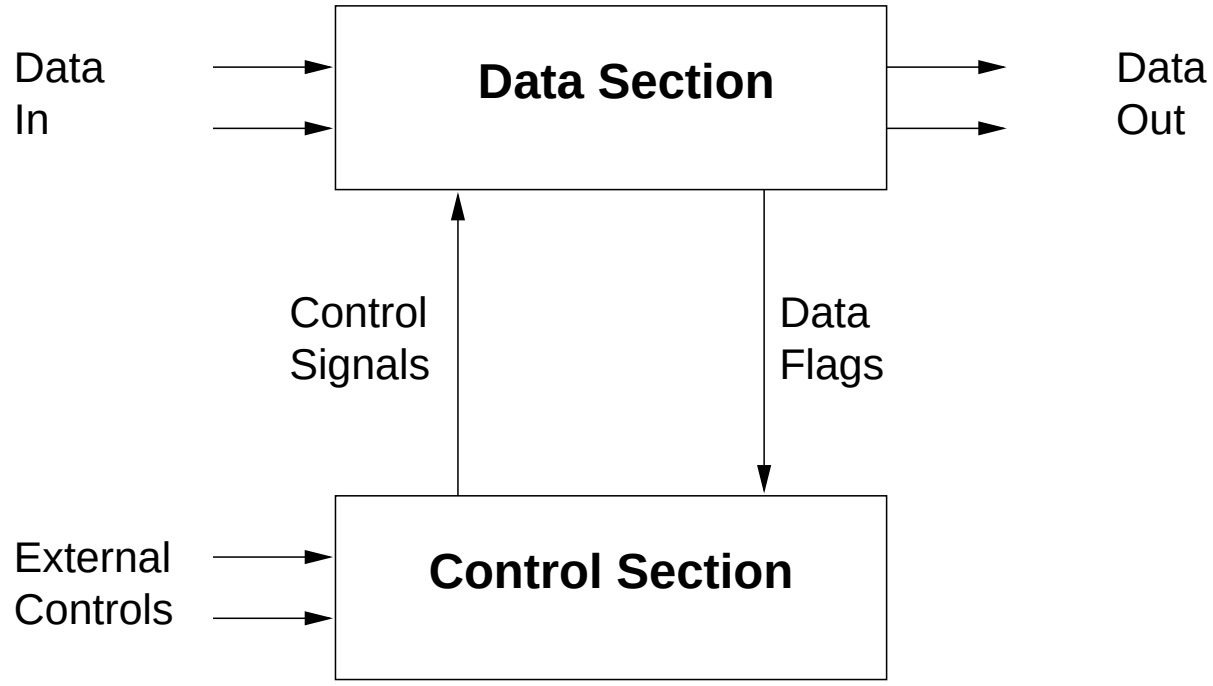


Figure 1.2: Data separated from control

It is of course possible to describe the control section of such a machine using ASM charts, but the language does not have the syntax to deal with data elements.

Instead we can and will use a hardware description language which separates data from control. It is called **Register Transfer Language**.

1.5 Register Transfer Language

Why Register Transfer? The transfer of data between two storage registers is the principal activity that occurs in the data section of the machine, and as we will see in Lecture 2 is the bread and butter operation in computers.

A register is a D-type latch which transfers input to output on a receipt of a clock pulse:

$$Q^{t+1} = D^t \quad \text{or} \quad Q^{t+1} \leftarrow D^t$$

In the figure by clocking register b, the transfer $b \leftarrow a$ is effected. Most often the register has several bits. In the right hand example $B \leftarrow A$ means that $B[i] \leftarrow A[i]$ for $i = 0, 1, 2, 3$. **Note carefully** that the **receiving register is clocked, NOT the transmitting register.**

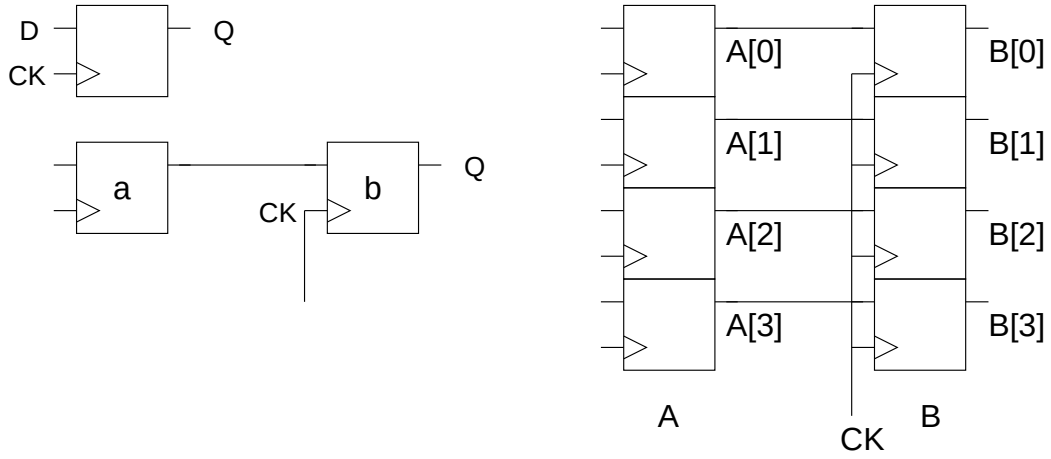


Figure 1.3:

The syntax is best introduced by example. This is similar to an example in Hill and Peterson, but with a few things added.

MODULE: DATAMOVER
MEMORY: $A[2]$; $B[2]$; $C[2]$; S
INPUTS: $X[2]$.
OUTPUTS: $Z[2]$; P.

1 $A \leftarrow X$
2 $C \leftarrow \overline{A}$; $S \leftarrow A[0]$
3 $B \leftarrow C$
4 $C \leftarrow A \vee B$
5 $Z = C$; $S \leftarrow 0$; $\rightarrow 1$

ENDSEQUENCE
ControlReset(1); $P = S$.
END

The first block defines the storage required in the data section. A, B, C are two bit registers, but S has just a single bit. X and Z are two bit input and output wires, NOT registers. At the end, the ControlReset(1) indicates that a reset pulse should set the control section to line 1. $P = S$ indicates that an output wire (NOT register) P

is permanently soldered to the output of register S. (If S had been a multi-bit register, P would have the same number of wires.)

On line 1, inputs X is transferred to A, so we need to clock register A. Two things happen on line 2: the inverse of A is transferred to register C, and S is set to $A[0]$, and so on for lines 3 and 4. The output Z is conected to C throughout line 5. At the end of line five, there is a goto arrow, which sets the next line as 1.

1.5.1 Control section: generating CSL and CSP signals

To realize the control section, you must count the number of lines of RTL, and study the flow from one line to another. In our example flows uninterrupted from line 1 to 5, where there is an "unconditional goto" $\rightarrow(1)$. (We shall see a conditional goto later on.)

The realization of the Control Secton is shown in Figure 1.4. The latch corresponding to the active line has output 1 — the remainder have output 0. Notice how the $\rightarrow(1)$ is achieved by looping the output of latch 5 into the input of latch 1. Notice too that the reset line sets flip-flop 1 high, and the rest low.

The level output from a latch supplies the CSL signal, so CSL1 is high during line 1, CSL2 is high during line 2, and so on. Transition from one line to the next occurs on the falling edge of a clock pulse. The clock is ANDed with the CSL to provide a pulse CSP which is used to fire the register transfers and the transition to the next line of RTL. So CSP1 only occurs at the end of line 1 of RTL.

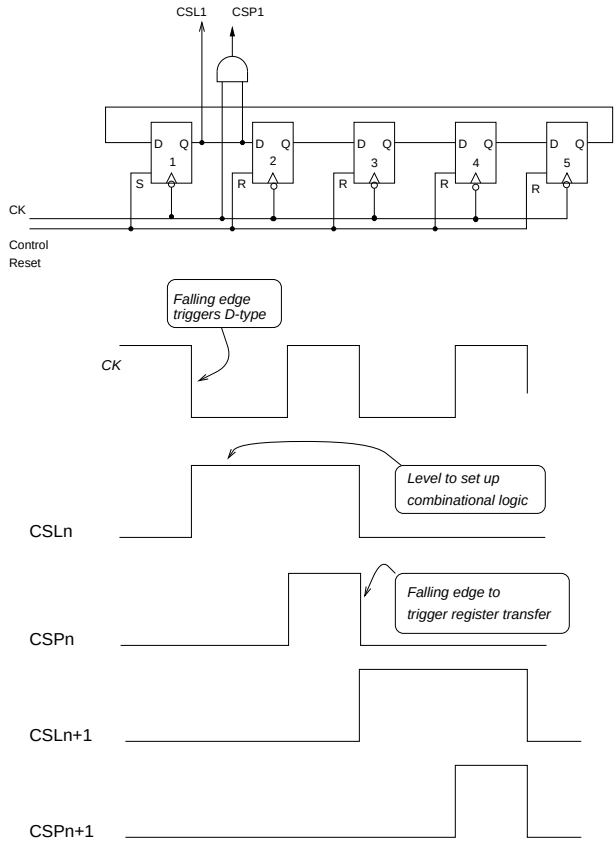


Figure 1.4: CSL and CSP signals from the control section.

1.5.2 Data section: using the CSL and CSP signals

The register hardware required for the data section is defined in the outer blocks of the module. Line 1 of sequence indicates that inputs X are connected to the input of register A, which is clocked by CSP1.

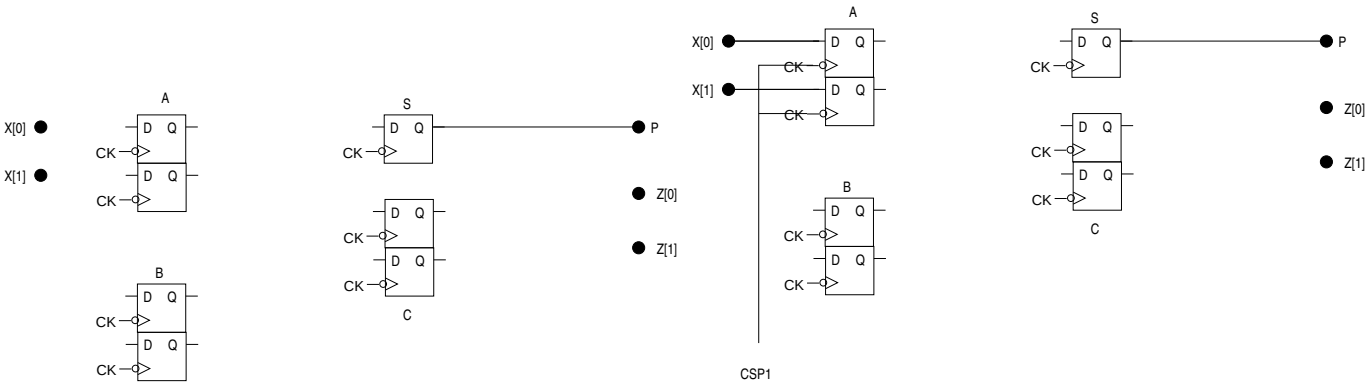


Figure 1.5: Steps 0 and 1 in the Data Section

Line 2 indicates that the inverted outputs of A are to be connected to C and that the output $A[0]$ should be connected to the input of S. CSP2 should be attached to the CK input of C and S. The extra connections for Line 3 should be obvious.

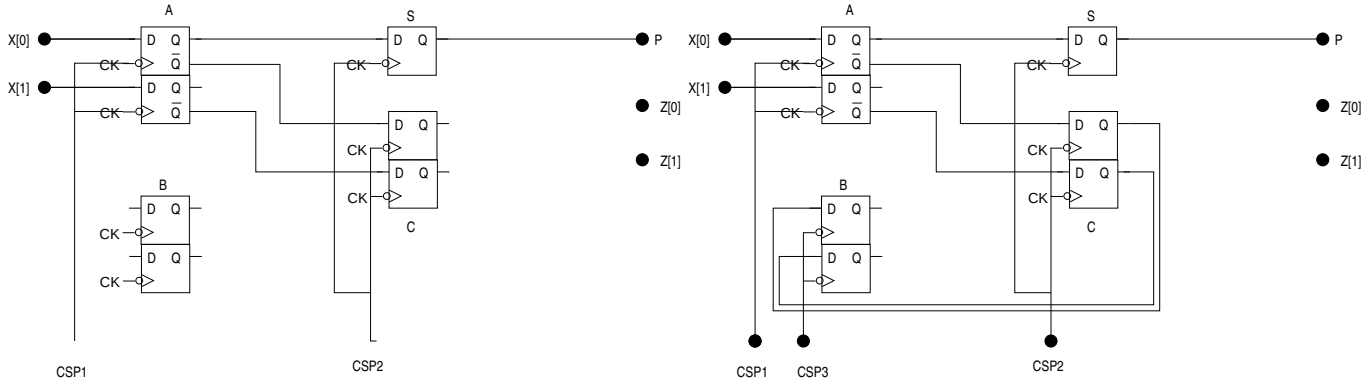


Figure 1.6: Steps 2 and 3 in the Data Section

At Line 4, our design begins to need some modification. Notice that C is clocked on line 4 as well as line 2, so C's clock input must be CSP2$\vee$CPS4. But on line 2, the inputs to C are connected to $\overline{A}$ and on line 4 they are connected to $A \vee B$. We need to insert AND gates, ANDing with CSL2 and CSL4 respectively, and then OR the inputs into C.

At line 5, $Z = C$ means that outputs Z should be connected to the output of C for the entire line — so we must AND the outputs of C with CSP5. In addition, we have to make alterations to the input of S and to its clock input. (See Question sheet.)

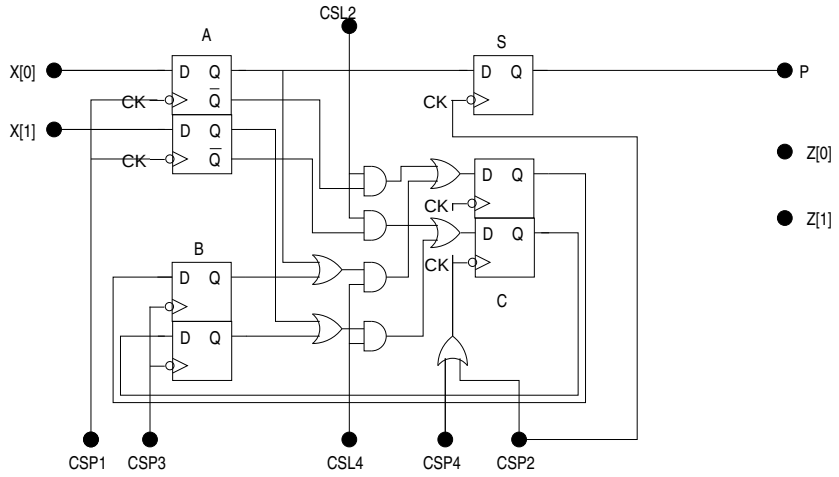


Figure 1.7: Step 4 in the Data Section

1.5.3 Using data to control flow

Let us alter the RTL somewhat:

MODULE: DATAMOVER
MEMORY: $A[2]$; $B[2]$; $C[2]$; S
INPUTS: $X[2]$.
OUTPUTS: $Z[2]$; P.

1 $A \leftarrow X$
2 $C \leftarrow \overline{A}$; $S \leftarrow A[0]$
3 $B \leftarrow C$
4 $C \leftarrow A \vee B$; $\rightarrow (A[1], \overline{A[1]})/(5, 6)$
5 $Z = C$; $S \leftarrow 1$; $\rightarrow 1$
6 $Z = \overline{C}$; $S \leftarrow 0$; $\rightarrow 1$

ENDSEQUENCE
ControlReset(1); $P = S$.
END

The construction $\rightarrow (l_1, l_2, ..., l_n)/(S_1, S_2, ..., S_n)$ evaluates logical expressions l_1, l_2, ... in turn. The first found to equal 1 causes a jump to the corresponding statement S. So, if at line 4 $A[1]$ is 1 the program will goto line 5, if $A[1]$ is 0, it will goto line 6.

We need to take a connection from $A[1]$ into the control unit. The controller must have 6 latches. How is $A[1]$ used to control the flow? Figure 1.8 shows the arrangement. $A[1]$ provides the selection line in a multiplexer. During Line 4 CSL4 is high, and if $A[1]$ is high the input to latch 5 is high, but if $A[1]$ is low the input to latch 6 is high. After the next clock pulse, CSL5 or CSL6 respectively will go high. Note that because there are multiple ways of arriving back at Line 1, the input to latch 1 requires an OR gate.

There are obviously some alterations required to the Data Section for the new line 6. You are asked to do these in the Problem Sheets.

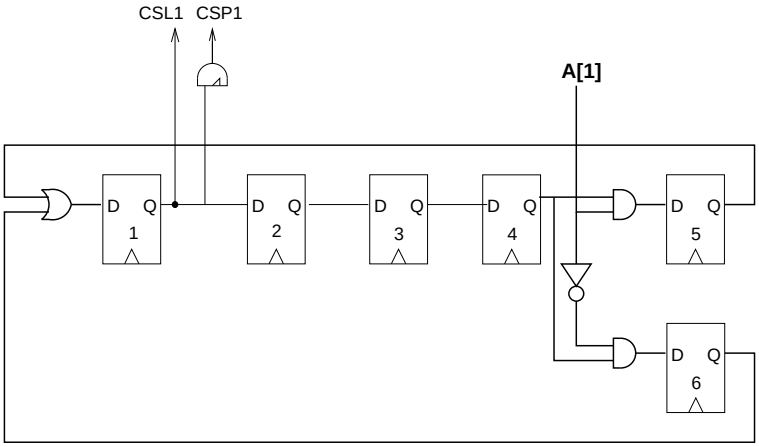


Figure 1.8: Additions to the Control Section required for conditional goto.

1.6 Timing

Let us consider the various register outputs as a function of time for particular inputs X.

1	$A \leftarrow X$
2	$C \leftarrow \overline{A}$; $S \leftarrow A[0]$
3	$B \leftarrow C$
4	$C \leftarrow A \vee B$;
	$\rightarrow (A[1], \overline{A[1]})/(5, 6)$
5	$Z = C$; $S \leftarrow 1$; $\rightarrow 1$
6	$Z = \overline{C}$; $S \leftarrow 0$; $\rightarrow 1$

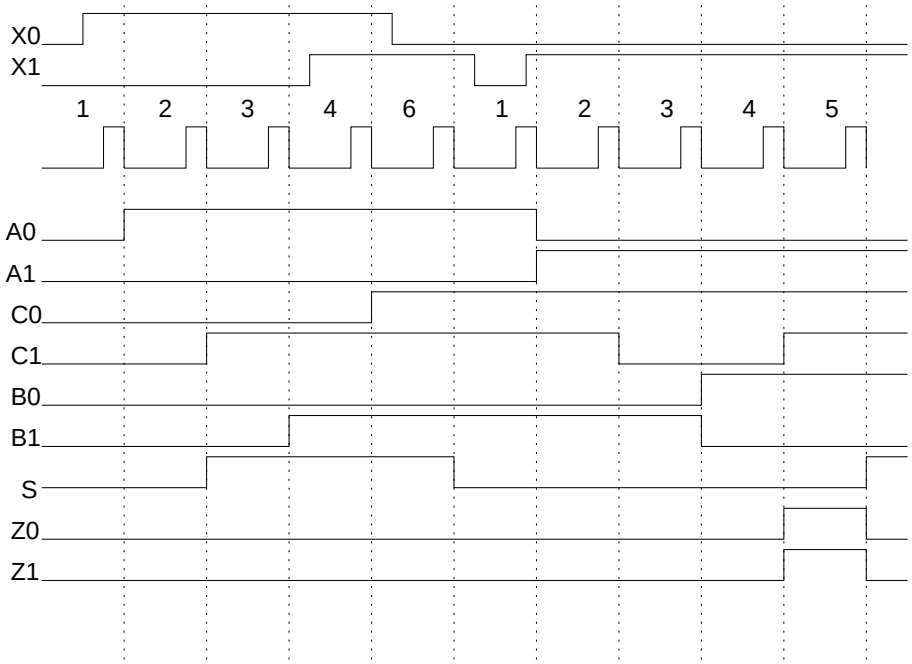


Figure 1.9: Examples of register outputs for different inputs X. Note that X can change asynchronously.

Lecture 2
CPU, Memory and Information Flows

2.1 Introduction

Before getting embroiled with the hardware of computation, it is worth thinking about what a computation is exactly. A computation is simply the evaluation of some output O, given some input I. O and I are a collections of symbols of some sort and in a binary computer we would think of them as strings of 0's and 1's Denoting a computation f as $O = f(I)$, it can often be broken down into elemental steps $O_1 = f_1(I)$, $O_2 = f_2(O_1)$, ..., $O = f_n(O_{n-1})$, where, note, the f_i need not necessarily be different. (Note too that it follows that the division of computation into hardware and software is arbitrary.)

Considerable research into the fundamentals of computation was carried out by logicians in the 1920's and 30's — Gödel, Church and Turing are leading names. Two of the premises of their work was that a reasonable computing machine (i) should not "merely" store the answers to all possible problems; and (ii) should not compute at infinite speed. We can all agree that these are reasonable, but do they restrict what can be computed? Gödel showed that there were very reasonable questions to which the reasonable computer could not give answers. (Note that this is nothing to do with the argument over whether computers can be intelligent.) In 1936 Turing a model of a computer that satisfied the criteria of reasonableness and which he proved could compute anything that was computable.

The Turing machine must by definition form the basis of any architecture, so it is worthwhile examining. It also forms a good introduction to the von Neumann architecture that we shall be using throughout the lectures. (It is worth being careful though. Although we shall describe Turing's machine as a physical device, for Turing it was the abstraction that was important.)

The machine comprises a sectioned tape and a processor which can move the tape right or left, and can read and write from a finite set of symbols σ off and onto the tape. The processor itself has a finite number of states s. With the processor in state s_a, and the symbol σ_b under the read/write head, the processor takes one of a finite number of actions:

- f_1: write a symbol σ_c onto the tape.
- f_2: move the tape left one section
- f_3: move the tape right one section
- f_4: halt

and then sets itself into a state s_d. (Note s_a and s_d are not necessarily different. σ_b and σ_c are not necessarily different, but a null action is a wasted action.)

As an example consider unary addition. Any positive number n can be represented by string of n □'s. Suppose the tape is loaded with the numbers 5 and 4 surrounded by blank sections, and the read/write head is positioned over the leftmost blank. Figure 2.2 shows the tape, and a sequence of computations that result in the sum being written on the tape.

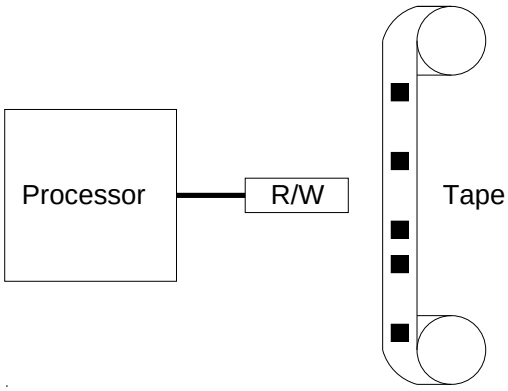


Figure 2.1: A Turing machine.

BEFORE: ○□□□□□ ○ □□□□□○

State	Tape	f	New state	Comment
s_0	○	R	s_0	Moving right from blank
s_0	□	R	s_1	Moving right from □
s_1	□	R	s_1	..
s_1	○	□	s_2	Replace blank in middle
s_2	□	L	s_2	Moving left
s_2	□	L	s_2	..
s_2	○	R	s_3	Gone too far
s_3	□	○	s_4	Delete leftmost □
s_4	○	H	s_0	Halt

AFTER: ○□□□□□□□□□○

Figure 2.2: Unary addition using a Turing machine.

Over and above its simplicity, there are several things noteworthy about the machine:

- the notion of storage
- the notion of processes
- the processor itself is a finite state machine — it has storage on board. In the example the FSM has 6 states.
- that data is separated from processes.
- that computation takes place with symbols. Computers don't have to be binary.

2.2 Separation of control and data

Through these lectures we will be concerned with a physical architecture of a digital binary computer with a processor and remote memory, the design of which is usually attributed to the American, John von Neumann. (In reality the attribution should be shared amongst several.) The picture already developed for the Turing machine is easily transmuted to look like the von Neumann machine. Note that the processor has a few storage Registers on-board, an Arithmetic Logic Unit (ALU) responsible for carrying out binary operations of addition, OR-ing etc, and a control unit (CU). Already we see control separated from data. The tape of the Turing machine is replaced

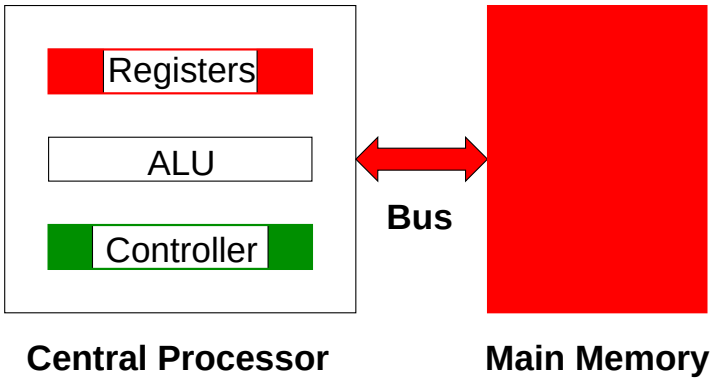


Figure 2.3: The von Neumann architecture

by a main Memory which is physically separate from the processor unit and which communicates over a bus. The main memory, note, also comprises part of the data section — it is, after all, just a load of registers.

The idea is that information runs from the memory to the processor on a bus, is stored temporarily, operated on by the ALU, and then sent back to memory. It is worth noting that over the years as memories became larger and processors became faster, the bus had to carry an increasingly large amount of traffic. The speeds of memory and particularly buses has not increased so fast (indeed there are obvious limits on bus speed — what are these?) and this led to "the von Neumann bottleneck", the processor being starved of information. Much ingenuity has been spent mitigating this problem in serial machines, but more radical solutions are seen in "parallel" architectures which have multiple processors and memory distributed between the processors. We will mention such architectures briefly at the end of the lectures.

2.2.1 Program is Data.

While stressing the split of control and data, you might have wondered where the notion of program fits in. On the one hand our computer has to be general purpose, but electronic components stuck on a board and wired together have, by their nature, to do something specific. The trick is to make the specific electronic part run a bland loop of RTL in which it fetches an *instruction* from memory, then *executes* it by configuring the registers and alu on the cpu in the desired way, then fetches the next instruction, executes that and so on. This set of instructions is the program, and so in our split of control and data is strictly data, but data which is pulled into the controller to affect its operation.

2.3 A Bog Standard Architecture

Rather than introduce each part and then stick the parts together, we will immediately dive in at the deep end by fleshing out the skeleton into our Bog Standard Architecture.

The cpu contains a number of registers (small units of storage), a controller, an arithmetic logic unit, and is connected to the memory (a large unit of storage) by a bus. Inside the cpu there are internal buses or data pathways which allow the output of one register to connect to the input of another.

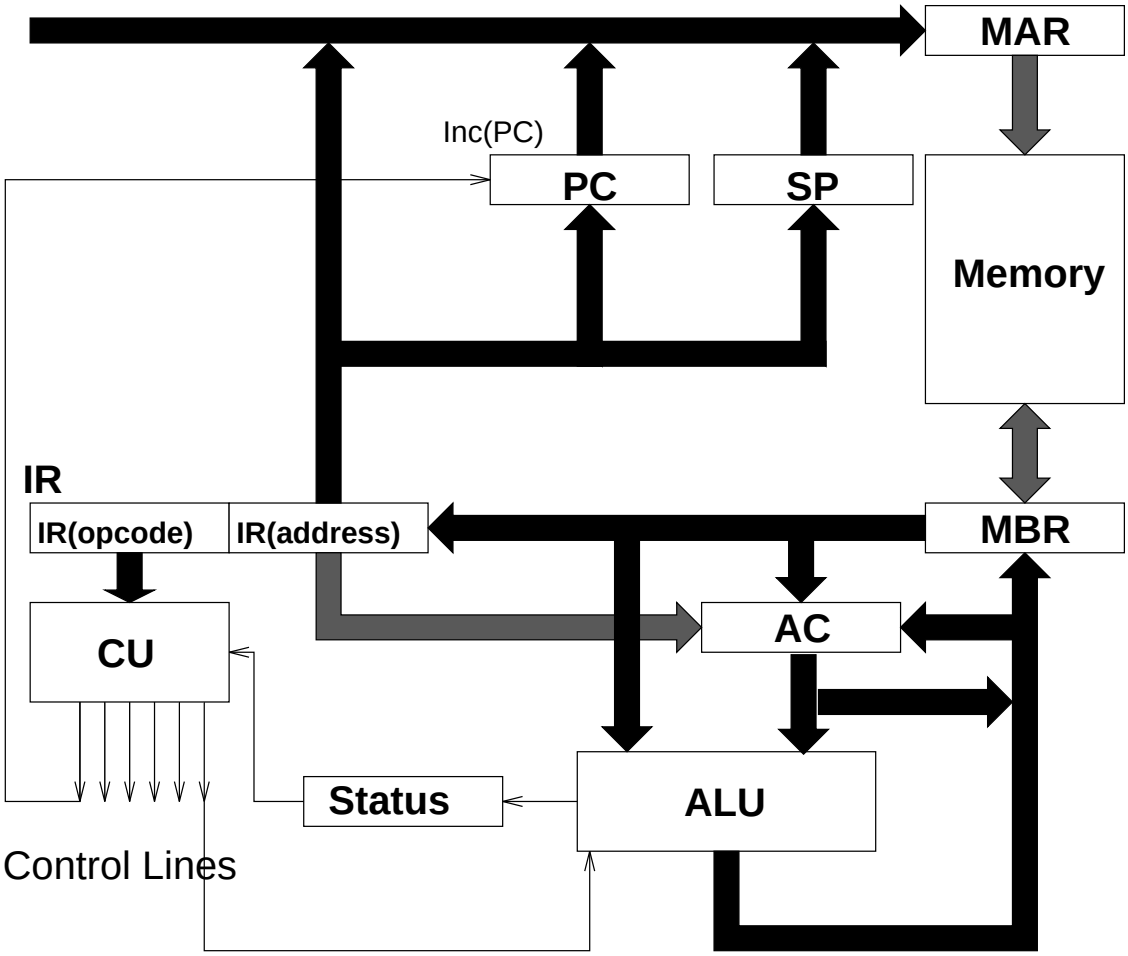


Figure 2.4: The Bog Standard Architecture

2.3.1 Registers

MAR The two memory registers, MAR and MBR, are "gatekeepers" for the two buses to the memory. The **Memory Address Register** is used to store the address that is currently being accessed in memory.

MBR The **Memory Buffer Register** (also sometimes called the memory data register MDR or MD) stores information that is being sent to or received from the memory along the data bus.

AC The **Accumulator** is used to store data that is being worked on by the ALU, and is the key register in the data section of the cpu. Notice that the memory can't access the AC directly. The MBR is an intermediary.

PC Connected to the address bus, the **Program Counter** holds the address in memory of the next program instruction. Notice that it doesn't connect directly to the memory, but must go via the the MAR.

IR When memory is addressed, the data first goes to the MBR. If the data is an instruction it gets moved to the **Instruction Register**. The instruction register has two parts:

1. **IR** (opcode). The most significant bits of the instruction make up the opcode. This the genuine *instruction* part of the instruction, that tells the cpu what to do. The instruction

in **IR** (opcode) gets decoded and executed by the control unit, CU.

2. **IR** (address). The least significant bits of the instruction are actually *data*. They get moved to **IR** (address). As the name suggests they (usually) form all or part of an address for later use in the MAR. (Sometimes — actually in immediate addressing — they are sent to the AC.)

SR The **Status Register** or Condition Control Word or Status Word is not quite the same as the other registers in that it really just a collection of 1bit flags that indicate the outcome of operations that the ALU has just carried out. There are the Carry C, and Overflow V flags you have already met, a negative flag N, and a zero flag Z. These are monitored by the CU.

SP The **Stack Pointer** is connected to the address bus is used to hold the address of a special chunk of memory used for temporary storage during program execution. See Lecture ...

For all their fancy names like accumulator and so on, the registers comprises nothing more than a row of D-type latches which share a common clock input providing temporary storage on the cpu.

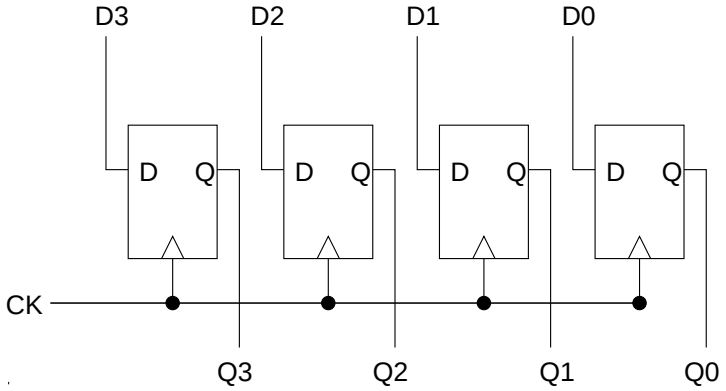


Figure 2.5: An 4bit register

2.3.2 Units.

CU The **Control Unit** is the nerve centre of the cpu, responsible for the timing and execution of the various register transfers required to fulfill an instruction held in the IR. It has a number of control lines coming out of it, which transmit CSL and CSP levels and pulses to the various registers and logic units.

ALU The **Arithmetic Logic Unit** is where bit crunching is carried out. It is responsible for bit operations on data held in the **AC** and **MBR** and for storing the results. We shall see that is contains arithmetic adders, logical OR-ers and so on.

2.3.3 Main memory.

The memory comprises mostly RAM with some ROM and is not on the cpu chip but is connected to the cpu via two buses, the data bus and the address bus, shown in grey in the Figure.

2.3.4 Buses and their widths

The buses carry *words* of information which are many bits wide, and on diagrams a bus is indicated either by a wide line, or by a single line with a dash through it often accompanied by the bus width in bits.

Data The data bus width ranges from 4bits for the simplest control microprocessor to 64bits the most sophisticated cpu. Over the last decade the width in the standard desktop micro has increased from 8bits, through 16bits and is now 32bits. Although the registers on the data side would ideally have the same width as the bus, this is sometimes not the case. Cpu's have developed more quickly than bus technology[1], and it is not unusual to have 32bit cpu sitting on a 16bit bus. Obviously they have to have two goes at the bus to fill their 32bit registers, so although they will be able to compute at 32bit speed, they will always be slowed by bus access.

Address The address bus doesn't have to be the same width as the data bus. Again the size has increased commensurate with memory technology and figures between 24 and 32 are now commonplace.

Three important points about the BSA and these notes are:

1. Inevitably, our architecture is a much simplified version of the real thing. However, none of the principle has been lost.

2. In these notes we will sometimes choose specific widths for various buses and registers. Making specific choices allows certain operations to be achieved and explained more simply, but might make other operations more obscure. You must try to distinguish between underlying principle and example. We shall assume that the data bus is 16bits or 2Bytes wide. All the registers on the data side are 16bits wide, except the **IR** . The **IR** (opcode) part should be wide enough to take the largest opcode. We assume the opcode is a fixed 8bits wide. The **IR** (address) part has to have the same width as the address bus. We shall use an address bus which is 24bits wide, so the **MAR** , **IR** (address) **PC** and **SP** are 24 bits wide.

2.4 Information flows

With the structure of registers, units, memory and buses laid out, we remind ourselves that the

Overall aim is simply to move information from memory to register, between registers, sometimes pass it through the ALU, and then stick it back into the memory, all in some nicely ordered sequence.

While the details can get complicated, they are far from difficult. Indeed, if you are in need of a comforting analogy, we are doing little more than "playing trains" with words of information. Having decided what computation we wish to carry out, the crucial design task in the cpu is to arrange things so that the control unit pulls its control lines at the right time to "set the points", allowing the information words to travel where we want them to.

2.5 Starting the processing: Fetching an instruction

2.5.1 The role of the Program Counter

As noted earlier, to start processing the cpu needs to *fetch* a program *instruction* from the main memory.

[1] one reason is that one manufacturer can drive cpu technology, whereas many manufacturers have built peripheral devices to sit on buses, and so don't want to see changes too often!

The PC is the important register in fetching the instruction. It holds the *address* of the next program instruction in memory. It is said to *point* to the next instruction. (Any memory address points to the memory contents at that address.)

Remember though that the memory address register acts as a gatekeeper to the memory, so the first thing to happen is that the program counter gets copied into the memory address register. Using RTL we have

MAR ←**PC**

Remember that under the rules of RTL this leaves **PC** unaltered.

Now we read the memory.

MBR ←**M**⟨**MAR**⟩

where **M**⟨**x**⟩ denotes the contents pointed to by **x**.

The next step is to move the instruction to the instruction register.

IR ←**MBR**

As noted earlier, in our standard architecture the **IR** is split into two parts, **IR** (opcode) and **IR** (address). As far as the instruction fetch is concerned it the **IR** (opcode) that is important. The opcode is decoded by the control unit, as described in Lecture 2.

Finally we do a touch of housekeeping. It is a reasonable assumption that the next instruction the cpu will fetch is in the next memory location, so the program counter is incremented.

PC ←**PC** + 1

This incrementing is in units of the memory word length. We have decided that our memory is 2Bytes wide, so that 1 Word=2Bytes.

So to summarize, the instruction fetch requires the following:

Fetch

MAR	←	**PC**
MBR	←	**M**⟨**MAR**⟩
IR	←	**MBR** ; **PC** ←**PC** + 1
decode ... using **IR** (opcode)		

Note that we can increment the program counter at the same RTL clock tick as loading the instruction register.

2.6 The example instruction set

For the purpose of explanation we give a few members of a model *instruction set* using Assembler Language mnemonics, together with overall RTL-ish description and binary value of the opcode.

Instruction	Description	Opcode	Meaning
LDA x	**AC** $\leftarrow$**M**$\langle$**x**$\rangle$	00000000	Load the accumulator with contents of x
STA x	**M**$\langle$**x**$\rangle$ $\leftarrow$**AC**	00000001	Store accumulator in memory at address x
ADD x	**AC** $\leftarrow$**AC** + **M**$\langle$**x**$\rangle$	00000010	Add mem contents at x to AC
AND x	**AC** =**AC** $\wedge$ **M**$\langle$**x**$\rangle$	00000011	Logical and ...
JMP x	**PC** $\leftarrow$x	00000100	Jump to instruction at x
BEQ x	if Z-flag=1 then **PC** $\leftarrow$x	00000101	if Z-flag is set then jump
CMP	**AC** $\leftarrow\overline{\textbf{AC}}$	00000110	Two's complement the AC
RSH	**AC** $\leftarrow$RightShift(**AC**)	00000111	Shift the AC 1bit to right

Assembler Language is a language designed around a particular cpu, and so there is no standard set of mnemonics. However, once you understand the purpose of the instructions, it is trivial to convert between languages — assuming that the hardware has the same capabilities, of course. For example, the above set is similar to that on an (ancient) TI 6502 cpu, but had we chosen to use Motorola 68000 assembler we would use the following.

6502	MC680000	Meaning
LDA x	MOVE x,D0	Load the accumulator (D0) with contents of x
STA x	MOVE D0,x	Store accumulator in memory at address x
ADD x	ADD x,D0	Add mem contents at x to AC
AND x	AND x,D0	Logical and ...
JMP x	JMP x	Jump to instruction at x
BEQ x	BEQ x	if Z-flag is set then jump
etc		

Notice that because the 68000 has multiple accumulators labelled D0, D1 etc, you have to specify the one you wish to use. The 68000 language has the simplifying feature that all inter-register transfers are described by "MOVE source,destination".

IMPORTANT! You do not have to learn any specific assembler language, but you should feel comfortable with the interpretation of any.

2.7 Executing an instruction

Let us look at LDA x. The opcode is %00000000, where the % denotes binary (0x will denote hexadecimal). During the instruction fetch this opcode gets places in the **IR** (opcode), and is decoded by the control unit (CU). The CU then tweaks its output control lines to effect the desired the register transfers etc.

Its action is to copy the contents of memory at address x into the accumulator. The variable x is called the *operand*. At its simplest in our BSA, the operand comprises the remaining 8 bits of the 16 bit instruction after removing the 8 bit opcode. These are the 8 bits in **IR** (address). Thus in six of our model instructions, the "instruction" contains an instruction to the control unit (the opcode), *and* a piece of data (the operand) which the opcode can utilize.

The overall effect we want is **AC** $\leftarrow$**M**$\langle$**x**$\rangle$. Looking at the BSA diagram we see that in detail this must be:

LDA x	**MAR**	$\leftarrow$	**IR** (address)
	MBR	$\leftarrow$	**M**$\langle$**MAR**$\rangle$
	AC	$\leftarrow$	**MBR**

Note that no computation has be been done by this instruction. Information has been moved around, from memory to accumulator, but not altered. The only part of the cpu that can alter data is the ALU.

The figure gives an example of the information flows in the entire fetch and execute of LDA x, where we assume that an 8bit operand is sufficient.

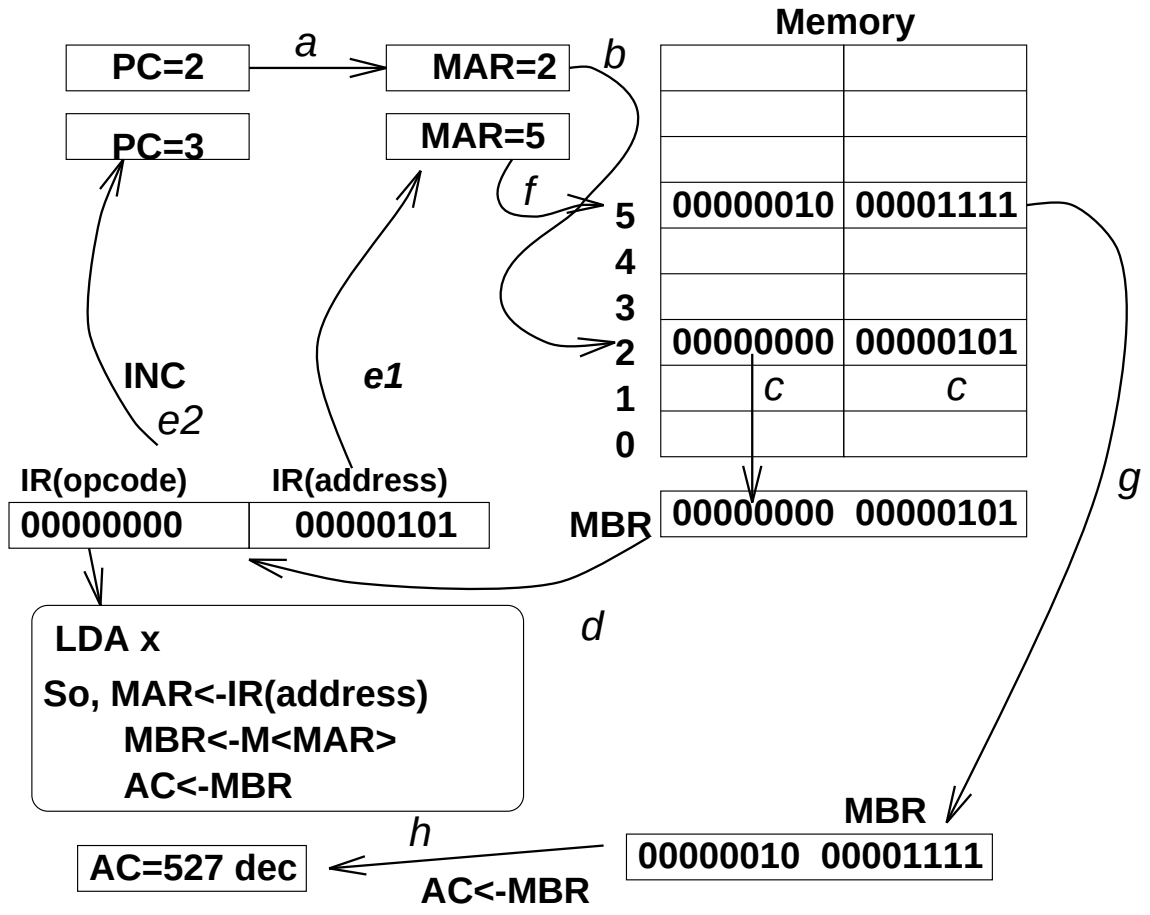


Figure 2.6: Example of the Fetch and execute of the LDA x instruction starting with **PC** =2. The steps follow a,b,c ... h.

2.8 An important note.

At its simplest in our standard architecture, the operand comprises the least significant 8bits originally in the **MBR** and then transferred to **IR** (address) at the same time that the most significant bits were transferred to **IR** (opcode).

But this appears to restrict operands in general to 8 bits, and to restrict the available range of addresses to 0–255 — which is obviously nonsensical, given that the address bus is 24bits wide!

In our architecture the obvious solution is to make the LDA instruction read the next word of memory into the **MBR** , and then into the **IR** (address), building up a 24 bit address for transferral to the **MAR** .

The detail of this step is often missed out in the ‘simplified’ explanations given in books, and the recommended text is no exception. It is just assumed in the first line of the execute phase that the operand in **IR** (address) has the required length.

The reason for this sleight is to avoid blurring the distinction between *instruction fetch* and *instruction execute*.

Clearly, because many instructions don’t require long operands it would be very wasteful if every instruction had to make an extra memory read to generate a redundant 24bit operand. This could be solved by opcodes containing a bit that indicated that a further read should be made to complete the operand. Unfortunately it then appears that part of the instruction *execute* phase is being used to complete the instruction *fetch*, and immediately the neat distinction is lost.

Lecture 3
Control Unit I: “Random Logic”

In Lecture 2 we saw that the core of the computer was the combination of cpu and memory, and that in practical terms computation required transferring words from memory to register, between registers, sometimes via the ALU, updating registers, then back to memory.

We introduced our Bog Standard Architecture, and saw how transfers of information could be described using Register Transfer Language. In particular we saw how the Instruction Fetch sequence and how the Load Accumulator command (with assembler mnemonic LDA) were described using RTL. We also introduced the eight-membered instruction set which we were going to explore on the architecture, making the restriction, for descriptive purposes only, that the opcode would have 3bits, and the operand 5bits, using an 8bit data bus. We also saw that the Control Unit (CU) is responsible for the appropriate sequencing of register transfers for each instruction.

In this lecture look at one way of implementing the control unit. There is in fact nothing random at all about the random logic controller — Clements suggest that *ad hoc* would be a better description, in that you just work out the specific requirements of your instruction set and implement them in hardware. The result is fast but inflexible. In Lecture 5 we will discuss a more flexible but slower method.

3.1 The Random Logic controller

3.1.1 Input and Output

Before starting to design the CU, it is useful to define what the inputs are and what the outputs are. For this exercise, let the input be any one of the 8 possible 3bit opcodes corresponding to our instruction set, together with the status register — in particular the Z bit of the status register. In addition of course, we have the central clock signal. The outputs are the required CSP and CSL signals required to drive RTL to execute the corresponding instruction.

3.1.2 Details of instruction fetch and execution

First, recall our exemplar instruction set:

Mnemonic	Description	Binary Opcode	Meaning
LDA x	**AC** ←**M**⟨**x**⟩	00000000	Load the accumulator with contents of x
STA x	**M**⟨**x**⟩ ←**AC**	00000001	Store accumulator in memory at address x
ADD x	**AC** ←**AC** + **M**⟨**x**⟩	00000010	Add mem contents at x to AC
AND x	**AC** =**AC** ∧ **M**⟨**x**⟩	00000011	Logical and ...
JMP x	**PC** ←x	00000100	Jump to instruction at mem loc x
BEQ x	if Z-flag=1 then **PC** ←x	00000101	if Z-flag is set then jump
CMP	**AC** ←$\overline{\textbf{AC}}$	00000110	Two's complement the AC
RSH	**AC** ←RightShift(**AC**)	00000111	Shift the AC 1bit to right

We now develop detailed RTL for these, first reminding ourselves of the fetch sequence for *any* instruction.

FETCH

MAR	←	**PC**
MBR	←	**M**⟨**MAR** ⟩
IR	←	**MBR** ;**PC** ←**PC** +1
(CU	←	**IR** (opcode) for decoding)

LDA x

MAR	←	**IR** (address)
MBR	←	**M**⟨**MAR** ⟩
AC	←	**MBR**

STA x This writes to memory.

MAR	←	**IR** (address)
MBR	←	**AC**
M⟨**MAR** ⟩	←	**MBR**

ADD x This instruction involves the ALU. Looking back at the BSA diagram we see that the ALU has two inputs one from the **AC** and the other from the **MBR** . To perform this line of RTL we must, before firing the transfer, set up the ALU to perform the binary addition of its inputs, rather than say the logical AND or logical OR. As you might guess, and as we will see in Lecture 4, this requires the configuration of combinational circuitry using a *level* signal. Obviously we are going to need a CSL in addition to a CSP for this line of RTL.

MAR	←	**IR** (address)
MBR	←	**M**⟨**MAR** ⟩
AC	←	**AC** + **MBR**

AND x This again involves the ALU, but now we are performing a logical AND.

MAR	←	**IR** (address)
MBR	←	**M**⟨**MAR** ⟩
AC	←	**AC** ∧ **MBR**

JMP x The PC is always incremented during the fetch cycle on the assumption that the next instruction is in the next memory location. This instruction allows an *unconditional branching* to a non-consecutive instruction.

PC $\leftarrow$ **IR** (address)

BEQ x This mnemonic is short for "branch if equal". To enable *conditional branching* (that is, if (condition is true) then branch") we need a logical condition to test for true or false. The condition is taken as one of the bits from the status register. Here we use the Z bit or Z flag, which is set Z=1 by the ALU when it makes a calculation whose result is zero, and Z=0 when it isn't. Note that because the PC is always incremented during the instruction fetch, if the condition is false (ie Z=0) the PC needs no alteration.

S_n	$\rightarrow (\overline{\mathbf{Z}})/(S_{n+2})$
S_{n+1}	**PC** $\leftarrow$**IR** (address)
S_{n+2}	*etc*

CMP Complements the contents of the **AC** . An ALU operation, so again we need a level to configure the ALU.

AC $\leftarrow$ $\overline{\mathbf{AC}}$

RSH This involves use of a shifter, often placed at the back end of the ALU.

AC $\leftarrow$RightShift(**AC**)

For each different action and/or transfer involved in the above instructions we a control signal CS. In some cases we need just an edge (CSP), in others we will also require a level (CSL) signal, but as one is derived from the other we will just specify CS.

A list of the separate actions is:

Control Signal	Action
C_0	**AC** $\leftarrow$**AC** +**MBR**
C_1	**AC** $\leftarrow$**AC** $\wedge$ **MBR**
C_2	**AC** $\leftarrow\overline{\mathbf{AC}}$
C_3	**MBR** $\leftarrow$**M**$\langle$**MAR**$\rangle$
C_4	**M**$\langle$**MAR**$\rangle\leftarrow$**MBR**
C_5	**MBR** $\leftarrow$**AC**
C_6	**AC** $\leftarrow$**MBR**
C_7	**MAR** $\leftarrow$**IR** (address)
C_8	**PC** $\leftarrow$**IR** (address)
C_9	**PC** $\leftarrow$**PC** + 1 (or INC(**PC**))
C_{10}	**MAR** $\leftarrow$**PC**
C_{11}	**IR** $\leftarrow$**MBR**
C_{12}	**AC** $\leftarrow$RightShift(**AC**)

Now let us construct an overall table showing the steps taken for all the intructions.

Opcode→ Phase↓	%0000 LDA	%0001 STA	%0010 ADD	%0011 AND	%0100 JMP	%0101 BEQ	%0110 CMP	%0111 RSH
0	$\longleftarrow C_{10} \longrightarrow$							
1	$\longleftarrow C_3 \longrightarrow$							
2	$\longleftarrow C_{11}; C_9 \longrightarrow$							
3	C_7	C_7	C_7	C_7	C_8	if Z=1, C_8	C_2	C_{12}
4	C_3	C_5	C_3	C_3	—	—	—	—
5	C_6	C_4	C_0	$---$	—	—	—	—

Notes:

1. The fetch instructions are the same for all.

2. The longest command now takes 6 steps, but there are some that take fewer. Some blank steps have been inserted, but see below for further details.

3.2 Implementation of the CU

The problem now is how to arrange for a piece of logic hardware to carry out what we require. It would certainly be possible to contruct a D-type latch controller of the sort shown in Figure 1.4, but here we explore two slightly different methods, one using a PLA, the other raw gates. The point to understand here is that there is no one ideal.

There are two parts which are common to both designs, but it is useful first to revise what inputs we have to the Control Unit.

3.2.1 The clock

It is conventional to think of the cpu's clock ticking once per instruction. As the longest fetch-execute cycle takes 6 lines of RTL, we have to subdivide this notional clock period into at least 6 subticks or subphases, $\phi_0, \ldots, \phi_5$. In practice we can always reset the subclock, and so we might as well generate 8 subphases. A circuit and its output are shown in Figure 3.1.

3.2.2 The opcodes

Let us produce a single line output representing each instruction using its opcode. This requires a decoder, as shown in Figure 3.2.

3.2.3 The flags

Here we are only using the Z flag.

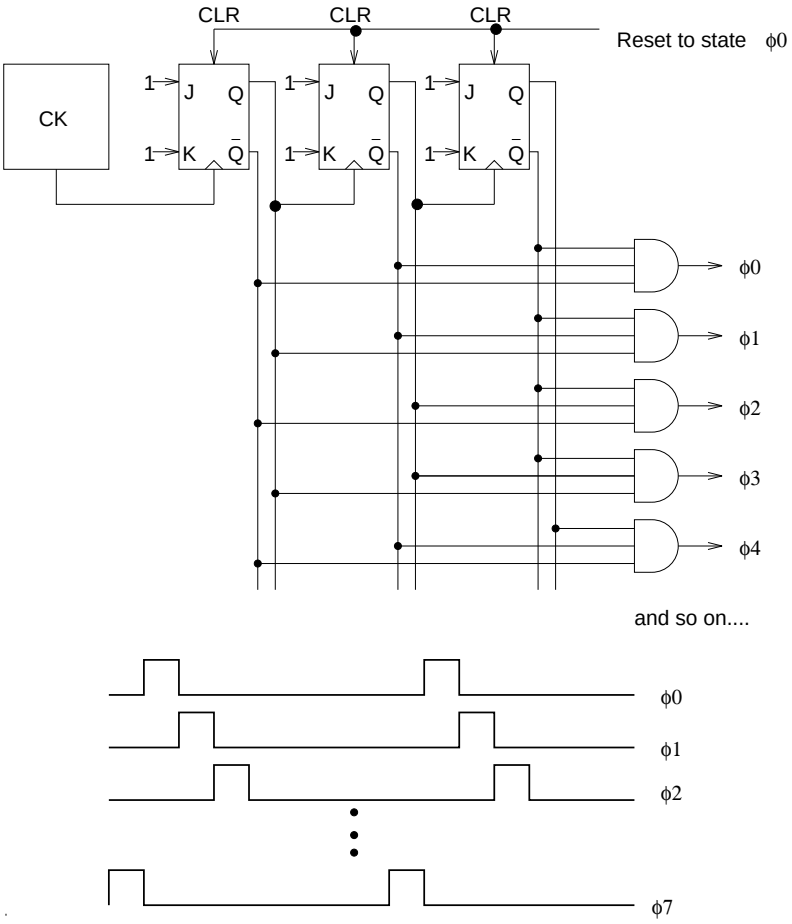


Figure 3.1: Producing 8 sub-phases to the clock

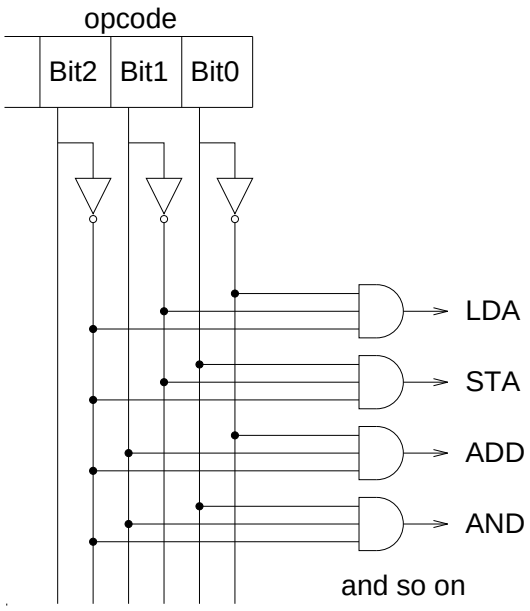


Figure 3.2: Decoding the opcode

3.2.4 Implementation using a PLA

Recall that a PLA is able to create a general sum of products. We can now write down the outputs C in terms of the inputs:

$$\begin{aligned} C_{10} &= \phi_0 \\ C_3 &= \phi_1 + \phi_4.(\text{LOAD} + \text{AND} + \text{ADD}) \\ etc & \end{aligned}$$

and implement the entire controller as shown in Figure 3.3.

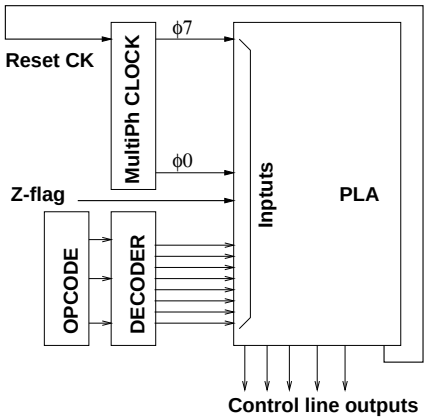


Figure 3.3: Implementation using a PLA

3.2.5 Using discrete components

Clements' book describes another way of implementation using discrete components. The circuit diagram, shown in Figure 3.4 should be self explanatory.

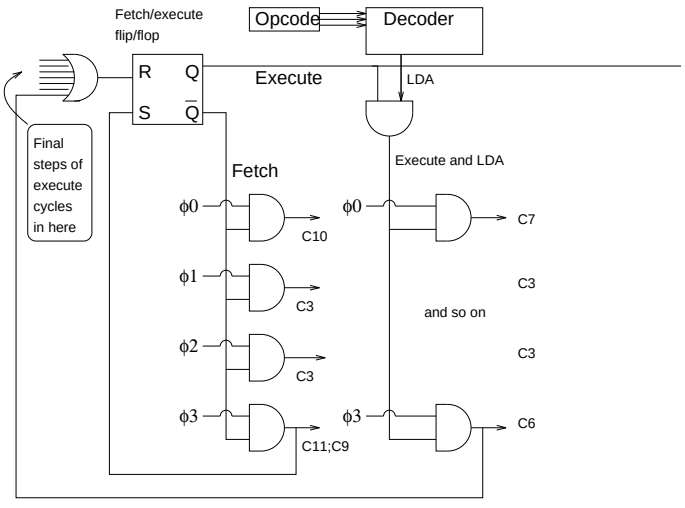


Figure 3.4: Implementation using discrete components.

The fetch and execute stages are separated using an RS flip-flop. Notice that in both parts only 3 phases of the 8-phase clock are utilized. At the ends of both the fetch and execute stages, a reset signal is send to the 8-phase clock.

3.3 Multiplexing or Output-Enabling using CSL signal

At the start we discussed the need for CSL signals to configure combinational circuitry such as the ALU, and the CSP signals to effect the actual register transfers.

There are other occasions when the CSL signals are required. In our BSA you will have noticed that several registers feed out onto the same bus. For example, via the ALU the **AC** and the **MBR** feed onto the data bus. Both the **PC** and the **IR** (address) feed onto the address bus. Clearly this will cause contention.

The piece of hardware that chooses one of several inputs is called a *multiplexer*. To select one of 2^n inputs, n selection lines enter a decoder, and the 2^n outputs from the decoder are AND-ed with the inputs. The outputs from the AND gates are OR-ed. It is sketched in Figure 3.5.

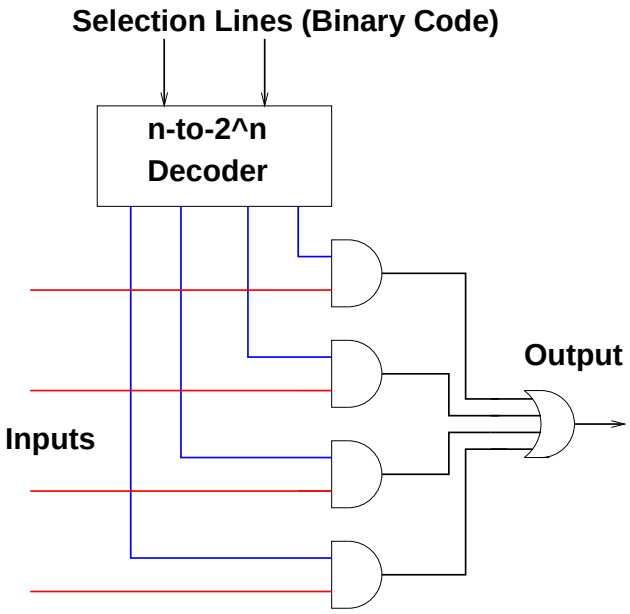


Figure 3.5: A multiplexer

3.3.1 Using tri-state logic

It is more common however when outputting onto buses to use an output-enable stage on the back of a register, driven by the CSL signal. OE inputs to some *tri-state logic* which then outputs to the bus. Tri-state logic has three states, assertive 0, assertive 1, and non-assertive. In the assertive mode, the output resistance of a tri-state gate is low, and the bus is forced to follow the output, whether it be 0 or 1. In the non-assertive state, the output resistance of the gate is high, and the gate's nominal output can safely be in a different state from that of the attached bus.

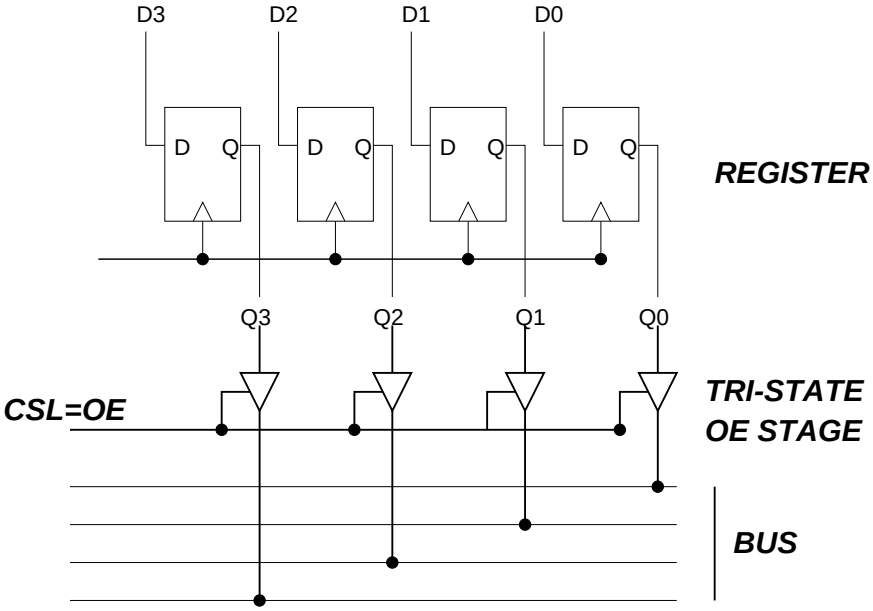


Figure 3.6: Tristate for output-enabling the individual bits of a register.

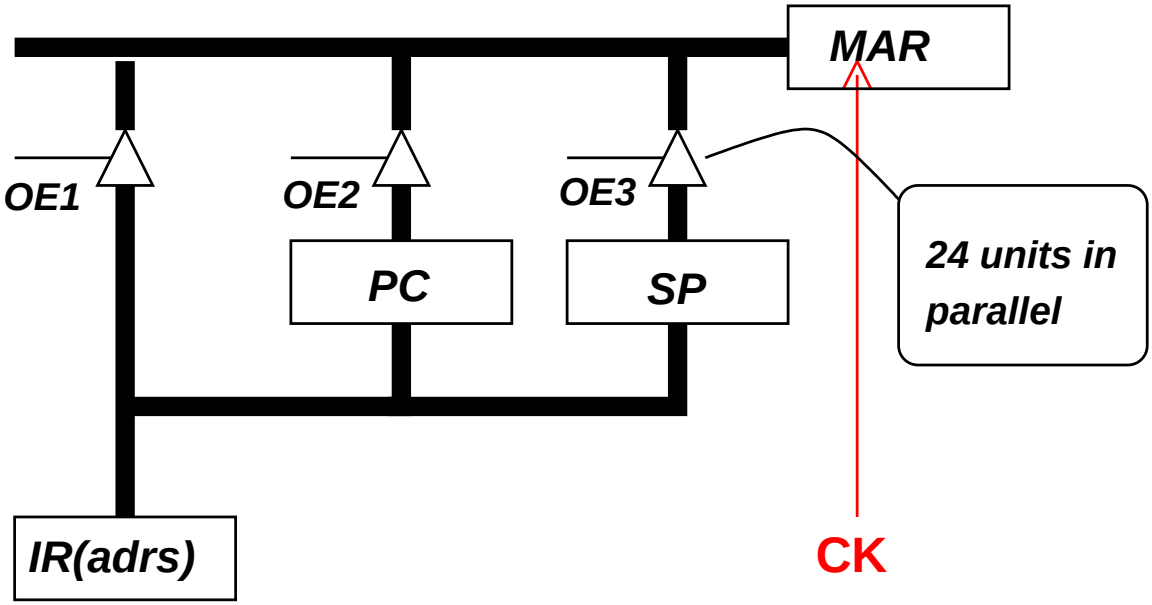


Figure 3.7: Example of the output-enabling required for the **IR** (address), **PC** and **SP** .

Lecture 4
Details of Hardware in CPU and Memory

In the last lecture, we saw how to break down a set of instructions into their individual and indivisible register transfers, and one method of arranging for the Control Unit to fire those transfers in the correct sequence. We saw that we need both CSL and CSP signals to drive combinational element together with the actual transfers.

In this lecture we will look more closely at the hardware associated with the ALU, Registers and Memory.

First, however, it is worth pointing out that you now know enough to build a programmable computer by looking at a small program written in assembler and seeing how it appears in the memory.

4.1 A small program

Let us take a preview at a small program written in assembler:

```
        LDA 20          ;LOAD AC with contents at loc 20
AGAIN:  SUB 22          ;SUBTRACT from AC contents of loc 22
        BEQ STOP        ;If ALU gives 0, jump to label STOP
        LDA 20          ;LOAD AC with contents of loc 20
        ADD 21          ;ADD contents of loc 21
        STA 20          ;STORE in loc 20
        JMP AGAIN       ;JUMP back to label AGAIN
STOP:   HALT
```

We have introduced two more assembler mnemonics SUB and HALT — let these have opcodes 8 and 9, ie %00001000 and %00001001.

Let us also assume that the first instruction gets stored at location 0 in memory. The location and contents of the memory relating to the program are as follows. Notice how the line labels are assembled as *memory locations*. This is because they are going to supply information to the **PC** during the JMP and BEQ instructions.

```
---------------------------------
Instr-  Loca-   HighByte  LowByte
uction  tion      OPCODE  OPERAND
---------------------------------
LDA 20    0     00000000 00010100
SUB 22    1     00001000 00010110
BEQ 7     2     00000101 00000111
LDA 20    3     00000000 00010100
ADD 21    4     00000010 00010101
STA 20    5     00000001 00010100
JMP 1     6     00000100 00010001
HALT      7     00001001 00010100
---------------------------------
```

In addition, we need some data into memory:

```
---------------------------------
        Loca-   HighByte  LowByte
        tion
---------------------------------
         20     00000000 00000101   (dec 5)
         21     00000000 00000001   (dec 1)
         22     00000001 00101101   (dec 300)
---------------------------------
```

When you initialize the **PC** to 0 and start the program, the **AC** gets loaded with 5 (from location 20), and then has 300 (from location 22) subtracted from it. Because 5 isn't equal to 300, the Z flag does not get set, and so we reload the **AC** with 5, add 1 (from location 21) to it, and then restore it in location 20. The program eventually halts when location 20 is incremented up to 300. (Halting simply stops the clock.)

Notice that there is no *requirement* for assembler mnemonics or for that matter a high level language. The raw form of the executable program and data is binary code, and you could, and in the earliest days they had to, load the binary into the memory by hand.

4.2 Decoder

A Decoder inputs an n-bit number on n lines and sets just one of it 2^n output lines high. You have already seen them used in Lecture 3 to decode the opcode in the control unit. In this lecture you will see their use in the ALU and in Memory. In the latter case the inputs are the address lines and the output selects a particular word of memory. The hardware was sketched in Lecture 3.

4.3 Multiplexer

A Multiplexer selects one from a number n of inputs. We saw their hardware in Lecture 3: recall that they included an x-to-n line decoder, n AND gates, and an n-input OR gate.

4.4 The Arithmetic Logic Unit

As noted earlier, the ALU is the only part of the CPU that does anything to the information — the rest just shovels information from one place to another. The ALU is designed to perform both logical operations and arithmetic operations. Looking back at the BSA diagram, you will recall that it has two inputs, one from the **AC** and the other from the **MBR** .

4.4.1 Logical operations

Logical operations between to n-bit numbers take place in parallel on a bit-slice basis, as shown in the Figure. In addition to AND, there will certainly be OR and INVERSION, the last operating only a single input of course.

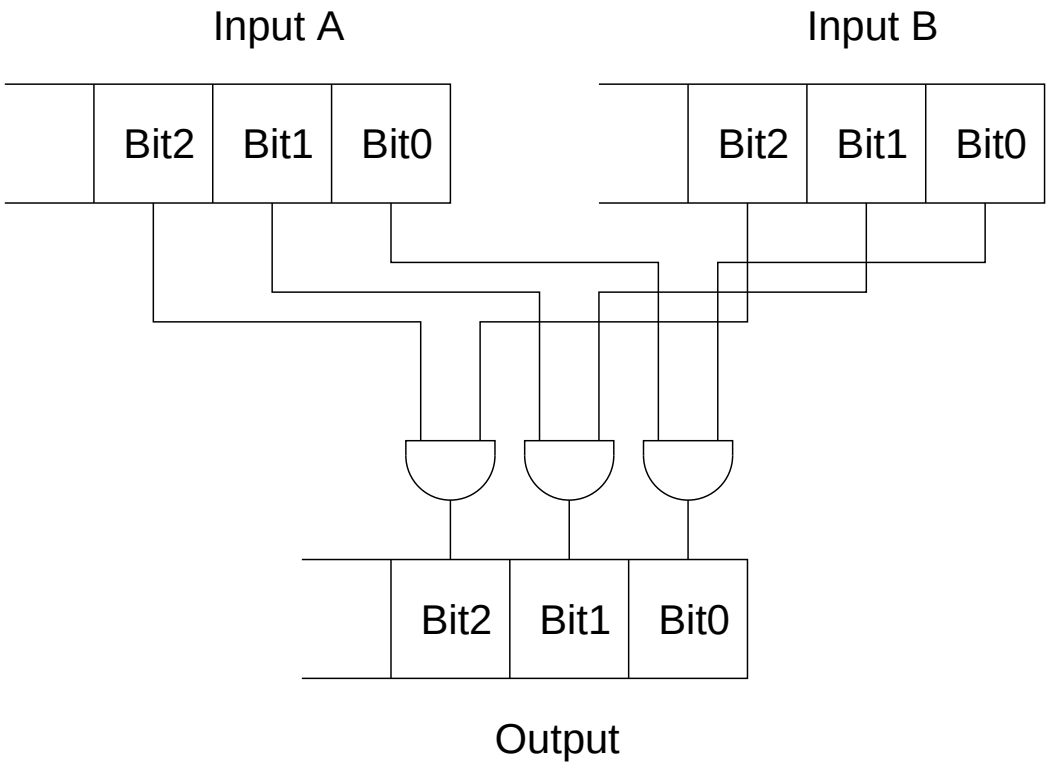


Figure 4.1: AND-ing operation.

4.4.2 Arithmetic operations

The ALU contains a set of full adders, each operating on 1bit. Each has two inputs plus carry inputs, the summed output and a carry bit.

4.4.3 The 1-bit ALU

With all the above devices in place, we can now build a 1bit ALU taking in 2 bits A and B, a carry-in, and, for this example, two selection lines F, and outputing 1 bit and a carry-out.

Notice that:

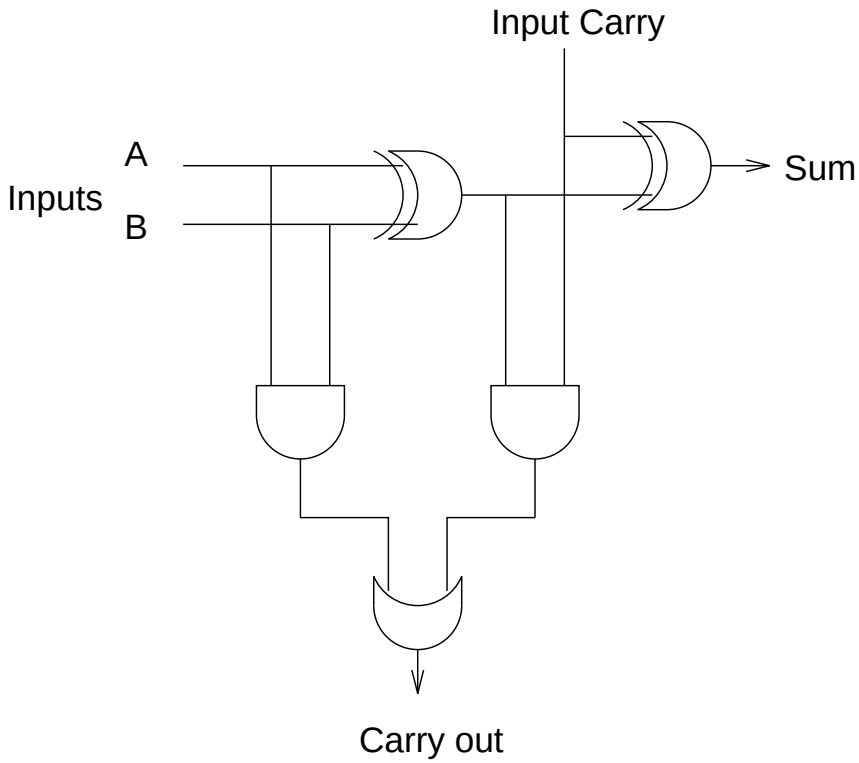


Figure 4.2: Full 1bit adder

1. All the logic is combinatorial. The speed of the ALU is limited only by delay time in the logic gates. There is no synchronizing clock. Effectively this means that the settling time should be less than the register transfer clock rate, so that for example **AC**$\leftarrow\overline{\mathbf{AC}}$ is performed correctly.

2. The ALU's overall function is controlled by a set of selection lines generated from a decoder. Note that these don't select the operation per se, but select the output. The back end of Figure 4.3 is jsut a multiplexer.

4.4.4 Multi-bit bit-slice ALU

To build a multi-bit ALU, we simply stick the 1bit ALUs together, as shown in the Figure 4.4. You learned in your 1st Year lectures on Digital Logic that using a ripple carry in this way is slow, especially in high width machines which are by definition meant to be fast! As you saw in the previous lectures, carry-look-ahead circuitry can be constructed.

4.5 Bits in the Status Register

An important function of the ALU is to set up bits or flags which give information to the control unit about the result of an operation. The flags are grouped together in the status register or word. Remember that the ALU has only an adder. To subtract one has to use 2s-complement arithmetic, but the ALU has no knowledge of this at all — it simply adds two binary inputs and sets the flags. It is up to control unit, or really the programmer's instructions executed by the control unit to interpret the results.

Z flag Z for Zero. This is set to 1 whenever the output from the ALU is zero.

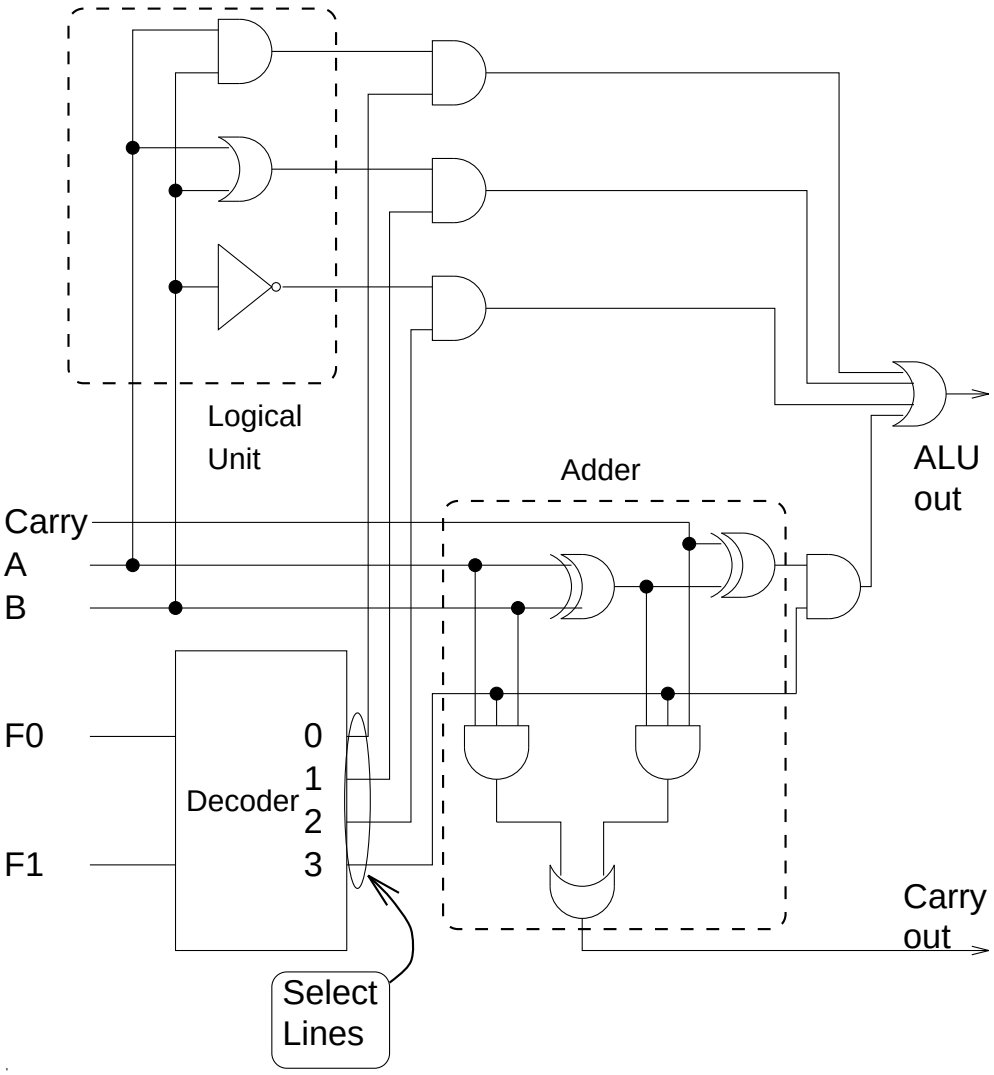


Figure 4.3: The 1bit ALU

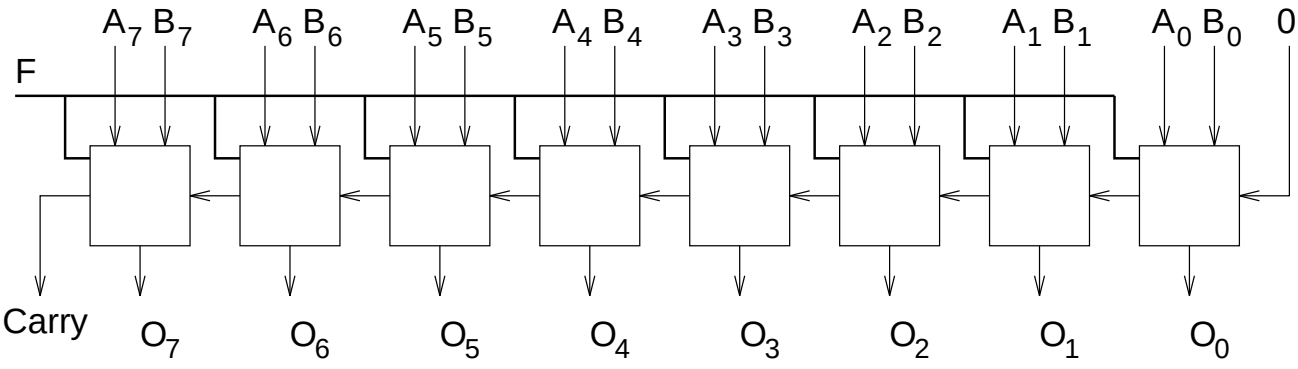


Figure 4.4: Multi-bit ALU. The carry-in on the right must be 0. Note that the selection line F is several bits wide.

N flag N for Negative. This is set to 1 whenever the most significant bit of the output is 1. Note that it is not correct to say that it is set when the output of the ALU is negative: the ALU doesn't know or care whether you are working in 2's complement. However, this flag is used by the controller for just such interpretations.

C flag C for Carry. Set to 1 when there is a carry from the adder.

V flag V for oVerflow. Set to 1 when $A_{msb} = 1$, $B_{msb} = 1$, but $O_{msb} = 0$; or when $A_{msb} = 0$, $B_{msb} = 0$, but $O_{msb} = 1$. Allows the controller to detect overflow during 2's complement addition. Note again that the ALU does not know em why it is setting the flag.

4.6 Clocks

The main clock in the cpu is derived from a crystal oscillator circuit.

Here we are with some dated statements about clocks ...

- 1989 — Dr Probert talked about top range processors with clock speeds of 40MHz.
- 1994 — more down to earth (or at least to desktop), the standard PC with an Intel 486 processor inside has a frequency of 25MHz ...
- 1995 — Frequencies in the most advanced cpus are now greater than 100MHz and DEC are bringing out their latest Alpha CPU with a clock of 300MHz ...
- 1998 — Desktops at 230MHz ...
- 1999 — Desktops at 500MHz ...
- 2000 — Desktops with Intel Xeon processors at 930 MHz.

Does doubling the clock speed does not necessarily make the machine twice as fast? Unfortunately not. Although the cpu is twice as fast, one is limited by bus transfers to memory and other peripherals. This is an example of the *von Neumann bottleneck*.

4.7 Standard Registers

We saw the basic hardware for a register in Lecture 3.

Recall the tri-state hardware associated with Output-Enabling a register onto a bus. The tri-state solution makes redundant the final OR-gate of the standard multiplexer. This is an advantage on a bus where spatial extent preclude bringing all the potential outputs together.

4.8 Counters — eg Program Counter

The PC is a counting register, which is either loaded from the **IR** (address) register (eg during a JMP instruction), or (more usually) is incremented. Its hardware is shown in figure 4.5.

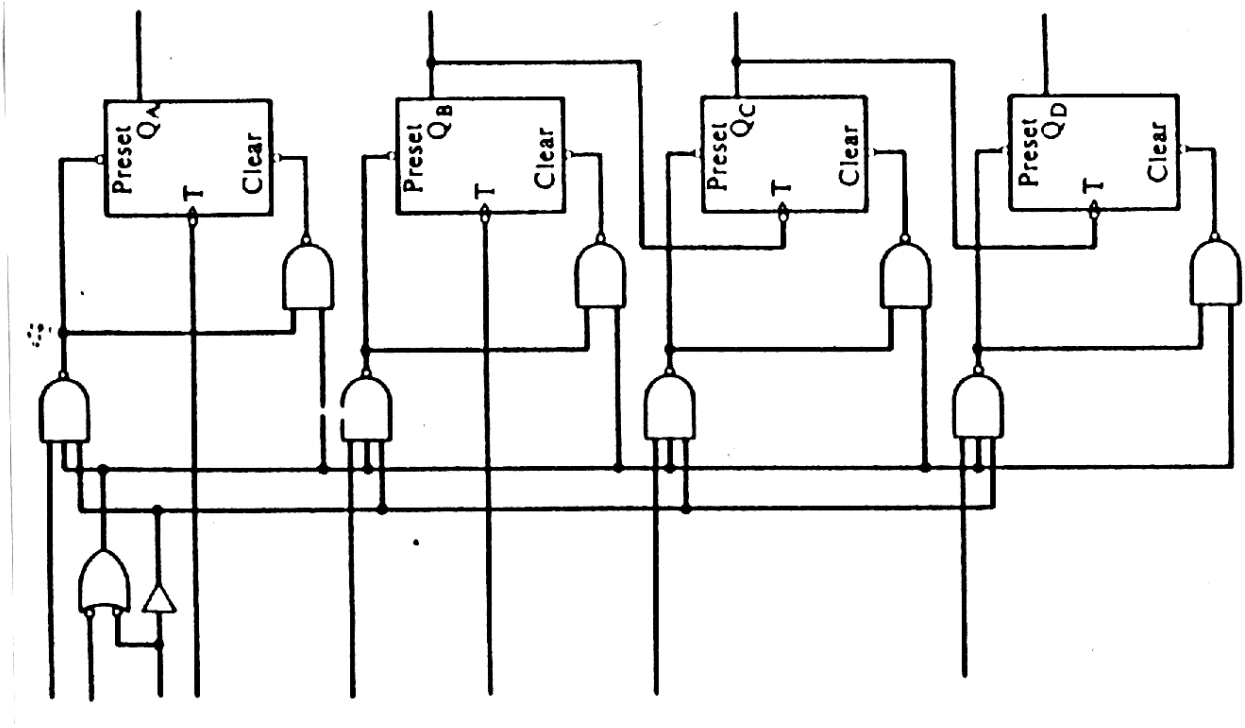


Figure 4.5: The program counter

4.9 Shifters

Most of the conventional registers in the cpu lead to bit[n] of the input going to bit[n] of the output. A shift register results in bit[n] being placed in bit[n+1] for a left shift, or into bit[n-1] for a rightshift. In a left shift, the msb gets put into a carry bit. A shift register is often placed at the output of the ALU. The hardware is shown in Figure 4.6.

4.10 Memory

Memory is no more than a very large collection of registers held on an array on a chip, one register being accessible at a time via an addressing mechanism.

Figure 4.7 shows the design of a 4×3bit memory.

The address lines A enter a decoder, which selects one cell — or one row of three D-type latches in the diagram. Notice that the enable lines split into two. After the split, one of the lines enables WRITING, by being ANDed with the clock; the other enables READING by ANDing the outputs from the Q pin on each latch.

Let us look more closely at writing. The 3 data lines in are always connected to all the D pins of the latches. But of course it is the output pin Q that ”holds the memory”, so it is only when the CK is enabled and pulse that input gets written. Going down to the bottom left you will see a chip-select signal CS you see that is is ANDed with a $\overline{RD}$, (.NOT.read) to provide the CK. RD is the clocked signal. CS is the chip selector, and is discussed later.

Looking at the outputs from the latches, the enabled outputs are ORed with all the disabled outputs, and the three outputs head off towards the data bus. Notice that the outputs are Output-enabled using tri-state logic onto the bus. The need for this will be become clear later.

Finally, note that in the diagram, it looks like the input data lines (top-left) and output data lines (bottom-right) are

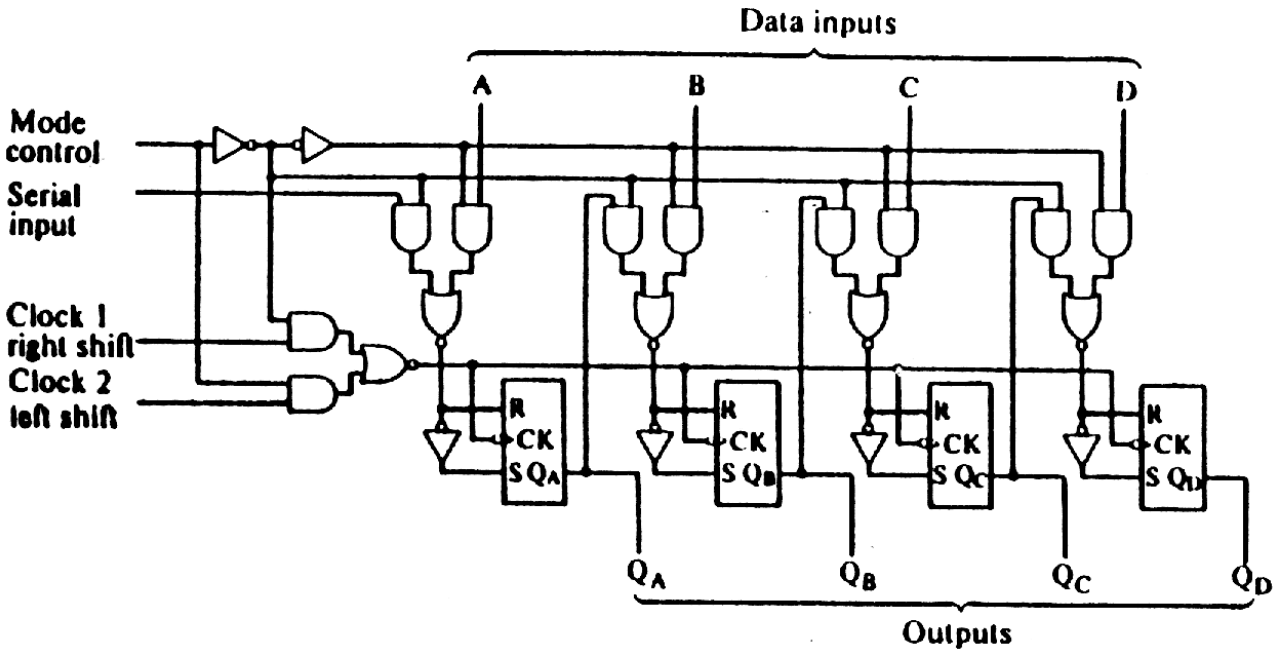


Figure 4.6: A Shift register able to shift either rightwards or leftwards.

different. They are not of course — they are the same data bus.

4.11 Memory organization

An address bus with n lines can address 2^n words of memory, so our BSA with an address bus 24bits wide could address 16Mwords.

Standard memories are 1Byte wide, so our 16bit data bus would require 2 chips in parallel. Note the use of the Chip Select lines.

The arrangement is shown in Figure 4.8.

Some Notes

Note 1. All the address lines go to each chip, whereas the data lines are selective.

Note 2. Using the CS lines, one can select both Bytes, upper Byte, lower Byte, or neither Byte.

Note 3. The picture is similar to the memory organization of the Motorola MC68000, but not identical (cf Figure 6.2 in Clements' book).

The difference, to do with the range of the address bus, can be more than a little confusing, the more so as the difference would never arise in a machine with an 8bit data bus.

We have assumed that the address in given in Words. That is, we talk about the upper and lower byte of Word 0, Word 1, and so on. Hence our 24 address lines $A_0 - - - A_{23}$ access 16MWords and hence 32MBytes of memory, as sketched in Figure 4.9(a).

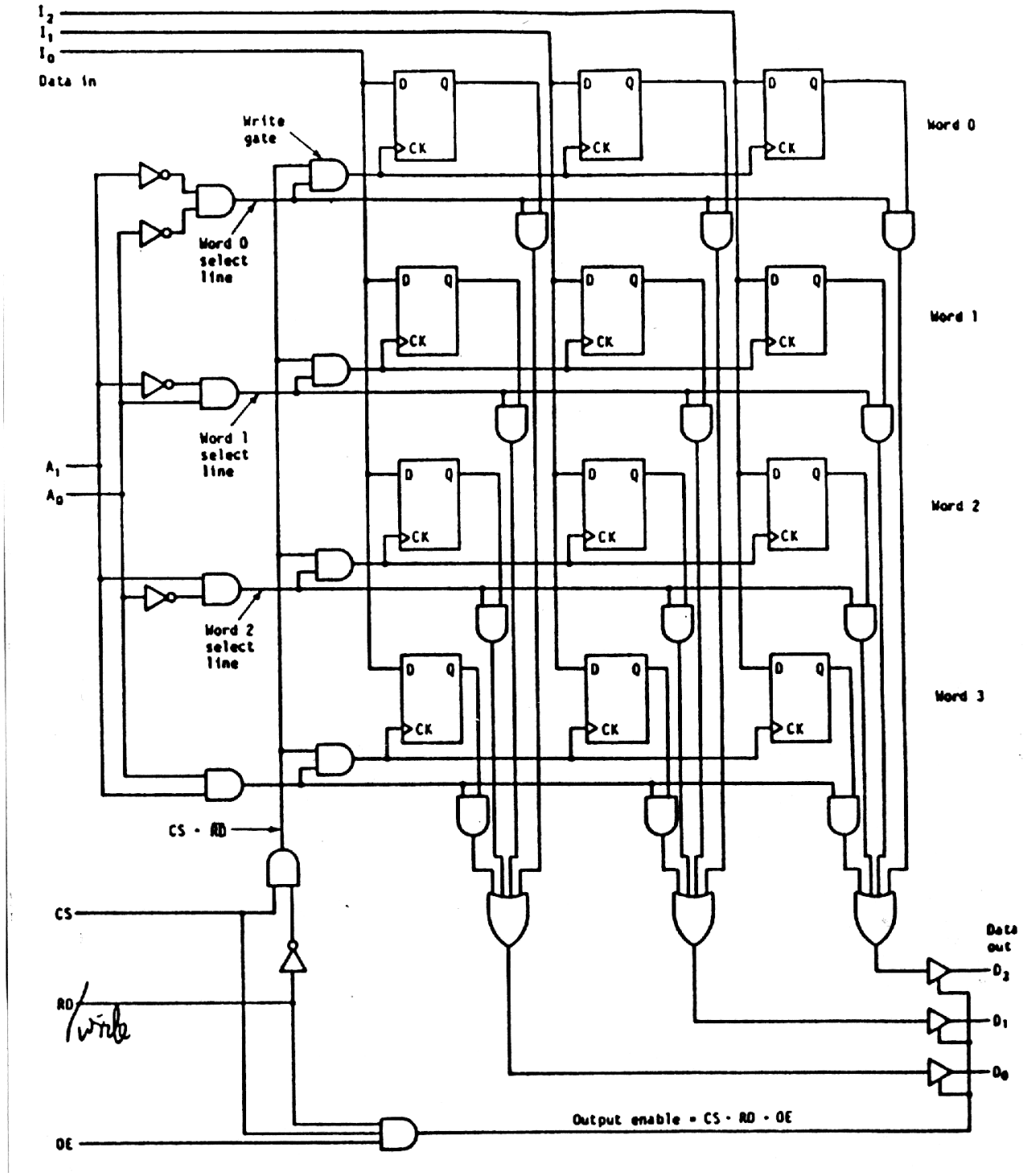


Figure 4.7: A 4×3bit memory (from Tanenbaum)

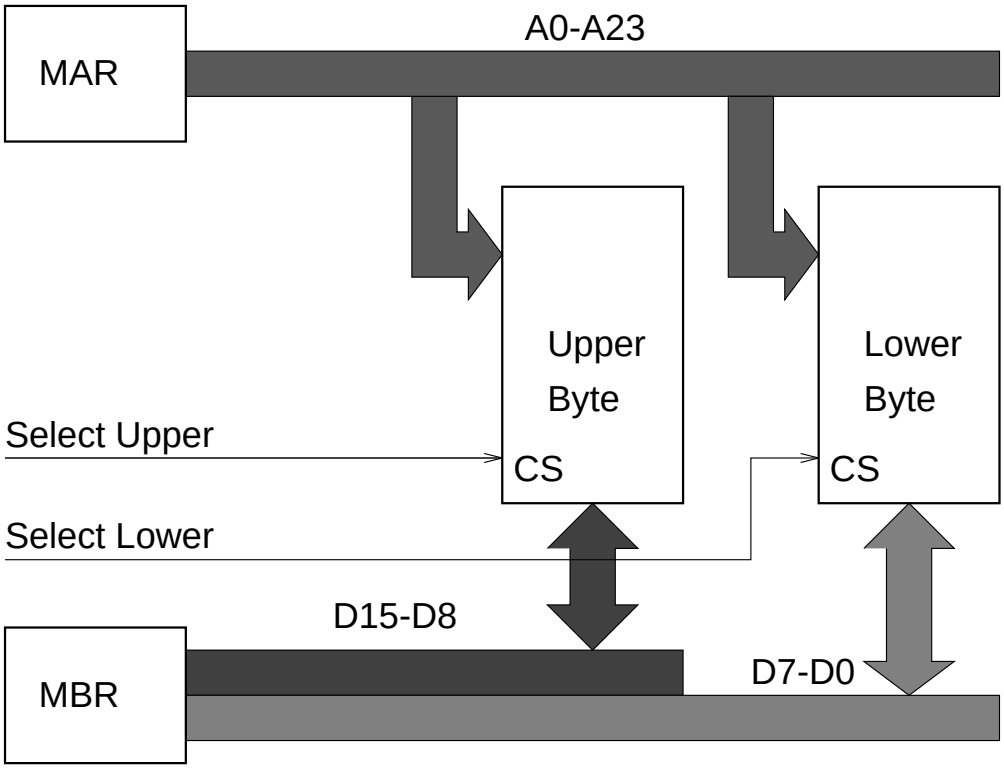


Figure 4.8: Memory chip organization

In fact, on the 68000 and indeed more generally, the address is given in Bytes. The 68000 adopts this convention shown in part (b) of the Figure, where you should note that the upper byte has an even address (0,2,4 etc), and the lower byte an odd address.

Clearly, using this scheme, with a 24bit address line one can access only 16M *Bytes* of memory rather than 16M *Words*.

To confuse us some more, the memory is then only addressable on *even Byte boundaries*. What this means is that the only valid addresses are even (eg LDA 0 is legal, LDA 5 is not). Now this makes accessing the memory very much like our BSA model — only words are addressable. However, because of this restriction, the least significant address line is redundant — A_0 would always be $A_0 = 0$. So the 68000 uses address lines $A_1 - A_{23}$ not $A_0 - A_{23}$.

Notice that this restore economic use of the address lines. In our scheme, 24 address lines A_0–A_{23} permits access to 16MWords or 32MBytes, whereas the MC68000 scheme uses 23 address lines $A_1 - A_{23}$ to address 8MWords.

4.12 Types of memory

4.12.1 RAM — Random Access Memory

The bulk of memory in a computer is openly available for reading and writing, and is called RAM. There are two technologies widely used:

Dynamic Ram This comprises an array of *capacitors* rather than latches. As the charge tends to leak out from charged capacitors to uncharged capacitors, the memory must be refreshed every few milliseconds. This involves checking the level and, if above a threshold, fully recharging, and if below the threshold, full discharging. Despite the need for complicated refresh circuitry,

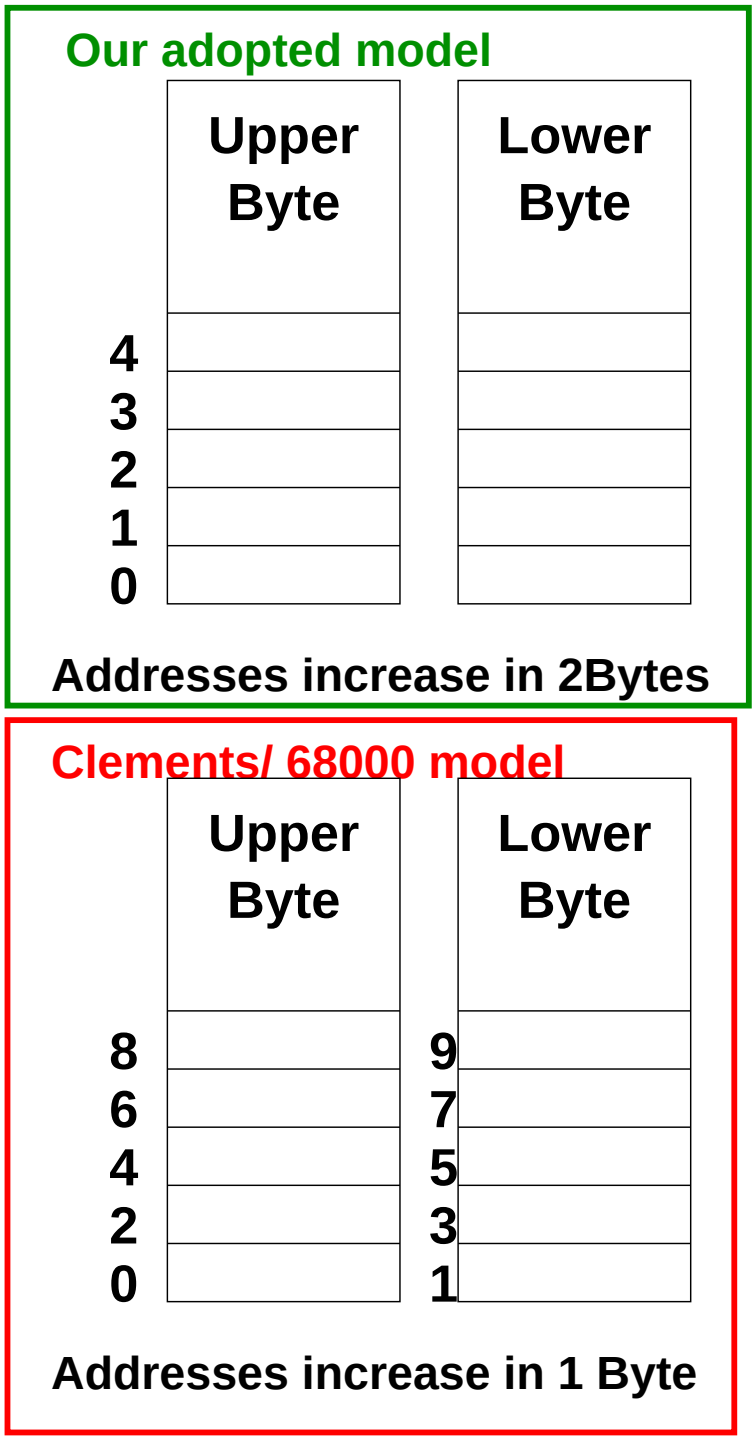


Figure 4.9: (a) Our model. (b) The 68000 scheme.

Technology	Access time (s)	Access Mode	Alterability	Permanence	Operation Medium
Bipolar	10^{-9}	Random	R/W	NDRO, volatile	Electronic
MOS	10^{-8}	Random	R/W	DRO/NDRO, volatile	Electronic
Mag Disk	10^{-2}	SemiRandom	R/W	NDRO, volatile	Magnetic
Mag Tape	10^{-1}	Serial	R/W	NDRO, volatile	Magnetic
CD ROM	1	SemiRandom	R	NDRO, volatile	Opto/Elect

Table 4.1: Main memory and bulk memory characteristics. (N)DRO stands for (Non)Destructive Read Out. (from Hayes Fig 5.5)

capacitors are so much simpler than latches that the cost of DRAM is considerably lower than its competitor, SRAM, and so the bulk of memory tends to be DRAM. Note that DRAM is slower too than SRAM.

Static RAM This is made of latches essentially equivalent to the D-type latch. Memory is retained provided power is maintained. SRAM is fast and expensive. Typically a desktop PC costing £1000 will have a cache memory (64–128KB) of SRAM. The use of cache memory will be discussed later in the course.

4.12.2 ROM — Read only memory

Read-only memory is useful not surprisingly for storing things that you don't want overwritten. In a general purpose computer, they are used to hold the bootstrap program, which is called on switching on the machine. This tells the machine about its basic hardware (keyboard, monitor and disks) sufficient for it to be able to load the operating system from hard disk. In special purpose computers for control, say the engine management system of my Porsche there may be no need for RAM.

In fact ROM now comes in various strengths:

ROM This really is read only, with the data inserted during the manufacturing process. Clearly useful if you are committed to produce lots of ROMs, the design of which will not change for batch to batch.

PROM Programmable ROM is programmable once after manufacture. Useful for smaller batch runs, but now superceded by EPROMS.

EPROM Erasable PROMS have windows in the top through which you can erase the contents using UV light. You can then reprogram the chip. Most ROMs are EPROMs.

EEPROM Electrically Erasable PROMs can be erased by applying pulses, and so don't required a box with a UV lamp in it. Large EEPROMs are now being produced. Their speed is lower than RAM, but they have a market as specialized disks.

4.13 Building a memory from standard chips

The size of memory chips has risen over the years. Though technically feasible to build very large single chip memories, potential sales and problems of yield make it much more economical to produce "reasonably sized" chips that find application in small memories, but can be built up into larger memories.

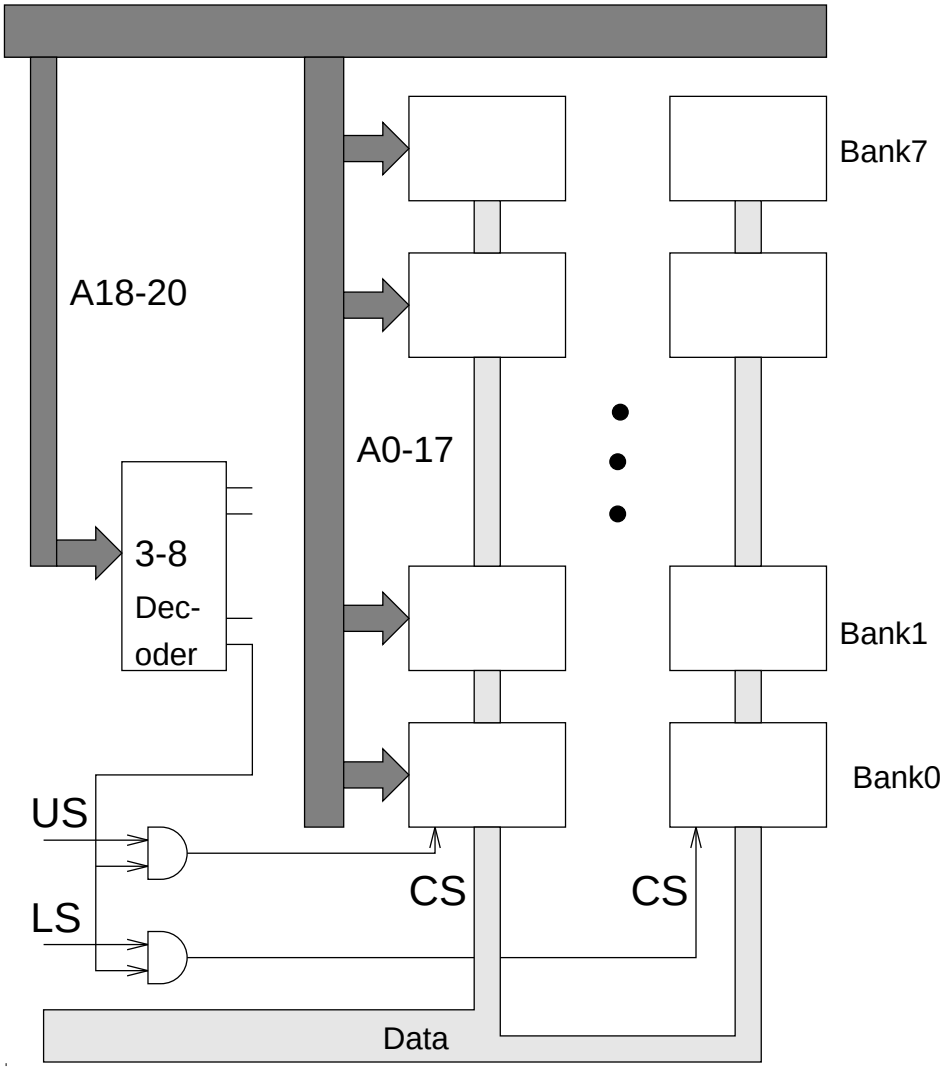


Figure 4.10: Building a large memory from smaller chips.

For example, as shown in Figure 4.10, to build a 2MWord (where Word=2Bytes) memory from 1/4MB chips one would arange them “8 high” to cover the address space, and “2 wide” to cover the the data space. The low 18 address lines are connected to each of the chips, but the remaining 3 address lines are entered into a decoder the 8 outputs of which are connected to the chip select pins. Two extra signals, LS (low byte select) and US (upper byte select) can be used if there is a need to select one or other bytes.

4.14 Address space and physical memory

The n address lines give the ability to address 2^n different locations. These locations span the *address space* 0x0 to 0xFFFFFF for our 24-bit address bus.

There is no need

- for the entire address space to be occupied by physical memory
- for the physical memory that is fitted to be located contiguously in address space. That is, there can be gaps.

Quite how the physical memory is mapped onto the memory space depends on how the address lines are decoded. Suppose we have 13 address lines A0–A12. These can address 8K (ie 8192) locations in memory. Suppose also that we have just two 1K (ie 1024) word memory chips M1 and M2. Each uses ten address lines A0–A9, but we must devise a way of selecting one rather than the other. One way of doing this is by partial addressing, where one of the remaining address lines, and its inverse are used to connect to the CS of the memory chips.

Let us first use A10, as shown in Figure 4.11. Although this is very simple, the disadvantage is that because we are ignoring A11 and A12 the memory chips are mapped FOUR times to the cpu's address space. That is, there are four addresses that the cpu could generate, that would access same memory location. For example 0x1d7b, 0x157b, 0x0d7b and 0x057b, all access location 0x057b of M2. (M2, because A10=1.) The mapping is shown in the table below.

Using another single line, eg A11 (Figure 4.12), will change the mapping, but because two lines are being ignored there is still a four-fold duplication, as shown in the table below.

Full address decoding means that all the remaining lines are decoded to obtain the CS. In our example, we would need a 3–8 line decoder, and connecting M1 and M2 to output lines 0 and 3 as shown in Figure 4.13 would place M1 between 0x0000–0x03ff and M2 between 0x0c00–0x0fff. The mapping from CPU to memory is unique, but notice that many of the addresses are now illegal — there is simply no memory there.

Base-address of 1K block Hex	Binary	Partial Decoding A10	A11	Full Decoding
0x1c00	1 1100 0000 0000	M2	M2	-
0x1800	1 1000 0000 0000	M1	M2	-
0x1400	1 0100 0000 0000	M2	M1	-
0x1000	1 0000 0000 0000	M1	M1	-
0x0c00	0 1100 0000 0000	M2	M2	M2
0x0800	0 1000 0000 0000	M1	M2	-
0x0400	0 0100 0000 0000	M2	M1	-
0x0000	0 0000 0000 0000	M1	M1	M1

* a,b,c denote A10,11 and 12, respectively.

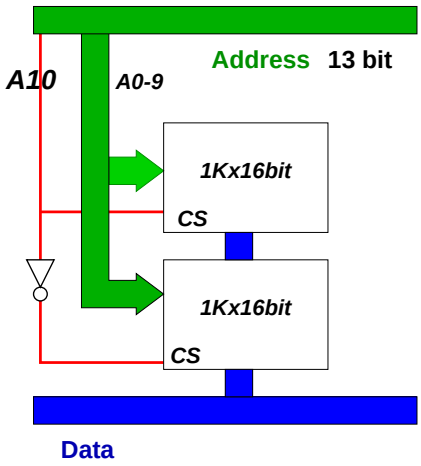


Figure 4.11: Address decoding. Partial decoding of A0-12 using A10.

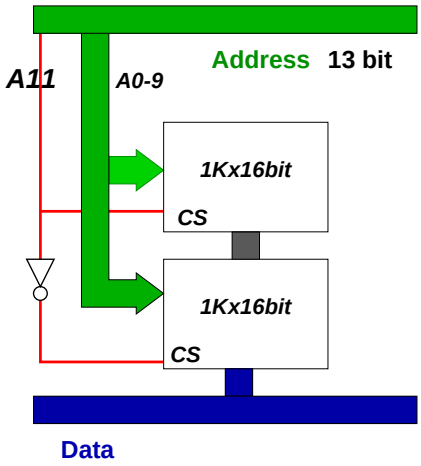


Figure 4.12: Address decoding. Partial decoding of A0-12 using A11

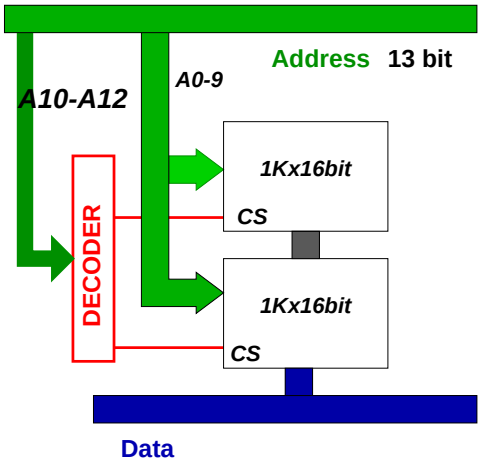


Figure 4.13: Full decoding of A0-12 uses all the address lines.

4.15 Read and write cycles

The timings for writing to and reading from memory are shown in the following Figure 4.14

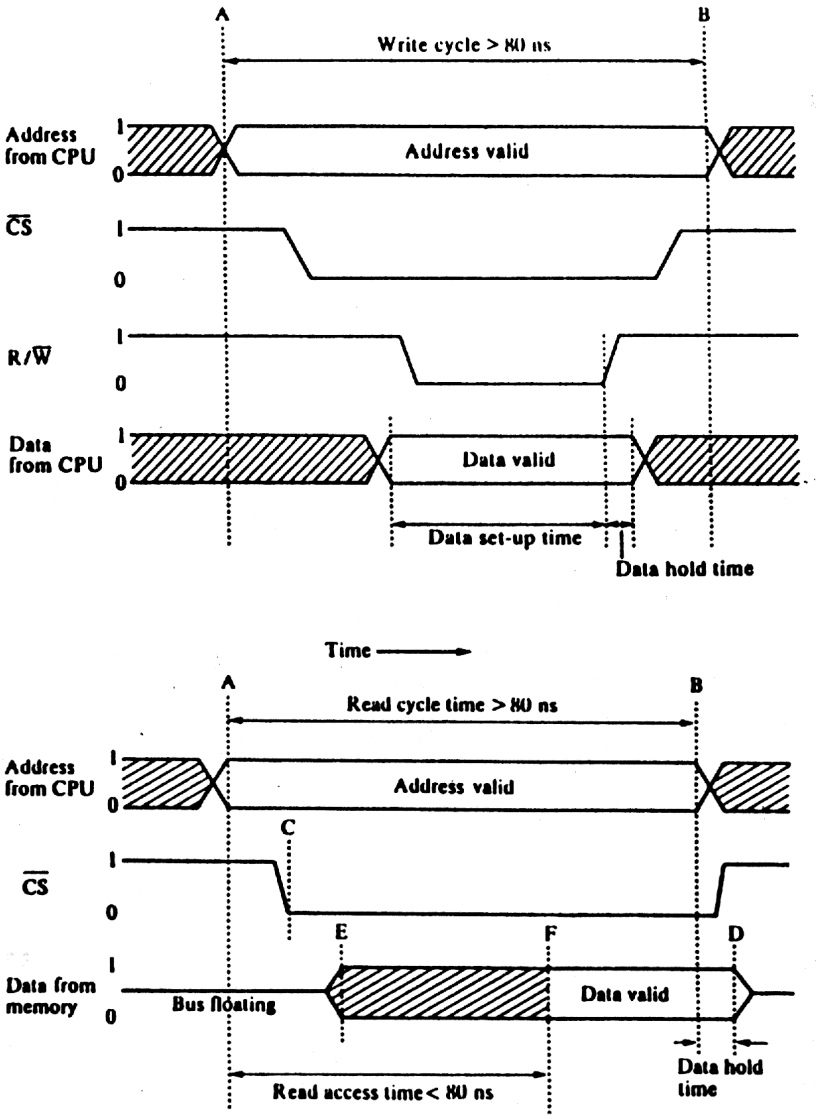


Figure 4.14: Timings for memory write and read

Lecture 5
Control II: A Microprogrammed Controller

With the material covered already, you should now be happy with the fundamental principles of computer architecture. Much of the time is spent shifting words of information around, and now and again the ALU actually does something. It sounds very much like the structure of any sizeable organization.

In Lecture 3 we introduced one method of implementing the control unit, putting off the most common way until this Lecture. This was to give more time for the basics to sink in.

What we are going to look at in this lecture is the design of a *microprogrammed control unit*. You may be puzzled — haven't you been learning about microprogramming? No, the assembler language level (LDA, STA etc) is *macroprogramming*.

What you should take from the lecture is that a microprogrammed control unit is just another computer within the main cpu, a computer equipped with the essentials of some memory, some registers and an inner controller based on a ROM.

5.1 Generating sequences using a ROM

You have already seen in your lectures in Year 1 on Digital Logic that a nearly effortless, and very flexible method of building a sequence finite state machine was to use a ROM, where:

- one part of the contents at a particular address tells you **what to do now**, and
- the other part tells you the address of **where to go next** (see Figure 5.1)

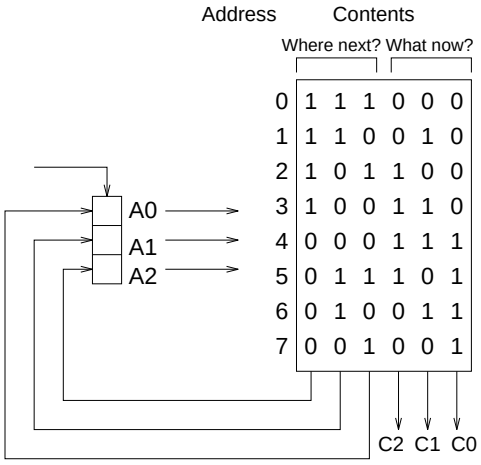


Figure 5.1: A ROM sequencer — in this case an odd and expensive way of counting from 0 to 7.

5.2 From sequencer to controller?

The obvious question is

- can we exploit this idea to build a control unit?

Given that we have a nicely pre-ordered sequence coming out from the ROM sequencer on the lines $C_{0,1,2}$ in Figure 5.1, it should be possible.

What would be the advantages over the random logic approach of Lecture 3? The answer must be flexibility — if you want to change instructions or add instructions you merely have to re-programme a ROM, rather than re-design hardware.

5.3 A first design

5.3.1 Stage 1

Let us first see what we might make of the fetch and execute stages of the LDA x instruction. Suppose we located the start of the fetch sequence at address 0, and the execute sequence of LDA x at address 8 decimal in what we will call the μPROM . (This term μPROM to remind us that this isn't anything to do with main memory. We are inside the control unit remember!)

Decimal μPROM Address	μPROM Contents: Where Next? Dec. Addr	Control signals C_0	C_1	C_2	C_3	C_4	C_5	C_6	C_7	C_8	C_9	C_{10}	C_{11}	C_{12}
0	0001	0	0	0	0	0	0	0	0	0	0	1	0	0
1	0010	0	0	0	1	0	0	0	0	0	0	0	0	0
2	**1000**	0	0	0	0	0	0	0	0	0	1	0	1	0
8	1001	0	0	0	0	0	0	0	1	0	0	0	0	0
9	1010	0	0	0	1	0	0	0	0	0	0	0	0	0
10	0000	0	0	0	0	0	0	1	0	0	0	0	0	0

Table 5.1 Basic sequence for fetch and execute of LDA x

Notice that at the end of the execute sequence, we jump back to μPROM address 0 to start another fetch. This looks promising — though there is an obvious problem. At the end of the instruction fetch, we always jump to address 8 of the μPROM and execute the LDA x instruction. What if the instruction is not LDA x? Clearly we need to generate a μPROM address on the basis of the fetched opcode now lying in the **IR** .

5.3.2 Stage 2

Now we will reorganize things a little. Let us put the fetch sequence at a higher address in μPROM , say location 60 (decimal) and let us consider more of the instructions, say 8 in all, placing their execution signals at addresses (decimal) 8, 12, 16, 20 and so on. In locations 0–7 of the μPROM we will place no control signal entries, but we will place where next addresses — specifically 8, 12, 16, 20 etc. To save on typing, I will replace the signal entries only by the symbols of those which are set.

μPROM Address	Where Next? Dec. Addr	Control signals set	Comment
0	8	—	Jump to address 8
1	12	—	Jump to address 12
2	16	—	etc
3	20	—	
4	24	—	
5	28	—	
6	32	—	
7	36	—	
8	9	C_7	Execute LDA x
9	10	C_3	
10	60	C_6	Now jump to instruction fetch
11			
12	13	C_7	Execute STA x
13	14	C_5	
14	60	C_4	Now jump to instruction fetch
15			
16	17	C_7	Execute ADD x
⋮	⋮	⋮	
20	60	C_8	Execute JMP x, then goto fetch
21	xx	xx	Dont care entries
22	xx	xx	as never here
⋮	⋮	⋮	
60	61	C_{10}	Instruction Fetch
61	62	C_3	
62	**?!**	$C_{11}; C_9$	**HELP! Where next?**
63			

Table 5.2: Revised μPROM allowing several instructions to execute to follow the fetch. There is still a problem obtaining data from the **IR** .

We haven't solved the problem yet, but the solution is clear. If the "where next address" at address 63 of the μPROM were taken direct from the instruction register, the sequencer would effect a jump to the location in μPROM where the relevant execute stage of the respective instruction. For example, if the **IR** held decimal 3, we would go to address 3, jump to address 20, step through the execute of the AND instruction, then jump to 60, step through the instruction fetch. Suppose the **IR** now contained 4, we would jump to 4, jump to 20, execute ADD, jump to 60, and so on.

So all we need to add to the μPROM is an extra bit in the contents, saying

- use the **IR** , not the where-next address

and outside we need a multiplexer.

μPROM Address	Use IR	Where Next? Binary Address	Control signals set	Comment
0	0	8	—	Jump to address 8
1	0	12	—	Jump to address 12
⋮	⋮	⋮	⋮	
8	0	9	C_7	Execute LOAD x
9	0	11	C_3	
10				
11	0	60	C_6	Now jump to instruction fetch
12	0	13	C_7	Execute STORE x
⋮	⋮	⋮	⋮	
60	0	61	C_{10}	Instruction Fetch
61	0	62	C_3	
62	**1**	xx	$C_{11}; C_9$	**Use the IR!**
63				

Table 5.3 The insertion of an extra bit in the contents indicates whether to select the where next address or the address from the **IR** (opcode).

The opcode from the IR supplies the 3 least significant, and the 3 msb's are set to zero. The multiplexer hardware is shown in Figure 5.2.

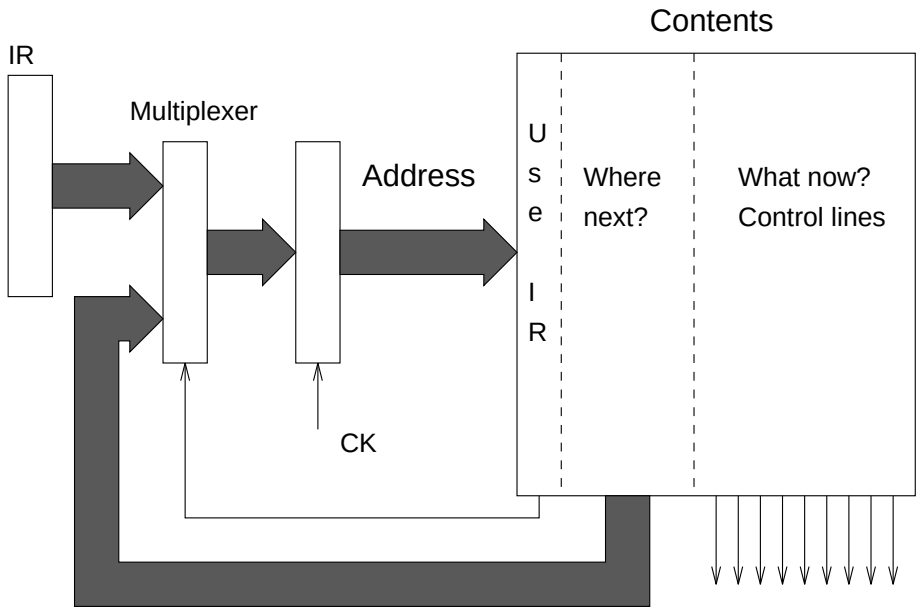


Figure 5.2: Using an IR branch flag.

If our μPROM has 64 locations, we need a 6bit where next address. There are 13 control signals, so we would need a 64×9bit PROM.

5.4 Refining the design

So far so good, but what our controller has so far neglected (and we carefully neglected the problem in Lecture 3 too) are the flags in the status register. Our BEQ instruction needs to use the Z-flag, and in a practical instruction set there are instructions that cause branching on the basis of results from the ALU, such as BNE (branch if negative) BEQ (branch if equal), and so on.

The refinement is shown in the Figure 5.3. The changes (which are more numerous than substantial) are:

1. For operational convenience a register is on the output side of the PROM. By analogy with the macro level, this is called the μIR .
2. The set of registers on the PROM's address lines are rename the μPC , and the assumption made that the most likely address to be needed next is adjacent to the current one. Thus there is no need to specify the address in μPROM , and the μPC is provided with a incrementer.
3. The what-next contents of the ROM need now only be filled if there is to be a jump to a *branch address*.
4. The single bit which chose between taking the address from the **IR** (opcode) or from the μPROM is augmented to 2 bits, known as the load conditions. They define how the μPC should be loaded.
   (a) if 00, never branch. Just increment the μPC .
   (b) if 01, always branch. The address mutiplexer takes the what-next address into the μPC .
   (c) if 10, conditional branch. If the Flag (see below) is set, use the branch address. If it is not, increment the μPC .
   (d) if 11, use the address from the **IR** (opcode).
5. Two new bits are introduced called the Flag Select bits. They decide which of the Z,N,C, or V-flag is to be used as the Flag against which to test for conditional branching.
   (a) if 00, Flag=Z flag.
   (b) if 01, Flag=N flag.
   (c) if 10, Flag=C flag.
   (d) if 11, Flag=V flag.

   All the flags are fed into a 1bit multiplexer, and the relevant one chosen. Is choosing the "relevant" one a problem? No. The programmer chooses the relevant *macroinstruction* which specifies the flag to be tested. Now the microcode for each macroinstruction is fixed, so there is no uncertainty about choosing the two bits.

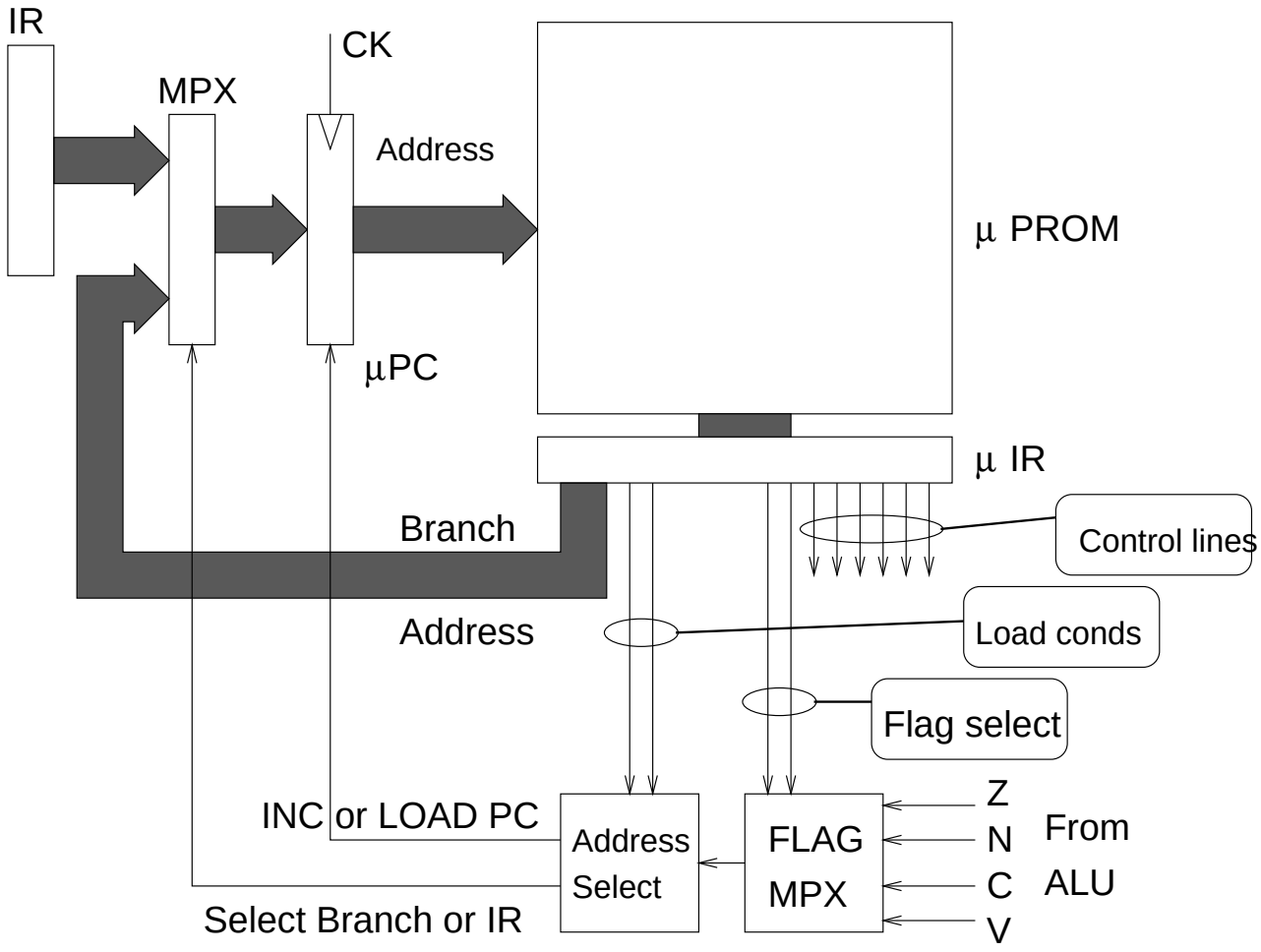


Figure 5.3: A more refined microprogrammed controller

5.5 Summary of lectures 2 to 5

In Lecture 2 we established the basic anatomy of cpu comprising control unit, arithmetic logic unit, and registers, linked to an external main memory via the address bus and data bus.

We saw that program instructions were pulled in, or fetched, to the cpu one by one and were interpreted by the control unit, whose job was to generate the correct pathways in the cpu, and to fire the appropriate register transfers to execute the instructions.

In Lecture 3 we described the execute sequences of a few commands in RTL, and discussed how we might build a control unit from discrete logic or a PLA. (A question on 2A5-3 asked you to explore a design using D-type latches using the RTL design methodology introduced in the Digital Logic lectures.)

Lecture 4 explored the hardware of the items of the cpu, including clocks, straight registers, counting registers, shift registers, decoders, multiplexers, the alu and memory.

This lecture has looked at a fourth method of implementing a controller, widely used by the computer industry because of its flexibility.

The microprogrammed controller effectively acts a computer within a computer.

With the cpu, register and memory hardware understood, you should now be able to write, and should practice writing the RTL and then the microprogram steps for the execute stage of a wide range of assembler language instructions.

Lecture 6
Assembler level support for high level constructs

In Lecture 2–4 we concentrated our efforts on the register level, looking both at the raw hardware and how it enables register transfers, and in Lecture 5 we developed the micro-programming level within the control unit.

In this Lecture we will move up to the macro-level, where we deal in terms of the overall effect of an instruction written in assembler language.

Nowadays, rather few engineers write code in assembler, using instead a high-level language such as C, Pascal, Modula-2, Fortran, and so on, which allows almost total abstraction from the details of the cpu that actually executes the program. Sometimes though if one is controlling machinery or working on something that is speed-critical, one might resort to assembler. Although modern compilers produce very efficient code, there are often always short cuts and dirty tricks that can be taken at the assembler level.

What we are particularly concerned with in this lecture is the way in which the macro-level supports the constructs that the programmer likes to use in a high-level language. We shall look at methods of addressing memory and the use of stacks for writing subroutines.

Throughout we will give examples in a couple of assembly languages. The first is similar to that used in Hill and Peterson for the TB6502 cpu, and the second similar to that used in Clements for the MC68000 cpu. The 68000 has multiple accumulators D0, D1 and so on, as well as multiple address registers. As noted in earlier lectures, the details of the assembly language are not important.

6.1 Memory addressing

Our knowledge of memory hardware tells us that a memory works in just one way — you stick the address on the address lines and then either read or write to the contents at that address.

The different modes of memory addressing refer then not to the hardware level, but to different ways of using what you read from the memory. In the next four sections we shall look at

- immediate addressing
- direct addressing
- indirect addressing
- indexed addressing

The books in the reading list will contain other methods, usually increasingly exotic variations on the last of these.

6.2 Immediate addressing

As you will see, Immediate addressing really doesn't involve further memory addressing after the instruction fetch. It provides a method of specifying a *constant number* rather than an address in the *operand*. For example

LDA #x MOVE #x,D0

loads the accumulator immediately with the operand x.

AC ← **IR** (address)

Looking back at our Standard architecture, you will see that there is a direct link from the **IR** (address) to the **AC** . Remember that although we have assumed a standard 8-bit operand, there is nothing to restrict our machine to this value.

Other instructions can use an immediate operand. For example

ADD #22 ADD #22,D0

indicates that 22 is not an address but a constant to be added immediately to the contents of the accumulator.

In RTL, **AC** ←**AC** + 22

6.2.1 Examples from high-level code

1. These statements of high level code reserve storage for an integer n, and later add 10 to the current value.

```
int n;
...
n=n+10;
```

   Assuming that after compilation the memory address of `n` is `man` (which of course will be replace by some specific location) might get compiled as

```
LDA man
ADD #10
STA man
```

   or

```
MOVE man,D0
ADD  #10,D0
MOVE D0,man
```

   In C, it is possible to request that a number be stored if space permits in a register on board the cpu. The 68000 has a number of accumulator registers connected to the alu, so assuming `n` got stored in register `D7`:

```
register int n;
...
n=n+10;
```

   would get compiled as

```
ADD  #10,D7
```

This obviously allows faster updating.

2. Suppose we were testing a condition such as

```
int n;
...
if( n>7 && n<25) {
     statements A ...
}
else {
     statements B ...
}
statements C ...
```

In assembler we might write this as

```
          MOVE ma_n,D0       Get the number from address nad into D0
          CMP  #8,D0        Compare D0 with 8. This sets the ALU flags on D0-8
          BMI  FALSE        Branch to label FALSE if the N flag is set.
          CMP  #25,D0       Set flags on D0-25
          BPL  FALSE        Branch if positive to FALSE
TRUE      statements A ...
          ...
          JMP  CARRYON
FALSE     statements B ...
          ...
CARRYON   statements C ...
```

Note the "line labels" TRUE, FALSE, CARRYON. As we saw in Lecture 4, and as we would realize here from looking at the JMP CARRYON, CARRYON must be an *memory address*. The assembler converts these labels into memory addresses — the memory locations of the labelled instruction.

6.3 Direct or absolute addressing

We have already use direct addressing in the lectures. This is where the operand is the address of the data you require. Another way of saying this is that the operand is a *pointer* to the data.

LDA x MOVE x,D0

MAR	$\leftarrow$	**IR** (address)
MBR	$\leftarrow$	**M**$\langle$**MAR** $\rangle$
AC	$\leftarrow$	**MBR**

Again, other commands can use the direct addressing mode, for example:

ADD x ADD x,D0

MAR	←	**IR** (address)
MBR	←	**M**⟨**MAR** ⟩
AC	←	**MBR** + **AC**

6.3.1 Example

1. Location 23 should contain 43 after the following:

```
LDA #21
STA 22
ADD #1
ADD 22
STA 23
```

6.4 Indirect addressing

LDA (x) MOVE [x],D0

In indirect addressing, the operand is the address of the address of the data. That is if we look in the memory at address x we don't find the data but rather another address. We then have to look at this new address to find the data.

Alternatively we could say that the operand is a pointer to a pointer to the data.

It is obvious that we need an extra memory access to use indirection:

MAR	←	**IR** (address)
MBR	←	**M**⟨**MAR** ⟩
IR (address)	←	**MBR**
MAR	←	**IR** (address)
MBR	←	**M**⟨**MAR** ⟩
AC	←	**MBR**

The example shown in Figure 6.1 shows LDA (2), compared with LDA 2 and LDA #2. The contents at address 2 are 47, so that we look in 47 for the data, 38, which is then loaded into the accumulator.

6.4.1 Why is indirect addressing used?

Given that indirect address takes more time, why is it frequently used? Books tend to stress different advantages of it.

For example, Hill and Peterson (Chapter 12) note that in some computers it is an advantage to have very small programs. By using indirect addressing, a short opcode (eg in our model 8bit) opcode address can access a 16bit address.

Other books show that it eases the use of pointers in a high level language, allowing complicated data structures to be constructed.

Hayes, however, notes that it is often useful to be able to change the location of x, without changing any of the address fields of instructions (ie operands) that refer to x.

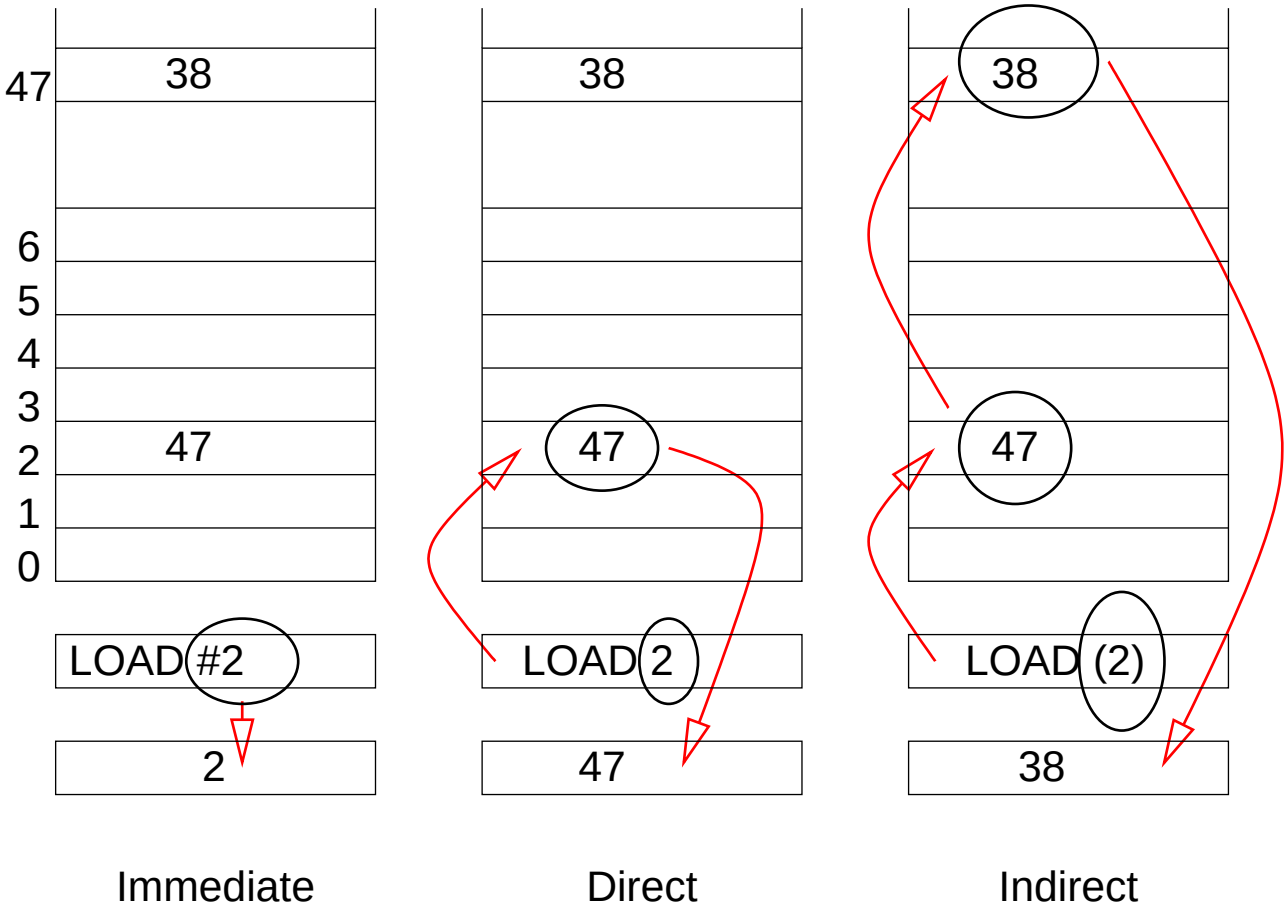


Figure 6.1: Addressing modes

It is this last point that is the crucial one, but it is best understand in the context of compilation of a high-level language. We will return to this later.

For now, let us look at a use of indirect address to access members of an array. An array is a strictly a vector of data that occupy contiguous locations in memory.

Let us suppose that the pointer to the first member of the array is held in address 60. That is, if we look in the memory at address 60 we shall see the address of the first element, not the first element itself. Let us assume that the array starts at address 410, the next element is in 411, and so on. This snippet of code adds up the members of the array.

```
       LDA #410        //Load AC immediate with 410
       STA 60          //Store this number in address 60. =>410 is the pointer
       LDA #0          //Zero the AC
NEXT   ADD (60)        //Add indirect. The contents of address 410 are summed
       INC 60          //Increment the contents of address 60.
                       //I.e. increment the pointer to 411.
       JMP NEXT        //Loop back
```

6.5 Indexed addressing: an example of register addressing

LDA address,offset

One disadvantage of the method given above of accessing arrays is that it requires an extra main-memory read to obtain the pointer, and a extra main-memory write to increment the pointer.

Cpus typically provide a number of registers for temporary storage which avoid the need to hold and access pointers in main memory, with obvious savings in time. Methods which use these registers are known as register addressing modes, of which there are an exotic variety. All are playing an indirection game of one sort or another.

Indexed addressing is a straightforward example of register addressing, and the only one we consider in these lectures. Here, an *index register* holds an offset from a given address, and the effective address is the sum of the two. The index register can be loaded separately, and can be incremented or decremented.

6.5.1 Examples

1. Supposing the register is called the X register, and the start of an array we wish to access is located at 0400 in memory, then the following snippet of code will add the elements of the array. (Note, I have not worried about termination of the loop — you should.)

```
       LDX #0       //set the index register to 0
       LDA #0       //set the ac to 0
NEXT   ADD 0400,X   //indexed addressing
       INX          //increment the index register X
       JMP NEXT
```

2. Another use is the addition of two arrays to create a third as in $c[i] = a[i] + b[i]$. Assume $a[]$ is at 400, $b[]$ is at 500, and the result $c[]$ is to be put at 600:

```
       LDX #0       //set the index register to 0
       LDA #0
```

```
NEXT   LDA 0400,X
       ADD 0500,X
       STA 0600,X
       INX         //increment the index register X
       JMP NEXT
```

6.6 Compilation and more on indirection

Earlier we noted that indirection is useful for some tasks, but there is one use for which it is essential. In the context of a high-level language

> Indirect addressing (in one form or another) is essential if access is to be made to memory allocated at run-time rather than compile-time by a high-level language.

To understand this statement we need first to understand a little more about what happens during compilation of a high-level program.

Before you can execute a high-level program on a computer, it has to be compiled into a set of instructions for the particular cpu being used. In their binary opcodes and operand form, the program is said to be in machine code. One level higher than this is assembler code, which uses mnemonics for the opcodes, and allows one to label locations. Now each cpu comes with its own assembler program which turns assembler into machine code (you will use one of these on the lab course), and so there is no need for a compiler to produce machine code: it need only produce assembler.

The aim, in the simplest cases, is to produce a program where instructions occupy one section of memory, where allocated data memory lies above the instructions, and where there is further unused memory above that.

The compiler works in two stages. On the first pass, it reads the file of high level statements, checks the syntax, and replaces each statement with the relevant sequence of assembler instructions.

Another thing it does is to place declared variables, and the names of labels, into a *symbol table*.

During the first pass, each time the variable or label is encountered, its name is replaced in the code by its location in the symbol table. The instructions are written out into a temporary file.

Since each instruction is of known length, the compiler can associate with it provisional memory addresses, relative to some arbitrary offset. So, after the first pass, the compiler knows the full extent of the program — or more strictly the instructions.

At this stage therefore it knows the locations where it can start placing allocated data. On the second pass it can replace locations in the symbol table by proper locations in memory.

For example, consider the fragment

```
int a,b,c;
a=1;
b=2;
c=a+b;
if(c!=0) {
   a=3;
}
b=4;
```

After reading the declaration of variables, the compiler could start making the symbol table, where BOD indicates the unknown Beginning Of Data.

Name	SymTab	Actual Location	Words
a	var0	BOD	1
b	var1	BOD+1	1
c	var2	BOD+2	1

Let the beginning of the program be located at BOP, and let us count in words. The compiler would start making its temporary file:

Address	Instruction
BOP	LDA #1
BOP+1	STA var0
BOP+2	LDA #2
BOP+3	STA var1
BOP+4	ADD var0
BOP+5	STA var2
BOP+6	BEQ lab0

At this point, the compiler knows that the label will be just after the closing bracket, but doesn't know where that is. So it adds an entry:

Name	SymTab	Actual Location	Words
a	var0	BOD	1
b	var1	BOD+1	1
c	var2	BOD+2	1
	lab0	?	-

Address	Instruction
BOP	LDA #1
BOP+1	STA var0
BOP+2	LDA #2
BOP+3	STA var1
BOP+4	ADD var0
BOP+5	STA var2
BOP+6	BEQ label0
BOP+7	LDA #3
BOP+8	STA var0
BOP+9	LDA #4
BOP+10	STA var1
BOP+11	HALT

At the LDA #4 instruction, the compiler knows that lab0 is actual BOP+9, so

Name	SymTab	Actual Location	Words
a	var0	BOD	1
b	var1	BOD+1	1
c	var2	BOD+2	1
	lab0	BOP+9	-

At the end of the program, the compiler knows that the beginning of data BOD = BOP+12. It then rewrites the table as:

Name	SymTab	Actual Location	Words
a	var0	BOP+12	1
b	var1	BOP+13	1
c	var2	BOP+14	1
	lab0	BOP+9	-

There are now two possibilities for the second pass. The compiler can either rewrite var0, var1 and var2 and lab0 leaving BOP as a variable to be filled in at load time, or it can set a value a value for BOP. Choosing the latter with BOP=100 (dec), we end up with:

Address	Instruction
100	LDA #1
101	STA 112
102	LDA #2
103	STA 113
104	ADD 112
105	STA 114
106	BEQ 109
107	LDA #3
108	STA 112
109	LDA #4
110	STA 113
111	HALT
112	0
113	0
114	0

Now simply replace the opcodes by the relevant binary, and we have executable machine code. Notice that the assembler has to use a different opcode for LDA, LDA #, and so on.

6.7 Back to indirect addressing

We should now be in a better position to understand why indirect addressing is essential for run-time memory allocation.

In the compilation example we allocated memory for three 1Word integer quantities at compile time.

```
int a,b,c;
...
```

This meant that the compile could work out exactly what memory it should reserve, and how to address it in the instructions.

Now suppose the high level programmer wishes to reserve some memory for two 1D arrays (or vectors) of 1Word objects. The programmer might declare

```
int array1[26];
int array2[26];
array1[0] = 'a'; array2[0] = 'A';
array1[1] = 'b'; array2[1] = 'B';
etc
```

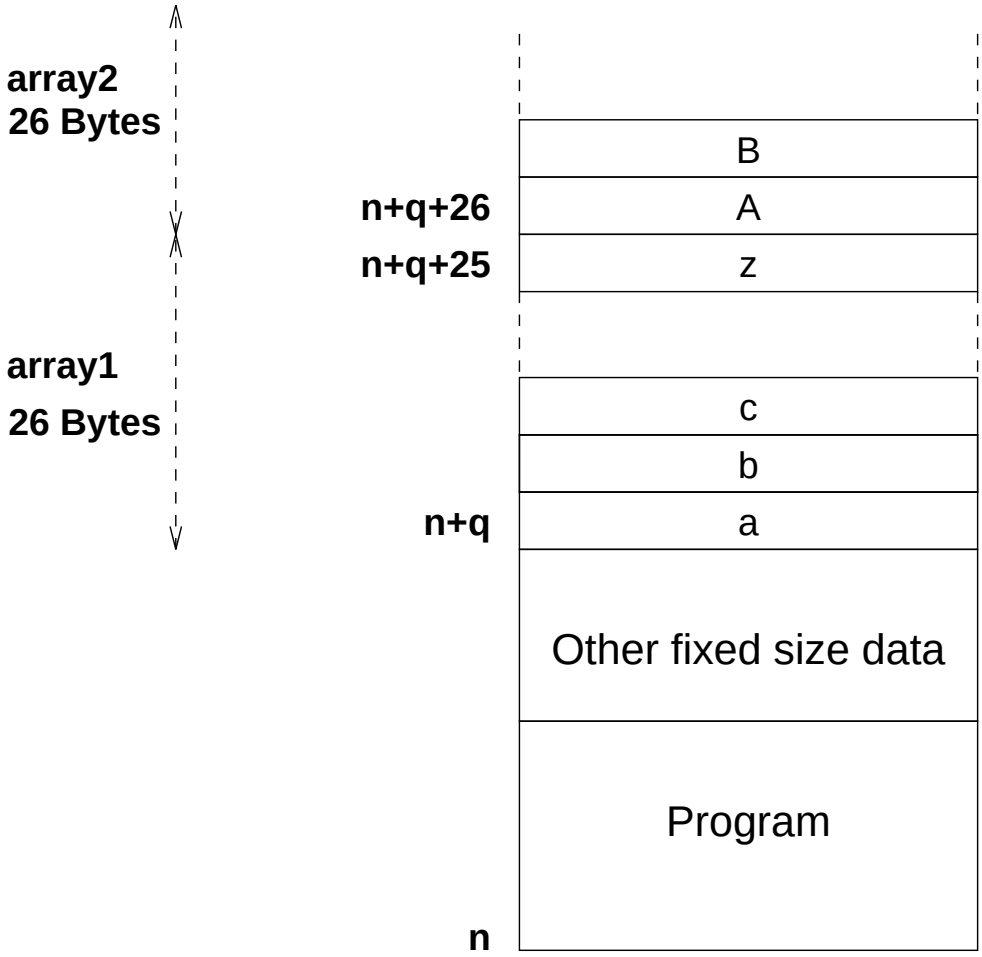


Figure 6.2: Addressing modes

The compilation stage will reserve space in the data section for the strings. Because after the 1st pass the compiler knows the size of program and all data it can work out how to access the arrays. Referring to Figure 6.2, if the base of the program is at address n and the size of the program and other fixed-size data is q, then the first element of array1, array1[1] would be referenced as

array1[1] $\leftrightarrow$ **M**$\langle$**n+q+1**$\rangle$

and the 0th element of array2 by

array2[0] $\leftrightarrow$ **M**$\langle$**n+q+26**$\rangle$

and so on. In conclusion, the program knows where the arrays are in memory after the compilation stage.

Suppose however that the sizes of array1 and array2 could not be determined until the program was running. For example, we might open a file which told us the size of the arrays, then supply the data.

How could the program refer to array2[0]? It couldn't use **M**$\langle$**n+q+28**$\rangle$, because when it was compiled the size of array1 was unknown.

Now one might imagine a mechanism that during program execution could rewrite the program, replacing unknown addresses. This is a recipe for chaos, and not done. Indeed conventional high-level languages would not allow changes to the *program* during execution (ie changes to opcodes and operands).

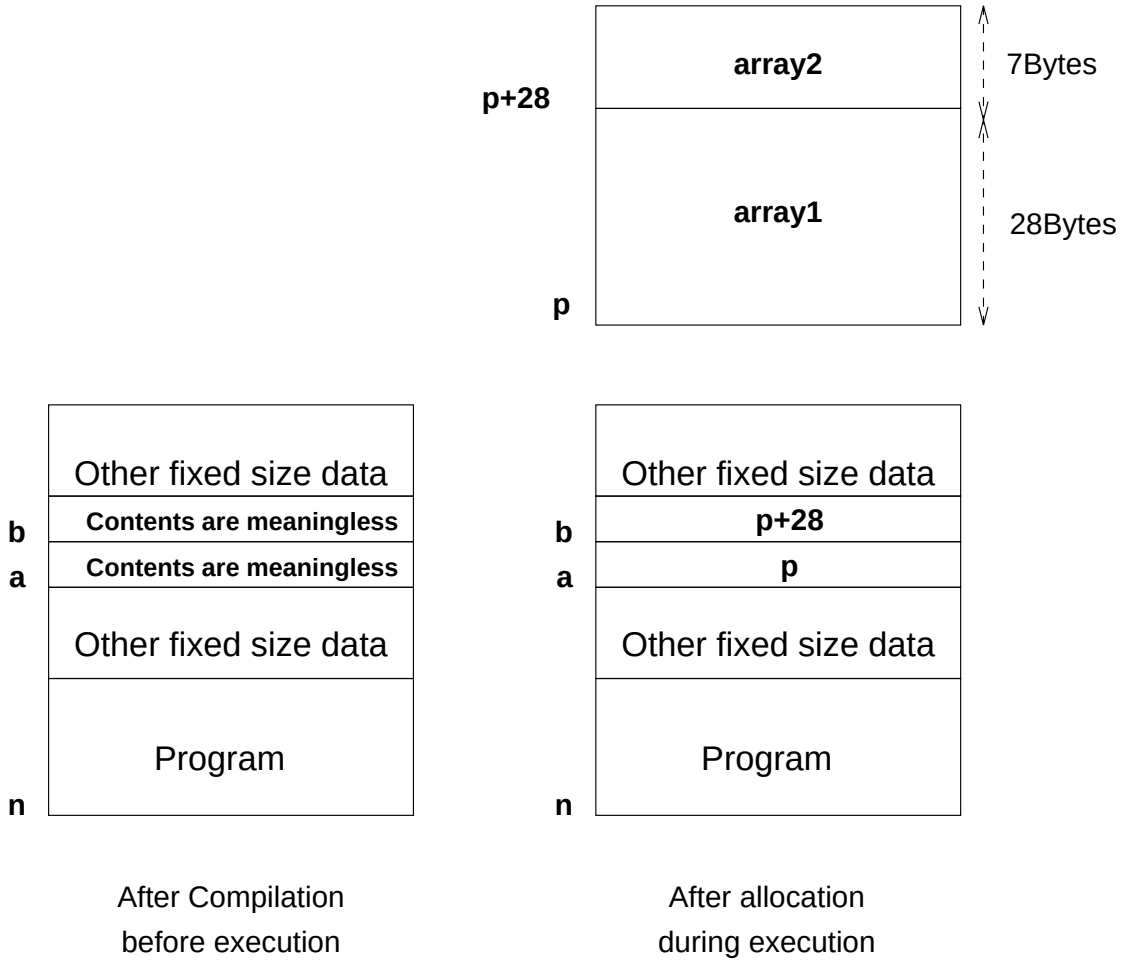


Figure 6.3: Memory allocated at run-time must be referenced indirectly

We are however allowed to change *data*. The solution therefore is the declare at compilation two pieces of storage that will hold the *addresses* of the arrays array1 and array2 *when they become known*. The addresses a and b of these pointers are known at compilation time and are therefore available to the program. During execution, as shown in Figure 6.3, when the memory for the arrays is allocated, the addresses p and $p + 28$ are placed in a and b. By using indirection, the program need no "know" what p is. For example, the i-th element of array1, ie array[i], would be accessed indirectly as $\mathbf{M}\langle\mathbf{M}\langle a\rangle{+}i\rangle$, and the i=th element of array2 as $\mathbf{M}\langle\mathbf{M}\langle b\rangle{+}i\rangle$.

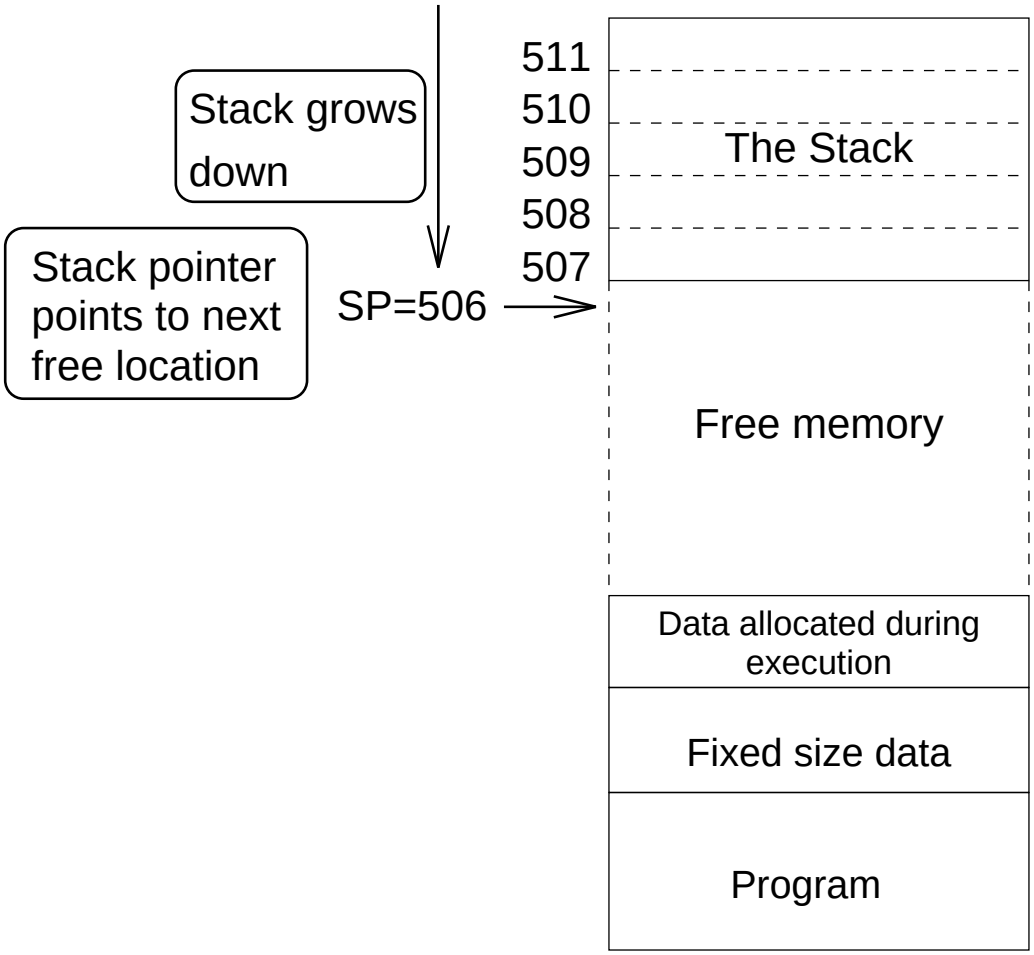


Figure 6.4: A stack

6.8 Stacks and Subroutines

Looking back at the Bog Standard Architecture, you will recall that there is a register called **SP** , the stack pointer. This register holds the address of the entrance to an area of memory reserved by a program as temporary storage area. The stack pointer uses memory as a last-in, first-out (LIFO) buffer. Usually it is placed at the top of memory, as far away from the program as possible.

Figure 6.4 shows a model of memory. The stack currently contains 5 items and grows downwards. The stack pointer points to the next free location. Two commands work the stack, PUSHing (PHA) and PULLing[1] (PLA).

PHA

MAR	←	**SP**
MBR	←	**AC**
M⟨**MAR** ⟩	←	**MBR** ; **SP** ←**SP** -1

The accumulator gets pushed onto the stack at the address pointed to by the stack pointer. The stack pointer is then decremented.

[1] often called POPing

PLA

SP	←	**SP** + 1
MAR	←	**SP**
MBR	←	**M**⟨**MAR** ⟩
AC	←	**MBR**

The stack pointer is incremented and the contents pointed to transferred to the accumulator.

6.8.1 Using the stack during a subroutine

One of the most useful constructs in a high level language is the *subroutine*. This allows the programmer to modularize code into small chunks which do a specific task and which may be reused. For example,

```
main() /* main program */
{
...
xcomp = 4;
ycomp = 2;
mod   = modulus(xcomp,ycomp);  /* subroutine call */
...
}

modulus(a,b)    /* subroutine specification */
{
msq = a*a + b*b ;
m = sqrt (msq ); /* the subroutine calls a subroutine! */
return(m);
}

sqrt (x)         /* most likely gotten from a library */
{
... blah ...
}
```

Just imagine what a pain it would be if each time we wished to evaluate the sqrt() of a number we had to write the code in-line.

When we come to compile into assembler what happens to the subroutines? One thing that could happen[2] is that the compiler writes the appropriate assembler instructions in-line. However, assembler languages support the use of subroutines.

From our discussion of compilation you know that the instructions in the different routines will end up in different parts of the program. To call a subroutine, we obviously need to

- jump to the subroutine's instructions
- transfer the necessary data to the subroutine.
- arrange for the result to be transferred back to the calling routine.

[2] and for speed sometimes does

Is this enough? When you consider transferring back to the calling routine, the answer has to be no! How do we know where to jump back to? What if the subroutine has messed up the registers? There is an obvious need to

- store the status quo before jumping and restore it after jumping.

Before going further, it is important to realize that there are two fundamentally different ways of treating the parameters passed to a subroutine — by *value* or by *reference*. The language determines which is used. In C parameters are called by value, whereas in Fortran they are called by reference. Passing value means that a copy of the value of the parameter is transferred. This means that even if the subroutine changes that value, when a subroutine returns, the value in the calling routine is unchanged. When a parameter is passed by reference, the *address* of the parameter is passed, so that the subroutine works with the original data. If it changes the value, that change is seen by the calling routine on return from the subroutine. We will continue assuming calling by value.

Let us consider the assembler for a subroutine where there are no parameters:

```
                \\CALLING ROUTINE
...
JSR MYSUB       \\ JUMP TO SUBROUTINE
ADD whatever ...



MYSUB  LDA whatever  \\ SUBROUTINE STARTS
       ...
       ...
       RTS           \\ RETURN FROM SUBROUTINE
```

Notice that the subroutine starts at labelled address, so JSR is very like JMP. The difference is that we have to get back after the subroutine. After the fetch, the program counter is already incremented to point at the next instruction in the calling program (ADD in this example). So in its execute phase, JSR pushes the current value of the **PC** onto the stack, and then loads the operand into the **PC** , causing the next instruction fetched to be the first in the subroutine. The RTS command ends the subroutine by pulling the stored program counter from the stack. Because the stack is a LIFO buffer, subroutines can be "nested" to any level until memory runs out.

Missing out the detail the JSR and RTS instructions perform the following:

JSR

M⟨**SP** ⟩	←	**PC**
PC	←	**IR** (address); **SP** ←**SP** +1

RTS

SP	←	**SP** -1
PC	←	**M**⟨**SP** ⟩

When the subroutine has parameters, we also have to worry how to transfer the parameters to the subroutine. There are various ways, described in Clements §6.5 and Hill and Peterson § 15.2. Here we mention just one method which again uses the stack. Two methods are

- to pass parameters on a reserved area of memory
- to pass parameters on the stack

In the latter method, the calling routine pushes the parameters onto the stack in order, then uses JSR which of course pushes the return PC value onto the stack.

Figure 6.5 shows a very correct way of using the parameters from the stack by popping and pushing. The return **PC** is pulled and stored temporarily, and then the parameters are pulled, and the return **PC** pushed.

The problem with this is that it is very time consuming. A more efficient method is to use the parameters by indexed addressing relative to the stack pointer, **SP** . That is, the subroutine would access parameter 1, for example, by writing

```
LDA SP,2
```

When the subroutine RTS's it will pull the return **PC** off the stack, but the parameters will be left on. However, the calling routine knows how many parameters there were. However, rather than pulling them off, it can simply increase the stack pointer by the number of parameters (ie, by three in our example). This leaves the parameters in memory, but as they are now outside the stack, they will get overwritten on the next PUSH.

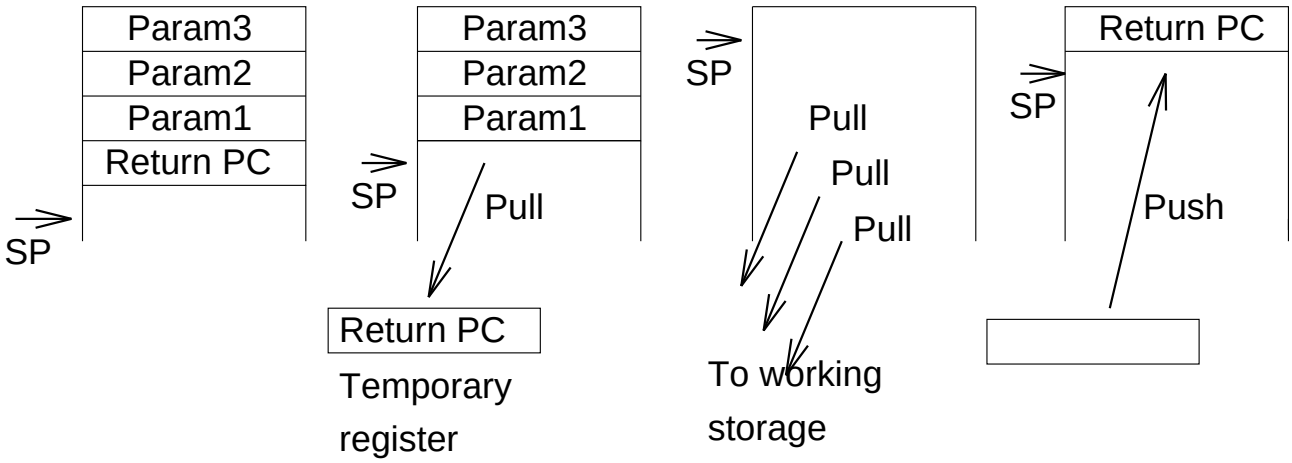


Figure 6.5: Passing parameters on the stack. Their use in this example is "polite and proper" but inefficient.

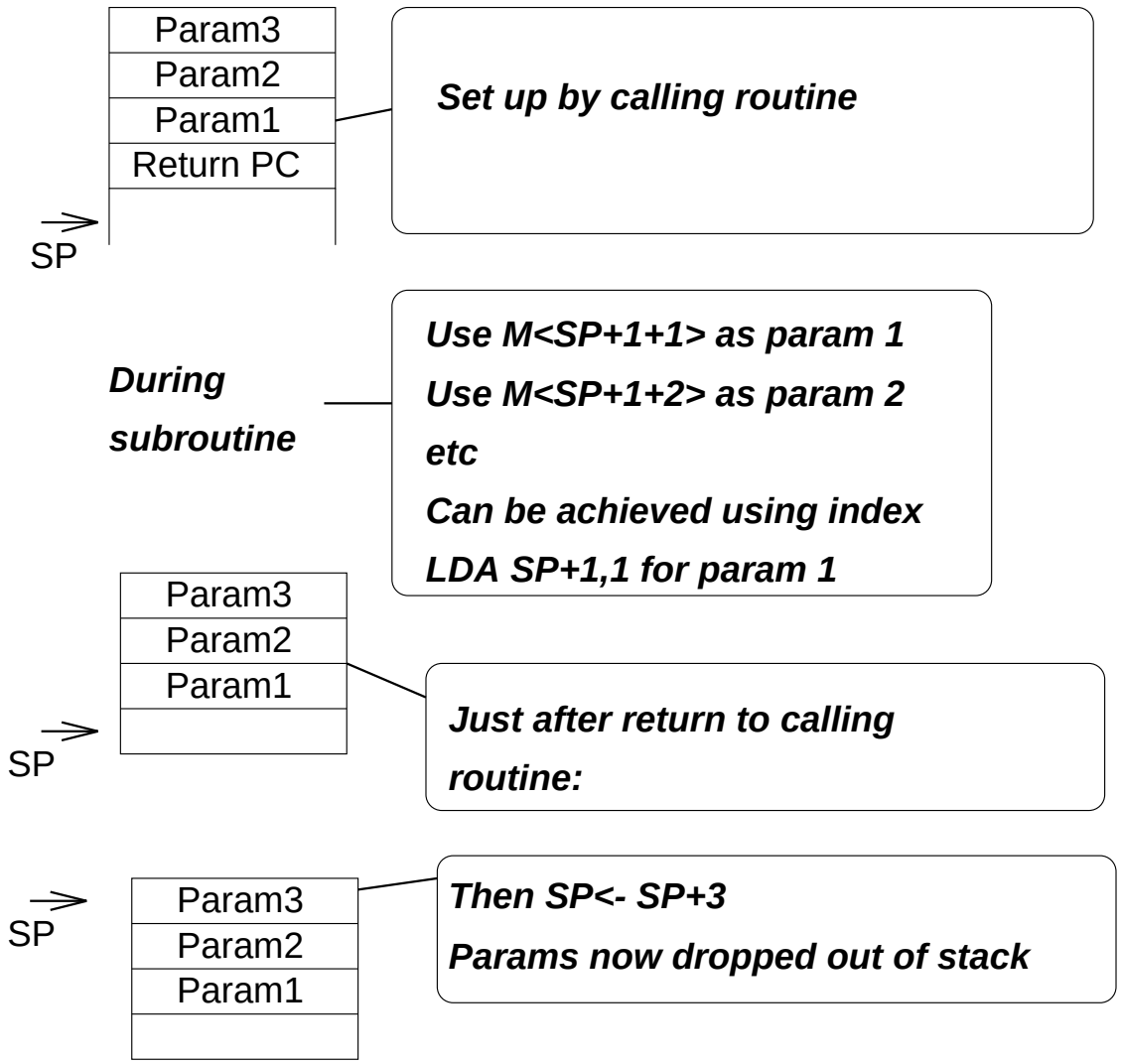


Figure 6.6: More efficient use of the stack

Lecture 7
Memory hierarchies

7.1 Cache memory

In Lecture 4 we noted that the large main memory in a computer was usually made from relatively inexpensive but relatively slow Dynamic RAM (DRAM). However, in addition, there is often a *cache* of fast Static RAM (SRAM). Although the cache is relatively small (say $\sim$ 1% of the main memory size) — it has a very significant effect on the performance of memory accesses.

The cache memory, shown in Figure 7.1, works by copying parts of the main memory to itself. Its controller intercepts an address requested by the cpu, determines whether it is in the cache or not, and then either declares a hit allowing the data to be recovered from cache, or a miss requiring it to be fetched from main memory.

We shall therefore look at

- the method used by the controller to determine if an address is in the cache.

Obviously a small cache memory will only have an appreciable affect on memory performance if the locations that are most frequently accessed are in the cache. So we shall

- the method used to decide what should be in the cache.

We will look at a *directly mapped cache*.

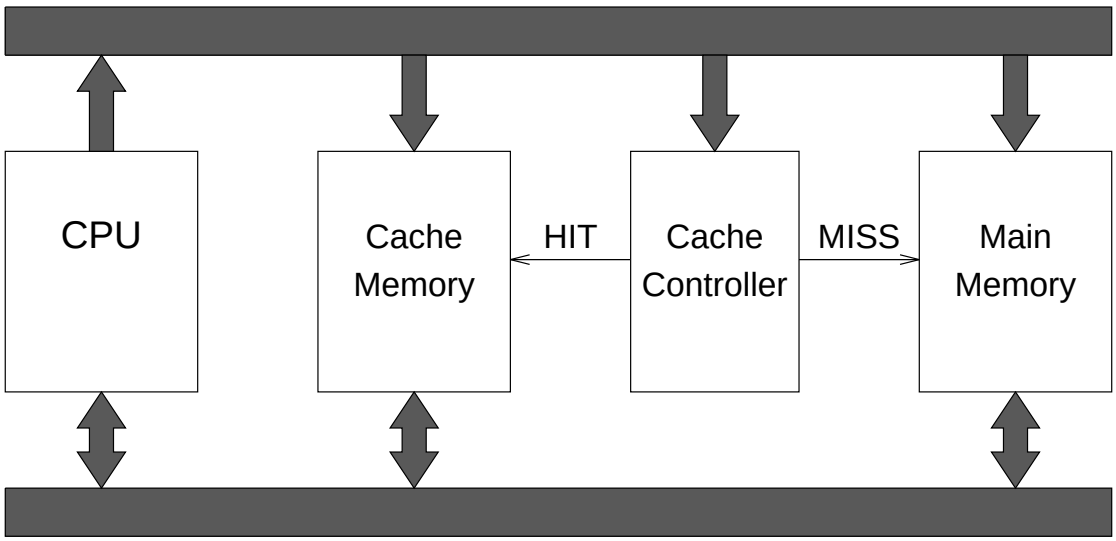


Figure 7.1: A cache memory and controller

7.2 The directly mapped cache

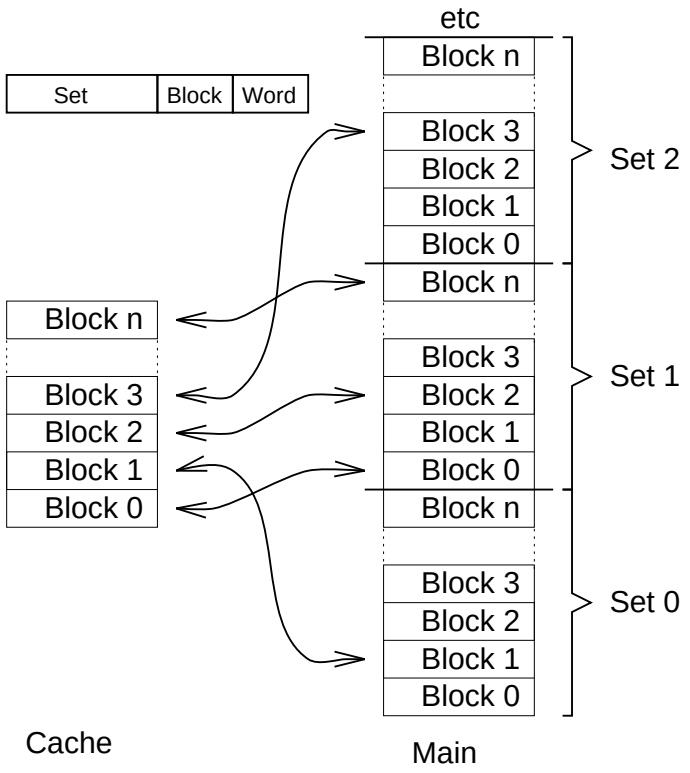
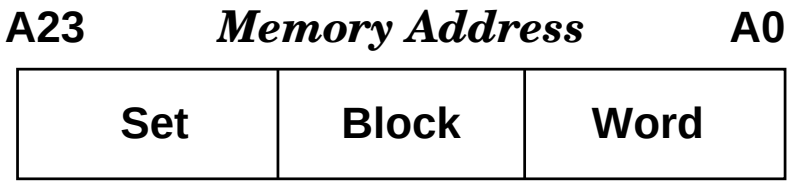


Figure 7.2: Blocks and sets

The cache is divided up into a number of memory *blocks*, each containing a number of *words*. The main memory is many times the size of the cache, and so we partition main memory into cached-sized chunks called *sets*.

A memory address is therefore divided up into three parts

- the LSBs which indicate which word in a block;
- the middle bits, which define which block in a set; and
- the MSBs which define which set.



First, we note that the controller need not worry about the LSBs — a block is the least significant unit of memory in the cache.

Now each block (0,1,2,. . .) in the *cache* is required to come from a block with the same number in the main memory. That is, block 0 in the cache **must** correspond to a block 0, from one set or another, in main memory, block 1 in the cache **must** correspond to a block 1, and so on, although the blocks themselves can come from different sets.

We see from this that

- the cache is not completely flexible
- the cache controller need only know which *memory set* the block came from — once it knows the set, it knows the set and the block.

7.2.1 Hit or miss?

To resolve to which set a block in the cache corresponds, the controller maintains a *cache tag ram* where

- The address into the tag ram is just the block address
- the contents are the set number

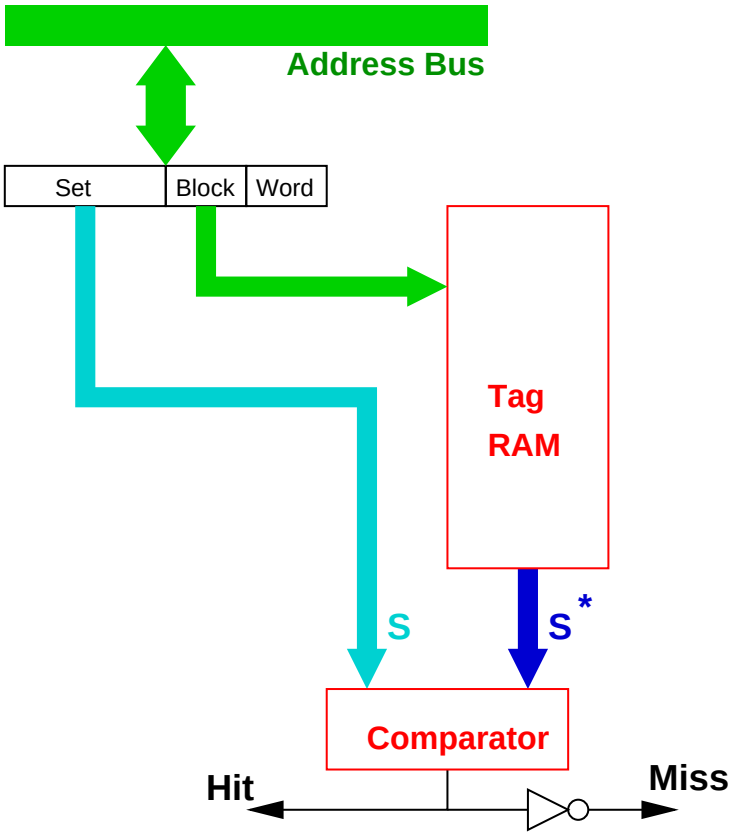


Figure 7.3: The tag ram

To determine hit or miss

- the controller takes the set and block parts of the address from the cpu, S and B say.
- It addresses the tag ram with B and recovers the contents S^*.
- If $S = S^*$, tested using a comparator, we have a hit, otherwise a miss.

7.2.2 Updating the cache

If a miss occurs, the required block is recovered from main memory: the required word forwarded to the cpu, and the block placed in the cache at block B, and the tag ram contents at address B changed to S.

At first sight, both the addressing scheme, and the updating scheme seem strange. Is not the fact that we cannot have block B from set S_a and block B from S_b together in the cache at the same time a major restriction?

Yes and no. Yes because one could image accessing data in such a way that the cache failed all the time. No, because, as our lecture on compilation indicated

- computation usually occurs on data clustered locally in memory.

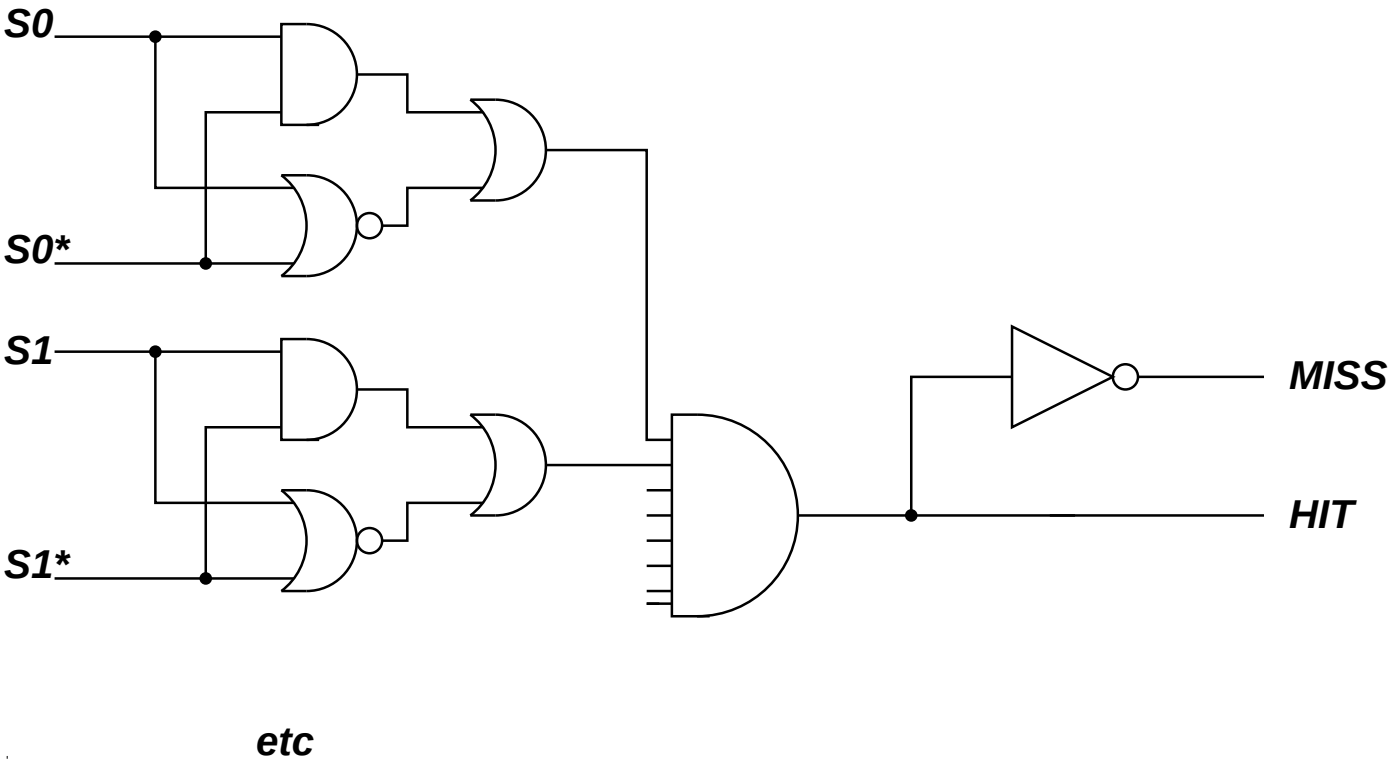


Figure 7.4: A comparator.

7.3 Hit rates and speed ups

If the access times to main and cache memories are t_m and t_c, respectively, and the hit ratio is H, then the average access time is

$$t_{ave} = Ht_c + (1 - H)t_m = H(t_c - t_m) + t_m \tag{7.1}$$

The speed up with the cache then is

$$s = \frac{1}{H(k-1)+1}$$

where $k = t_c/t_m$ is around 0.2 to 0.1. The graphs show the dependence of speed-up on hit rate for a cache 10 times faster than main memory, and one an (unreasonably) impressive 100 times faster. The point here is that the

graphs do not have the same shape: replacing a cache with one 10 times faster does not give you a 10-fold speed up. You must increase the hit-ratio.

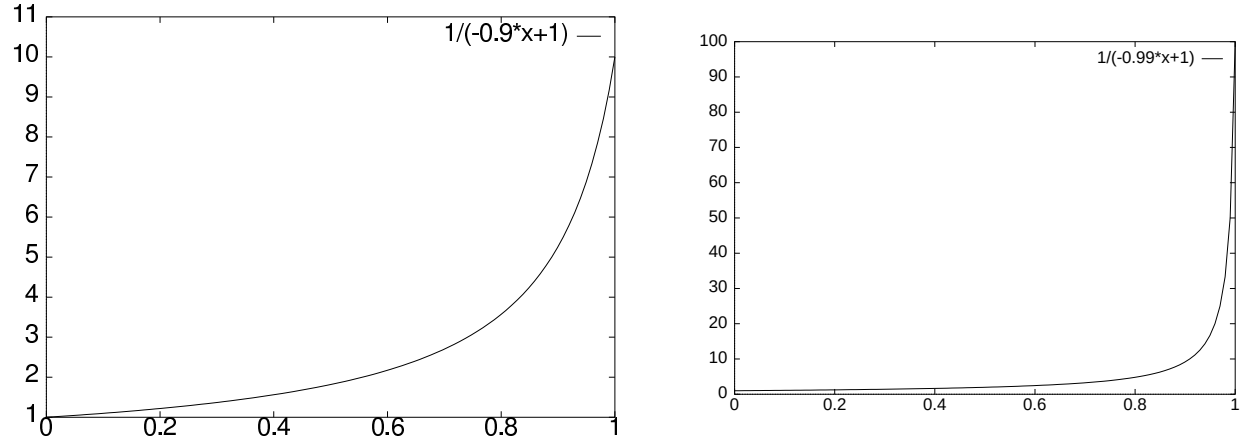


Figure 7.5: Speed up versus hit-rate for $k = 0.1$, $k = 0.01$.

7.4 Virtual memory

The idea of using a fast small cache memory to hold as much of the current working set of data from a larger slower memory is very similar to the idea of virtual memory.

Here the main memory holds the current working set of a much larger memory. This larger memory is held on disk.

It is useful to distinguish two address spaces

- a *logical* or *virtual address space* and
- a *physical address space*

The cpu deals in logical addresses using the full width of the address bus. For example, a 32bit address bus allows logical addressing of 4GBytes of memory. The physical address refers to an actual address in main memory, which may be only a few MB in size.

Memory is divided up in *pages* (rather than blocks in the cache) and each page of physical memory can map onto any page of logical memory (which is a more flexible approach than a directly mapped cache). A *page table* (the analogue of the tag ram) is maintained which describes the mapping between logical and physical addresses. When the cpu request an logical address, the relevant page is determined, and the page table read to see whether that page is in physical memory, and if so at what physical address. If it is not in memory, the page table indicates where it is on the disk. The page is read from disk and installed in physical memory. Of course this displaces a page of memory which has to be written back to disk, and an algorithm determines which is the best one to remove — the least used, or that used least recently, are obvious choices. The need to access disk is called a page fault.

Just as a cache was a method of speeding up access to main memory, one might regard main memory as a method of speeding up access to disk. This perspective indicates that one has to be careful about the hit rate. Whereas DRAM is an order of magnitude slower than SRAM, disk is some *six orders of magnitude* slower than DRAM, and so $k = t_m/t_d$ can be all but ignored, giving

$$s = \frac{1}{1 - H}$$

so the speed is dominated by the hit rate. The size of page and number of pages in a memory are important factors in maintaining a high hit rate. Again success requires data to be *localized*, so that program does not repeatedly generate page faults and lead to disk thrashing.

The use of virtual memory is important in multiuser or multitasking systems, allowing several users to feel they have access to large memories. The topic is discussed in depth in Tanenbaum's book.

An important feature of the hierarchy built from cache, main memory and disk is its transparency to the user. The hierarchy can be extended further to slower online storage media such as CD Roms. The idea here would be that when accessed, large quantities of material would be brought onto faster disks, but would "decay" away if unused over a period of hours.

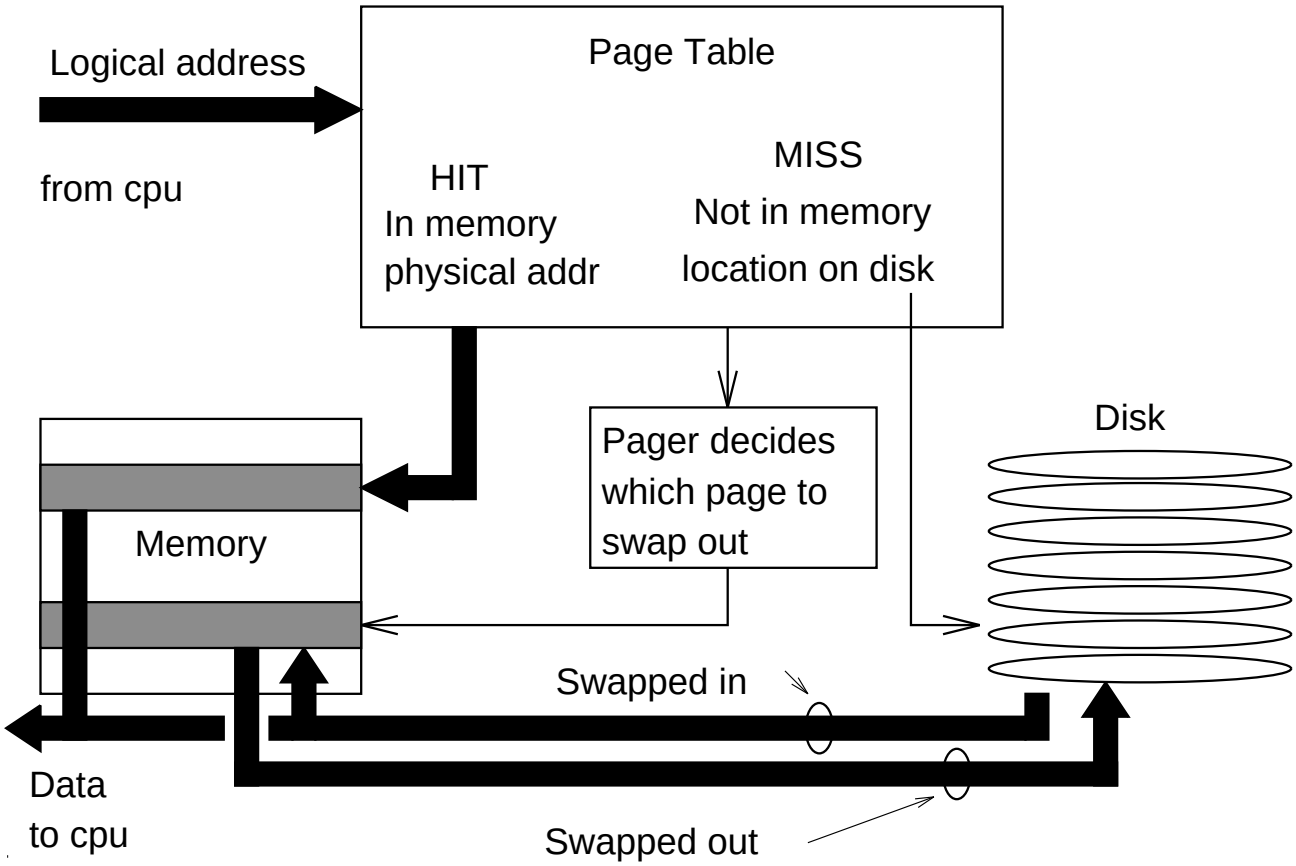


Figure 7.6: Page swapping in a virtual memory environment.

Lecture 8
Re-designing the cpu

Throughout this course we have stressed the importance of the controller in the central processing unit. In lecture 3 we designed a controller in a couple of ad hoc ways (discrete logic, then a PLA). In the 3A3D you designed an RTL control unit for three of four simple instructions. In lecture 5 we introduced the microprogrammed controller, making the point that a ROM where some of the outputs were fed-back via delaying D-type latches was a flexible, "low-thought" way of designing a sequencer, and one especially appealing to a manufacturer would might wish to upgrade the instruction set without designing new hardware.

In this lecture we first consider the need for a re-appraisal of the so-called Complicated Instruction Set Computer (CISC machine) and why it has now been superceded by the Reduced Instruction Set Computer, the RISC machine.

8.1 The need for re-appraisal

We have suggested that flexibility was a contributory factor to the rise of microprogramming. It was indeed the case that the flexibility of the μPROM led to upgrades of the instruction set, which to the hard-pressed assembler language programmer must have seemed a good thing. The Motorola 68000 is a good (or a bad!) example. In our lectures we have used simple examples of the MOVE instruction, but look at this:

```
MOVE 12(A3,D0), D2
```

This adds the contents of the address register A3 to the data register D0, adds 12 to the result, uses the whole as an address, and loads the contents of that address into the data register D2. It does everything except make breakfast.

However although flexibility was the reason microprogramming still lives, it was not the reason it was born. The reason was (of course) based on **sound engineering design**.

Recall that the microprogrammed controller is nothing short of a little computer within a computer, albeit without a alu. Why go to the trouble of building a wheel within a wheel? The response to the mechanical analogy is gearing.

In the 1970's main memory (ferrite core memory) was slow — typical access times were 1μs, some 10^2 times slower than modern memory. It made sense *at that time* to make one macro instruction (ie one instruction called from main memory) to do as much work as possible using the smaller μPROM which could be made of more expensive faster semiconductor memory. With faster main memories is that still the case.

Another sound reason to use main memory as little as possible, was the von-Neumann bottleneck, the low band-width path between memory and cpu. In an 8 bit machine, to get an opcode and a decent operand might take several memory reads. (For example in Q1 of 3A3E it takes 2 reads of the memory — that is why the PC increases by 2.) By decent, we mean with enough width to hold a operand which can address a memory of affordable large size.

Here relatively modest improvements pay enormous dividends, because the address space rises as the 2^n. With the advent of 32bit data busses it suddenly became possible to address inaffordably large memories with one

instruction fetch (or memories a mere 16.8 million times as large as with 8bit machines). Again, one can ask whether the approach to cpu design is correct under this new regime.

The third reason for re-appraisal comes from a greater appreciation of what a computer spends its time doing. Throughout the lectures we have seen again and again that most of the time the cpu is just shovelling information around from one register to another, rather than *computing*.

Clements quotes the figures of Fairclough:

Instruction Type	% of time
Moving (ie Load and Store)	45.28
Jump, call sub, return from sub	28.73
Arithmetic	10.75
Compare	5.92
Logical	3.91
Shift	2.93
Bit twiddling	2.05
IO	0.44

Whether one believes these figures (do you really believe the number of significant figures; and the IO figures look suspisciously low — perhaps much of it is accounted for in moving), the fact that around three quarters of the time the cpu is moving and jumping suggests that a great deal of care should go into the design of methods for moving and jumping.

Researchers at IBM and UC Berkeley were the originators of the re-think on design, and the latter coined the misleading term RISC. As Clements notes you must guard against the idea that RISC machines are somehow stripped down CISC machines: they are highly designed and highly optimized processors. (An analogy might be that RISC is F1 racing and NOT saloon car racing!)

8.2 Characteristics of RISC

1. RISC processors should have sufficient on-chip registers to help overcome the worst effects of the von-Neuman bottleneck. Procedures require on average around 12 words for local storage and parameter passing, and so on-cpu registers are easily feasible.

2. Procedure calls are so common that an efficient method of parameter passing must be implemented in specialized hardware, and not via the stack or other main memory.

3. The processor should execute almost one instruction per clock cycle.

4. RISC processors should not implement infrequently used instructions. They waste space in silicon, are inevitably the slow, complex ones, and can anyway be made from the frequently used instructions.

5. RISC machines should not be microprogrammed.

6. The machine should have a single instruction format to simplify instruction decoding in hardware.

7. There are no multilength instructions (ie those requiring multiple memory reads). Therefore the instruction length must be sufficiently large not to require these. This however means that some instructions will waste memory. (But memory is cheap, so we don't care.)

8.3 Special hardware for subroutines

The reason for so much jumping is that programmers like to write *procedures*.

We saw in Lecture 6 that a function or subroutine call involves the stack, pushing and pulling parameters of which there may be many. The stack you will remember is in main memory.

In Lecture 6 we also saw that subroutines could be nested to any depth — or rather any depth until the stack runs out of space. In practice this is any depth.

An obvious way of reducing time spent on subroutines is to install special hardware replacing the stack with some registers on the cpu. But how much storage is required?

Again statistical analysis of the run-time operation of programs gives some clues.

- **What depth of subroutine nesting caters for most programs?**
  Answer: on average 5. The Berkeley RISC architecture allows 8.
- **How much storage is needed by subroutine for its *local* storage and parameters?**
  Answer: 32 $\times$ 32bit words seems to cater for 97% of the cases.
- **How much global data does a subroutine access?**
  Answer: of order 10 $\times$ 32bit words.

What the statistical analysis shows is that a relatively modest number of on-chip registers can remove the need to access main memory.

So, we have seen that each procedure requires of order 10-30 words of storage, and that procedures are nested on average to a depth of 5-10, and so somewhere between 50–300 registers are required.

The Berkeley cpu was endowed with 138 registers. This seems rather fewer than the suggested decent maximum of 300, but the register were made available to procedures using a technique called *windowing*, which uses them efficiently allowing 8 levels of nesting before recourse has to be made to main memory.

To understand this idea, we must first revise memory allocation for procedures.

8.3.1 Memory storage for procedures or subroutines

The first three we have seen already, but the last is new to us.

1. Local space is address space private to a particular procedure.

2. Imported parameter space holds the parameters imported to the current procedure from its calling procedure or parent.

3. Exported parameter space holds the parameters exported by the current procedure to a procedure it is calling — a child procedure.

4. Global space is a region of address space that is accessible by all procedures.

How do these relate to a bit of high level code?

```
int max_number=100;             /* GLOBAL memory declaration*/
main()
{
    int i,j,k;                  /* LOCAL memory  declaration */
    for(i=0;i<max_number;i++) { /* using GLOBAL */
        j=i+1;
        k=product(i,j);         /* calling child subroutine */
    }
}

int product(int a, int b)       /* procedure with parameters */
{
    int product;                /* LOCAL memory declaration */
    if(a<max_number && b<max_number){ /* using GLOBAL memory */
        product=a*b;
        return(product);
    }
    else {
        whatever ...;
    }
}
```

8.3.2 Windowing and overlapping registers

The instruction length in the Berkeley RISC machine is 32 bits, but the opcode only took up 7 bits. A further 15 bits are taken are taken up by three sections each of 5 bits length. Each section is used to provide immediate control of three-fold operations on up to 32 registers.

For example, the instructions might look like

`ADD R3,R12,R25` which means `R25 <- R3 PLUS R12`

But the cpu has 138 registers on it, and 32 is obviously less than 138! How are the other registers used?

The answer is that *each active subroutine* has a *share* of the registers. But immediately you notice that $138/32 < 8$, where 8 is the number of levels of subroutine nesting catered for in the Berkely RISC.

The reason for this puzzle is the way register memory is shared by overlapping, which is illustrated in Figure 8.1. Global memory is shared by all, then parameter memory is shared with either the calling routine (parent) or called subroutine (child).

Thus, referring to the figure, the instruction

```
ADD R3,R12,R25
```

sets the contents of register R25 equal to the sum of the contents of R3 and R12, where R25 is in the private local memory of the current subroutine (the one pointed to by the window pointer), R12 is the parameter space shared with its parent, and R3 is part of global memory.

To connect this to the high level code

`ADD global,parentshared,local` or `local <-- global PLUS parentshared`

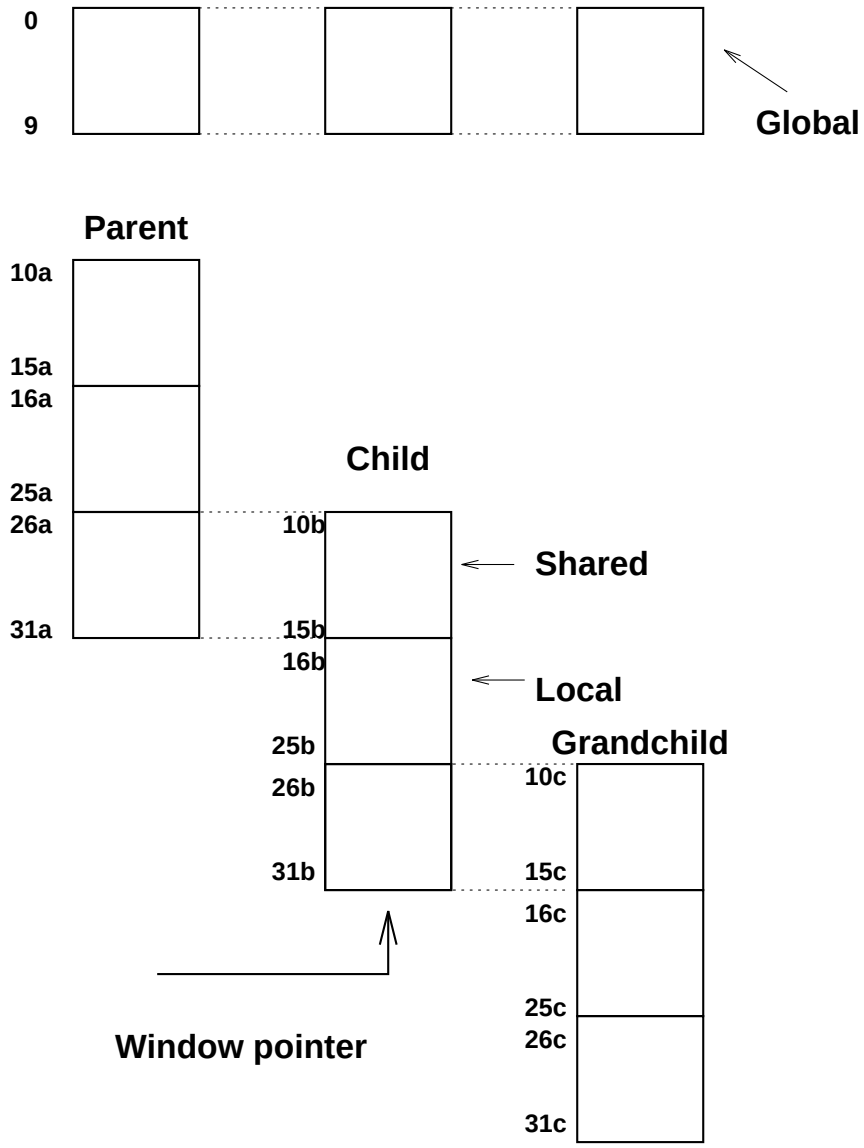


Figure 8.1: Overlapping register windows. Note that 6, 10, and 6 registers are parent-shared, local and child-shared respectively.

would have arisen if (for no good reason) we had written the following in the subroutine product()

```
product = max_number + a;
```

When a subroutine is called, the window pointer is incremented, and the **PC** stored in one of the local registers in the new window. It is thus available for the return from subroutine. Notice therefore that a subroutine call need not use the memory at all, and the stack is not required.

8.3.3 What happens when the registers run out?

The machine does not stop! It uses memory, but not as you might expect. Instead it moves the highest level out, so that the average level of nesting moves up one. (That is, in the 8 levels rather than Main, Subr1, ... Subr 7, we hold Subr1, ..., Subr7, Subr8.)

Again this model is based on the sound observation that the rate of change of nesting level is usually limited (effectively after smoothing by the 8-wide buffer). Once we have shifted the window set (window overflow) it is possible for window underflow to occur. The reverse procedure is followed to restore subroutines.

For example, suppose the depth of subroutine called followed this pattern

Main 1 2 3 4 5 6 7 **8** 7 8 **9** 8 7 8 9 8 7 6 5 6 4 3 2 **1 Main**

This would require only two shifts to memory, at the time when the first 8 appears, and when the first 9 appears. 9 and 8 would not be shifted out until 1 and Main re-occured.

8.3.4 The analogy with cache memory etc

It is quite obvious that we have in one sense extended the idea of memory hierarchies down a further level, by giving the cpu just enough local memory to deal with a large fraction of eventualities. Again, the strategy works because computation is localized.

This is worth thinking about just a little more. We saw that compilation produces localized code *within a subroutine*, but that different subroutines would be separated. Using more and more subroutines in your program actually *fragments the code*, introducing more jumps. There are a few ways round this. Some compilers replace the subroutine with “in-line” code, and some allow the declaration of macros that look like subroutines, but generate in-line code. The RISC machine’s overlapping registers are doing much the same thing. The are re-localizing the code for you, bringing together the fragments into on-chip registers.

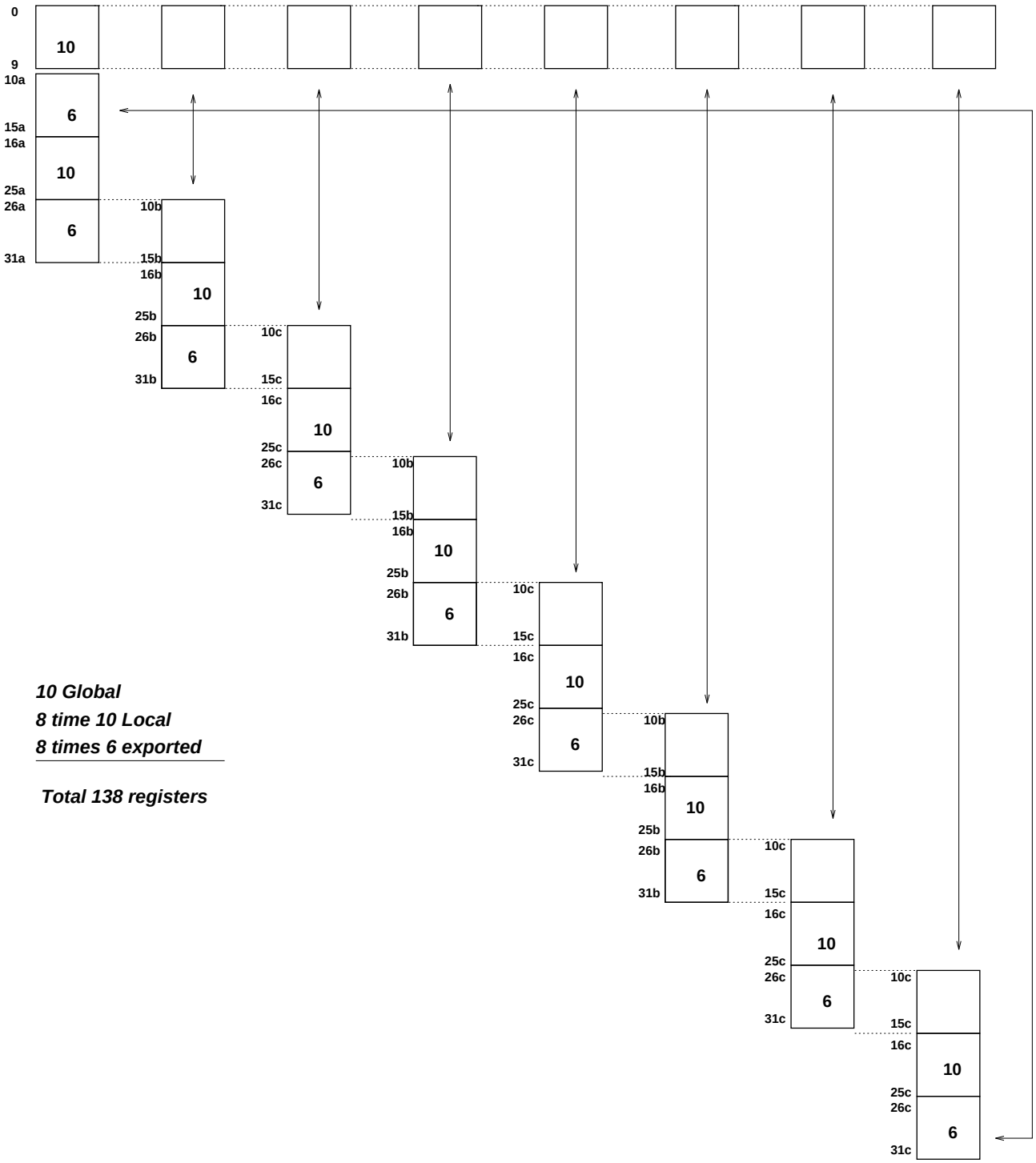


Figure 8.2: The entire register ram. Notice that the ram forms a ring buffer.

8.4 Pipelining

We have tended split ad instruction into

1. Instruction Fetch; followed by
2. Instruction Execute.

However, we could subdivide a typical execute phase (say of ADD x) into 3 parts, giving 4 distinct stages:

1. IF: Instruction Fetch and decode (as usual)
2. OF: Operand Fetch. Get any $\mathbf{M}\langle\mathbf{x}\rangle$ required, (either from memory, or for on-chip register).
3. EX: Do the addition operation
4. ST: Re-store the result (in any on-chip register, or in memory).

Whereas in a CISC machine, decoding an instruction may take some long and variable time (they use variable length opcodes, and there are a lot of them — you will see this on the lab course) because a RISC machine uses fixed length instructions and decoding is short. Let us suppose that each of the four phases can be made to happen within the same time.

Figure 8.3 shows the sequence in a simple machine — the instructions are brought in one after the other.

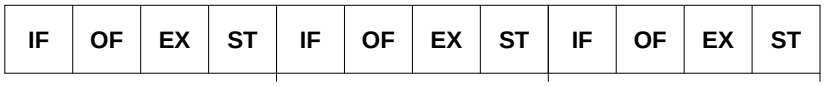


Figure 8.3: Strictly sequential instructions

It is obvious that in terms of utilization of resources this is wasteful. The logic to do instruction fetch is used for only 25% of the time. We could run four times as fast if it were possible to overlap instructions in the way shown in Figure 8.4.

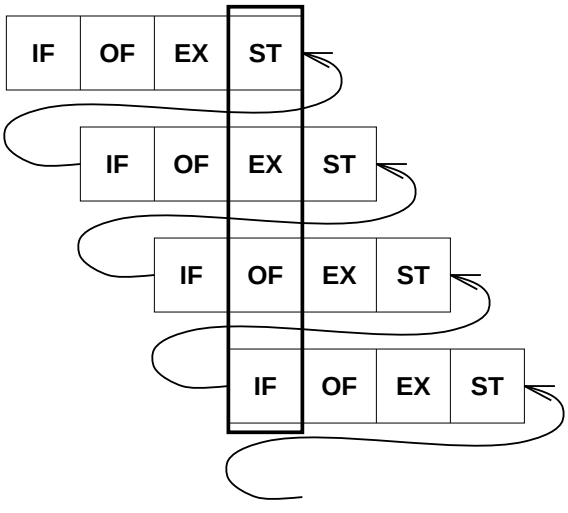


Figure 8.4: Instruction pipelining

How is this possible? It is possible because instructions follow one after the other in memory. The pipeline is apparent in the figure. The top instruction is in the STore phase, while the one after is in the EXecute phase, and so on. After each phase, the pipe is moved on four phases.

However, we must be careful about Figure 8.4. You might start thinking that, for example, the execute part of the instruction was somehow performed *before* the instruction fetch was completed. That would defy causality. So the pipe is not full — all the hashed "compartments" to the right of the vertical processing block must be "empty" — that is, that part has not been done yet. When is the process running at maximum efficiency? When all the compartments are "full" by the time they emerge at the top from the last part of the processor.

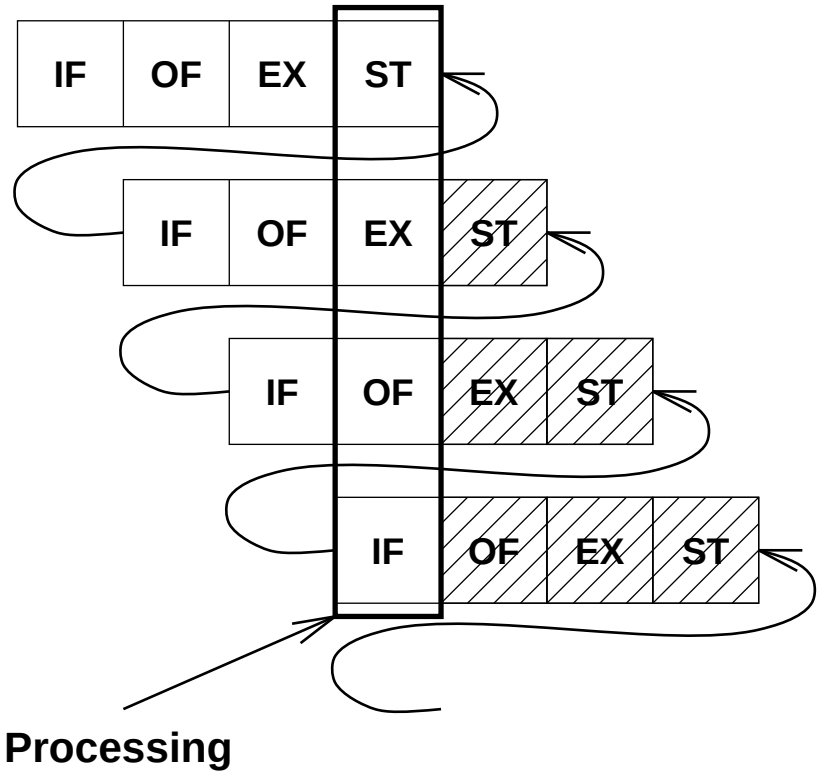


Figure 8.5: Causality: the hash parts have not been completed.

8.5 Pipeline Bubbles

The difficulty is that executing instruction after instruction in this interleaved fashion cannot be maintained. Suppose there is a branch or jump which skips out a couple instructions (Figure 8.6). The instruction fetch and Operand fetch parts of the next two instructions are wasted effort — an this introduces a "bubble" into the pipeline (Figure 8.7).

Let us be careful about how long the bubble is. Although 2 instructions are wasted, we do NOT lose $2 \times 4 = 8$ phases. Nor do we waste IF+OF+IF = 3 phases.

Only 2 phases are wasted. The best way to see this is to think how many units currently in the "processor box' are useless.

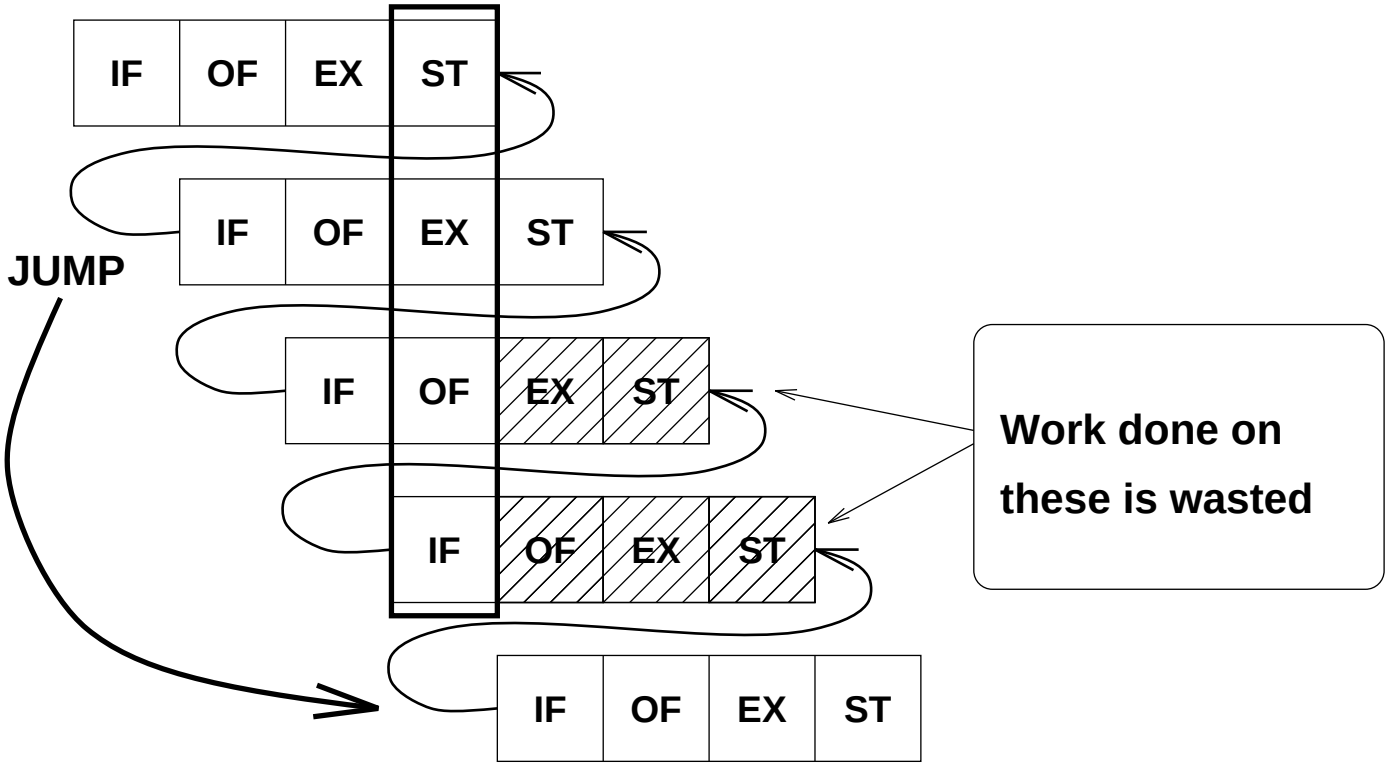


Figure 8.6: A jump

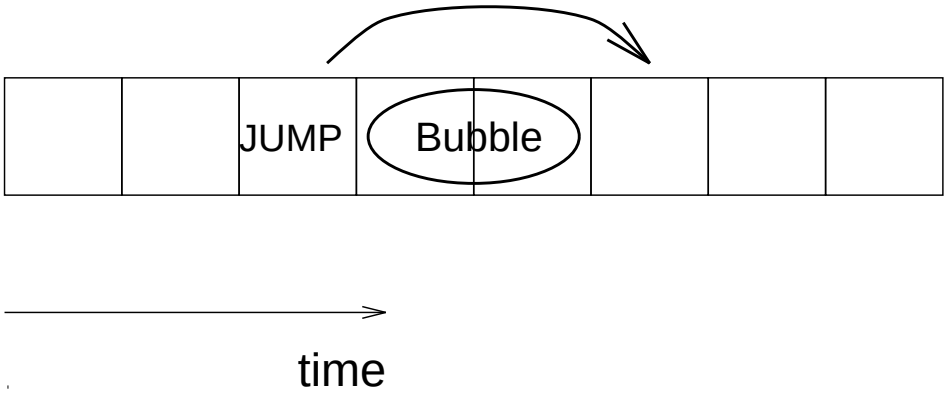


Figure 8.7: A bubble which lasts two phases.

8.6 Removing bubbles using delayed branching

Consider the following piece of high level code

```
i = 3;
j = 4;
k = 5;
if ( i == 3) {
  statementsA...
} else {
  statementsB...
}
etc ...
```

This is just the sort of code that will introduce a jump and hence a bubble in the pipeline.

The conventional code might look something like:

```
MOVE #3,ma_i
MOVE #4,ma_j
CMP #3,ma_i
BNE statementsB
etc ...
```

Notice however that after the first statement, `i=3`, one is committed to jumping, *no matter what the value of* `j`.

RISC compilers exploit this to reduce the sizes of bubbles by introducing "delayed branching". After the branch instruction the compiler trys to place an instruction – or even better here, two instructions — whose execution does NOT depend on the condition in any way. (Such non-dependent instructions can come from before or after the branch in the high-level code.) Then, rather than throwing away the two statements in the pipeline after the branch, it allows then through, and then branches, as shown in (Figure 8.8.

```
MOVE #3,ma_i
CMP #3,ma_i
DELAYEDBNE statementsB
MOVE #4,ma_j
etc ...
```

If the compiler could find only one non-dependent instruction, the bubble is reduced in size to one phase.

(Don't be confused by Clements pages 282 onwards. He appears to get rid of all the bubble in a four phase pipeline by introducing just one independent instruction. However you will notice that whereas his Figure 5.29 has four phases, his Figure 5.30 has mysteriously reduced to only 3 phases. In a 3 phase pipe, one instruction would do it.)

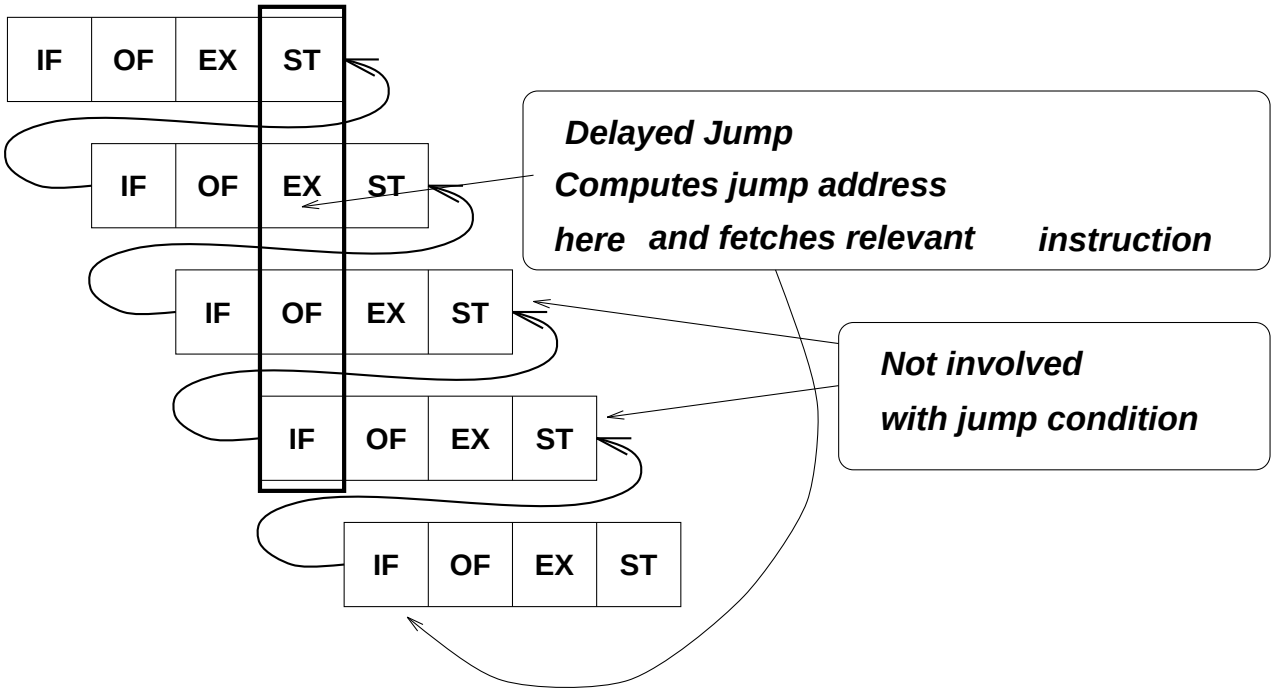


Figure 8.8: Delayed branching.

8.7 Pipelines and data dependency

We have already seen that care has to be taken with consequential instructions if the pipeline is to work flat out. Consider the following fragment of high level code

```
x = (A+B) .AND. (A+B-C)
```

which might get compiled as

```
ADD A,B,TEMP              TEMP1 <- A     + B
SUB TEMP1,C,TEMP2         TEMP2 <- TEMP1 - C
AND TEMP1,TEMP2,X         X     <- TEMP1 .AND. TEMP2
```

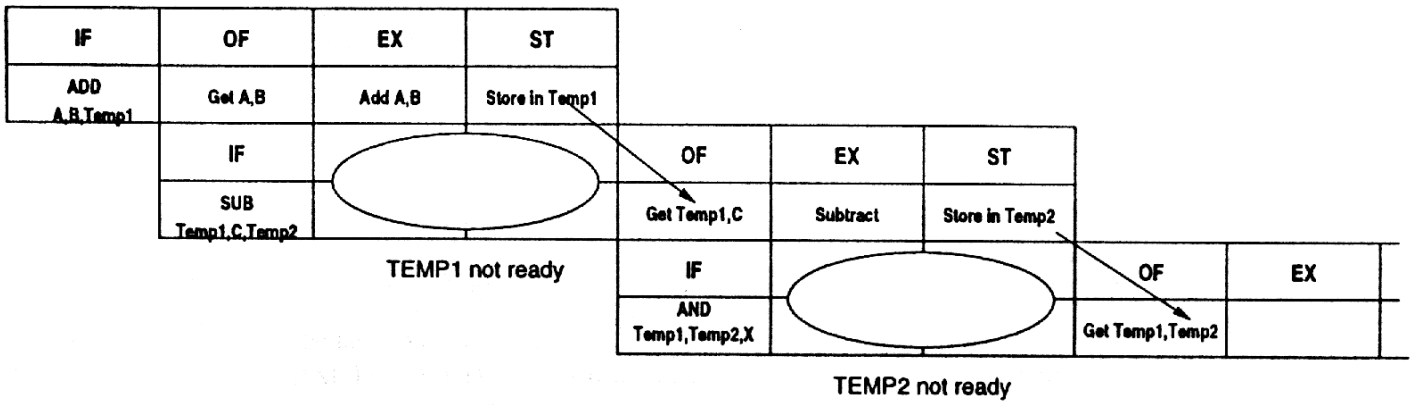


Figure 8.9: Bubbles arising from data dependency

The figure (redrawn from Clements) showed that two bubbles arise from this seemingly innocuous operation.

8.8 Internal Forwarding

There are two methods of alleviating this problem.

Both are illustrated in the following example, and following Clements we use a 3-phase pipeline where operand fetch and execute are rolled into just one time phase.

The data dependency problem is that line 2 depends on line 1, and line 3 depends on line 2.

```
1. ADD R1,R2,R3          R3 <- R1 + R2
2. ADD R3,R4,R7          R7 <- R3 + R4
3. ADD R7,R1,R8          R8 <- R1 + R7
```

- The first way round is interleaving. If the compiler can find a and independent instruction that needs to get done at a similar time.

  Notice that register R4 is involved in both, but only as source of of data, not as the recipient of data.

```
     1.  ADD R1,R2,R3          R3 <- R1 + R2
Insert.  ADD R4,R5,R6          R6 <- R4 + R5
     2.  ADD R3,R4,R7          R7 <- R3 + R4
     3.  ADD R7,R1,R8          R8 <- R1 + R7
```

- The second wheeze is to install an internal pathway that avoids the explicit need to store the data back into R7 between lines 2 and 3.

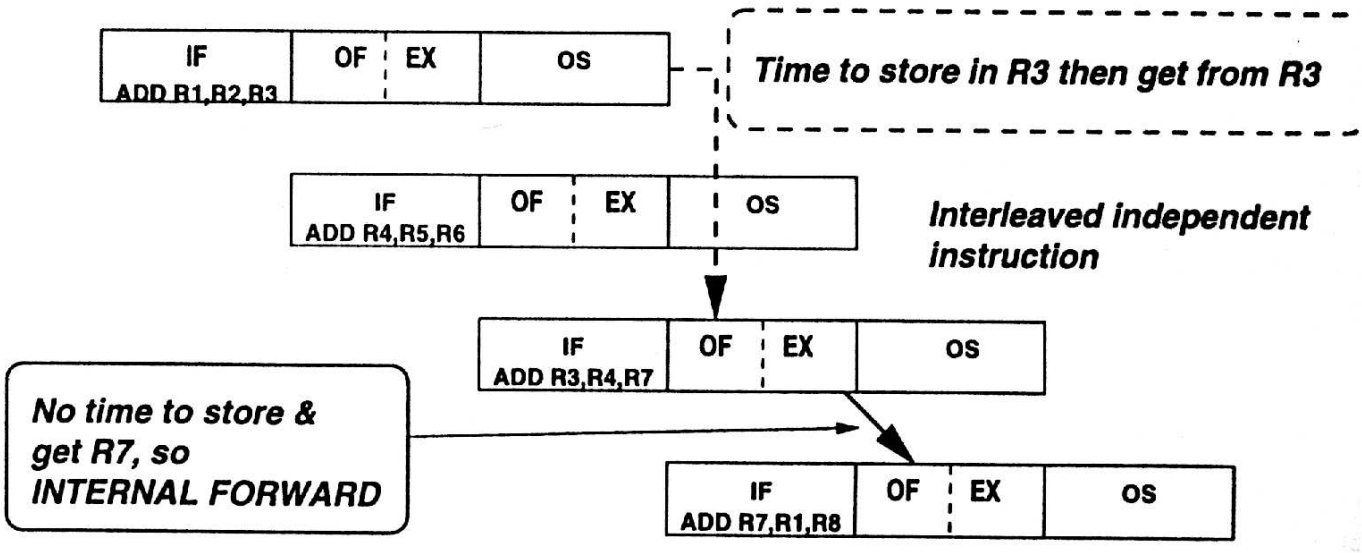


Figure 8.10: Internal forwarding in a 3-phase RISC machine